SURFACE RECONSTRUCTION FROM POINT CLOUDS TO POLYGONAL MESHES AND REMESHING

FILIPPOS ROSSIDIS

Convertion of physical objects to coarse quad meshes and G^1 B-Splines

Masters degree
School of Electrical & Computer Engineering
Aristotle University of Thessaloniki
Supervisors: Ioannis Emiris, Angelos Matzaflaris

September 2026 – Thessaloniki

Filippos Rossidis: *Surface reconstruction from point clouds to polygonal meshes and remeshing*, Convertion of physical objects to coarse quad meshes and G^1 B-Splines, Masters degree, © September 2026

This thesis was written using the work I did during my internship at *Institut national de recherche en informatique et en automatique (Inria)* at *Université Côte d'Azur* during the period of March-June 2026.

SUPERVISORS:

Ioannis Emiris	National and Kapodistrian University of Athens
Angelos Matzaflaris	Centre Inria d'Université Côte d'Azur
Bernard Mourrain	Centre Inria d'Université Côte d'Azur

DEFENCE COMMITTE:

Ioannis Emiris	National and Kapodistrian University of Athens
Traianos Yioultsis	Aristotle University of Thessaloniki
Nikolaos Kantartzis	Aristotle University of Thessaloniki

DEFENDED ON:

Friday, October 30, 2026 at Aristotle University of Thessaloniki.

LOCATION:

Sophia-Antipolis
Thessaloniki

Dedicated to the memory of my uncle, Kostas Kesinis, whom I've come to resemble ever more with the passing of time.

ABSTRACT

Surface reconstruction is the process of reconstructing a physical object into a digital surface representation. Usually a set of points is scanned from the physical object and a triangle mesh is created. This field is studied widely as it provides numeral applications. On the other hand, quadrilateral or quad meshes are dominant in the industry for representing surfaces due to certain topological criteria they can provide over triangle or other types of meshes. Even though automated algorithms for creating said quad meshes, either from a triangle mesh or directly from a point cloud have been researched over the last decades, the industry standard is still the time consuming process of modelling the quad meshes by hand, as they are of higher quality than the automatically generated ones.

Meanwhile, coarse or layout quad meshes have been introduced for applications where only the shape and topology of an object are of interest, and capturing the details is not important, as these meshes would require less space to store and less time to perform actions on. Coarse meshes are still in the infant stage of research, even though various different algorithms for deducing them exist. Often in these cases a parametrization of the surface, i.e. a definition of a two dimensional parameter space on it, is important to provide a link between the coarse mesh and the original detailed one, so that details can be mapped on it.

This thesis is part of the struggle to automatically generate quality coarse quad meshes from physical objects. A pipeline is developed that can generate a coarse quad mesh from a point cloud, using a triangle mesh as an intermediate step. A parametrization method is provided and data from the detailed model are projected on the coarse mesh. In the end a B-Spline surface can be defined that closely follows the physical object, as a test of the capabilities of our method.

Surface reconstruction to create the triangle mesh is bibliographically studied, as also existing quad meshing algorithms. The propagation of smooth cross fields is discussed and an open source quad mesher (Nico Pietroni's *quadwild*) is modified to produce coarse results, by altering the topological criteria that the mesh is encouraged to have. Our parametrization and data projection methods are studied. All the prerequisites, such as data structures and algorithms, that are needed in this pipeline are discussed.

Figures 7.2, 7.3 showcase this entire process.

ΠΕΡΙΛΗΨΗ

Ανακατασκευή επιφανειών είναι η διαδικασία μετατροπής ενός φυσικού αντικειμένου σε μια ψηφιακή αναπαράσταση επιφάνειας. Συνήθως ένα σύνολο σημείων σαρώνεται από το φυσικό αντικείμενο και δημιουργείται ένα τριγωνικό πλέγμα. Η διαδικασία αυτή εμφανίζεται συχνά στην έρευνα λόγω των πολλαπλών εφαρμογών της. Ταυτοχρόνως, τα τετραπλευρικά πλέγματα είναι κυρίαρχα στη βιομηχανία για την αναπαράσταση επιφανειών, εξάιτίας τοπολογικών ιδιοτήτων τους που τα καθιστούν βέλτιστα σε σχέση με τα τριγωνικά ή άλλα πλέγματα. Μέθοδοι για την αυτόματη δημιουργία τετραπλευρικών πλεγμάτων έχουν μελετηθεί αρκετά, είτε ξεκινώντας από τριγωνικό πλέγμα, είτε από νέφη σημείων, όμως η βιομηχανία ακόμη βασίζεται στην αργή διαδικασία της χειροκίνητης μοντελοποίησης τους από καλλιτέχνες ή μηχανικούς, διότι παράγει σχεδόν πάντα ανώτερης ποιότητας αποτέλεσμα.

Τα αραιά πλέγματα έχουν εισαχθεί στη βιβλιογραφία για εφαρμογές που ενδιαφέρονται μόνο για το σχήμα και την τοπολογία ενός αντικειμένου και όπου η λεπτομέρεια του δεν είναι χρήσιμη. Αυτά τα πλέγματα απαιτούν λιγότερο χώρο στην αποθήκευσή τους και λιγότερο χρόνο στην επεξεργασία τους. Όμως, παρότι υπάρχουν αλγόριθμοι δημιουργίας τους, τα αραιά πλέγματα βρίσκονται ακόμη σε πρώιμο στάδιο έρευνας. Επιπλέον, στις εφαρμογές όπου χρησιμοποιούνται, συχνά απαιτείται μια παραμετροποίηση της επιφάνειας, δηλαδή ο ορισμός ενός δισδιάστατου παραμετρικού χώρου πάνω της ώστε να παρέχει μια σύνδεση του αραιού πλέγματος με το αρχικό λεπτομερές. Έτσι μπορούν να αντιστοιχηθούν οι λεπτομέρειες μεταξύ τους.

Αυτή η διπλωματική αποτελεί μέρος της προσπάθειας προς την αυτόματη δημιουργία ποιοτικών αραιών τετραπλευρικών πλεγμάτων από φυσικά αντικείμενα. Δημιουργούμε μια μεθοδολογία που παράγει ένα αραιό τετραπλευρικό πλέγμα από ένα νέφος σημείων, με το ενδιάμεσο βήμα ενός τριγωνικού πλέγματος. Παρέχεται μια μέθοδος παραμετροποίησης και οι πληροφορίες από το λεπτομερές τριγωνικό πλέγμα προβάλλονται στο αραιό. Στο τέλος μια B-Spline επιφάνεια μπορεί να οριστεί χρησιμοποιώντας το, ώστε να ακολουθεί πιστά το φυσικό αντικείμενο.

Η ανακατασκευή επιφανειών σε τριγωνικά και τετραπλευρικά πλέγματα μελετάται βιβλιογραφικά. Η διάδοση διασταυρούμενων πεδίων συζητείται και ένας αλγόριθμος παραγωγής τετραπλευρικών πλεγμάτων ανοιχτού κώδικα (*quadwild* του Nico Pietroni) τροποποιείται ώστε να παράγει αραιά πλέγματα, μεταβάλλοντας τα τοπολογικά κριτήρια που ενθαρρύνονται. Η μέθοδος μας της παραμετροποίησης και προβολής δεδομένων μελετάται. Όλα τα προαπαιτούμενα, όπως δομές δεδομένων και αλγόριθμοι, που χρησιμοποιούνται σε αυτή την διπλωματική παρέχονται.

Στα γραφήματα 7.2, 7.3 σκιαγραφείται ολόκληρη η μεθοδολογία.

ACKNOWLEDGMENTS

First and foremost, I would like to thank my supervisor Ioannis Emiris for helping me create these connections that proved so fruitful:

I am very grateful of the AROMATH team for integrating me this quickly in their environment and of Bernard Mourrain for his interest, his effective suggestions and most certainly for the altitude seminars.

I am grateful also of Panagiotis Rigas and Christos Konaxis who have given me support during the start and the end of this thesis, in undertaking to guide my research.

Most importantly I would like to thank Angelos Matzaflaris for his help, accommodation, guidance and trust in supervising me and for helping me feel at home in France.

And finally, I cannot but thank my parents, without whose support none of this would be possible.

I will also use this spotlight to greet my colleagues and friends from around the world, who in one way or another helped keeping the moral high during the development of this work: Udesh, Alberto, Silvia, Dimitris, Matteo, Michelangelo, Oriol, Henri, Jana, Giulia, Eliot, Axel, Anh, Kelt, Marysia — in order of desk proximity…

Michelangelo, thank you for your hospitality, I hope I can repay it some day.

CONTENTS

I INTRODUCTION

1 INTRODUCTION 2
- 1.1 Definitions 2
- 1.2 Topology in meshes 3
  - 1.2.1 What makes a good quad topology 4
- 1.3 Why a coarse mesh? 5
- 1.4 Implementation 6
- 1.5 Note to the reader 7

2 PRELIMINARIES 8
- 2.1 Data structures 8
  - 2.1.1 Doubly-connected edge lists 8
  - 2.1.2 Octrees 12
- 2.2 Catmull-Clark subdivision scheme 15
  - 2.2.1 Boundaries and sharp edges support 16

II PIPELINE

3 SURFACE RECONSTRUCTION 19
- 3.1 State of the art 19
- 3.2 Poisson Reconstruction 20
  - 3.2.1 The Finite Element Method 21
  - 3.2.2 Galerkin formulation 22
  - 3.2.3 Adaptive meshing - Basis function refinement 25
  - 3.2.4 The Poisson algorithm 28
- 3.3 Making the triangle mesh 32
- 3.4 Results 33

4 TRIANGLE TO COARSE QUAD MESH 36
- 4.1 Fine quad mesh using quadwild 36
  - 4.1.1 Cross field 37
  - 4.1.2 Tracing lines 44
  - 4.1.3 Creating the patches 45
  - 4.1.4 Optimizer 47
- 4.2 Messing with the optimizer to get a coarse result 48
- 4.3 Results 49
- 4.4 Finding the sharps on the quad mesh 49
- 4.5 Critique 51

5 PARAMETRIZATION / PROJECTION 53
- 5.1 Why parametrize 53
- 5.2 General view 53
- 5.3 Defining the u-v space 54
- 5.4 Creating the grid points 55
- 5.5 Projecting a point on a mesh accurately 57
  - 5.5.1 Triangle projection - T-neighborhoods 57

5.5.2 Projecting a point on a triangle 60
5.5.3 Projecting a point on an edge 61
5.5.4 Combined projection 62
5.6 Sharp retention 62
5.6.1 Projecting on sharp edges 63
5.6.2 Fixing degeneracies 63
6 FITTING OF A B-SPLINE 66
6.1 What is a spline 66
6.2 Basis functions 66
6.3 Different levels of continuity 68
6.4 Fitting a point cloud on a quad mesh 69
6.5 Results 70

III FULL SHOWCASE
7 FULL SHOWCASE 73

IV APPENDIX
A APPENDIX 78
A.1 Convolution 78

BIBLIOGRAPHY 80

LIST OF FIGURES

Figure 1.1 Vertex/edge examples. 3
Figure 1.2 Topologically equivalent meshes. 4
Figure 1.3 Edge flow follows natural features. 4
Figure 1.4 Fine versus coarse quad mesh. 6
Figure 2.1 Dihedral angle. 9
Figure 2.2 A doubly-connected edge list example. 10
Figure 2.3 Vertex neighbors and counter-clockwise rotation of a half-edge. 11
Figure 2.5 Octree example. 12
Figure 2.4 Subdivision of octrees. 13
Figure 2.6 Visualization of the trilinear interpolation. 14
Figure 2.7 Sequential steps of the triangular midpoint subdivision scheme. 15
Figure 2.8 Catmull-Clark showcase. 15
Figure 2.9 Catmull-Clark with sharps. 17
Figure 3.1 Showcase of the indicator function. 21
Figure 3.2 A triangle mesh and a linear basis function. 22
Figure 3.3 T-vertices are introduced in adaptive mesh refinement. 26
Figure 3.4 Poisson reconstruction of organic model 1 (kitten). 33
Figure 3.5 Poisson reconstruction of organic model 2 (fertility), depth comparison. 34
Figure 3.6 Poisson reconstruction of organic model 3 (dragon), depth comparison. 34
Figure 3.7 35
Figure 3.8 Poisson reconstruction of a noisy sphere point cloud. 35
Figure 4.1 Quadwild steps overview. 36
Figure 4.2 The remeshing step. 37
Figure 4.3 N-RoSy vector examples. 37
Figure 4.4 Cross field singularities / singularities to evs correspondence. 39
Figure 4.5 LC-parallel vectors. 40
Figure 4.6 Triangle splitting before cross field propagation. 41
Figure 4.7 Cross field calculation, first mechanical example. 43
Figure 4.8 Cross field calculation, second mechanical example. 43
Figure 4.9 Calculated cross field on a torus model. 43
Figure 4.10 Tracing paths conforming to a cross field. 44
Figure 4.11 Jagged paths. 45
Figure 4.12 The two patch quadrangulation methods. 46
Figure 4.13 Straightening the borders of the patches. 46
Figure 4.14 The reconstruction of a first mechanical mesh. 49
Figure 4.15 The reconstruction of a second mechanical mesh. 50
Figure 4.16 The Stanford bunny reconstruction. 50
Figure 4.17 Opera of Sydney reconstruction. 50

Figure 4.18 A sharp and an almost sharp edge. 51
Figure 4.19 Almost sharp edges present a problem. 52
Figure 4.20 A detailed mesh will not provide a very coarse result. 52
Figure 5.1 A u-v parameter space defined on a face of a quad mesh. 53
Figure 5.2 Grid points projection. 55
Figure 5.3 u-v space definition. 55
Figure 5.4 Different options for the quad-triangle projection. 56
Figure 5.5 A face after 3 Catmull-Clark subdivisions. 56
Figure 5.6 Updating the u-v values after one CC subdivision. 57
Figure 5.7 T_1 and T_2 neighborhoods. 58
Figure 5.8 Modified T_2 neighborhood stopping at sharp vertices. 59
Figure 5.9 Vector projection. 60
Figure 5.10 Projection of a point on the surface of a triangle. 60
Figure 5.11 Projection of a point on an edge. 61
Figure 5.12 Examples of when projecting on edges or keeping the closest vertex will be necessary. 62
Figure 5.13 The grid points on sharp edges. 63
Figure 5.14 Fixing wrong side projection. 64
Figure 6.1 9th degree Bernstein polynomials / a Bézier curve example. 67
Figure 6.2 Examples of a B-spline curve and a Bézier surface. 68
Figure 6.3 Geometric continuity examples. 69
Figure 6.4 Fitting a spline on the venus sculpture model. 70
Figure 6.5 A mechanical spline example (BendTube). 71
Figure 6.6 Model of the author's bike. 71
Figure 7.1 The Poisson reconstruction smoothens the sharp edges. 73
Figure 7.2 The Stanford bunny full pipeline. 74
Figure 7.3 Fandisk model pipeline from triangle mesh. 75
Figure 7.4 Coarse quad mesh collage. 76
Figure A.1 Visualization of the convolution between two square functions. 78
Figure A.2 Using convolution as a smoothing filter. 78
Figure A.3 Repeated convolutions of a square pulse approximate a gaussian. 79

Part I

INTRODUCTION

1

INTRODUCTION

Quadrilateral (quad) meshes are one of the dominant forms of representing surfaces. They are used almost exclusively in computer graphics and are also widely used in design, simulation etc [8]. Historically triangle meshes have been studied extensively, however, since quad meshes provided certain advantages – as discussed below – the literature has started to shift towards them.

Meshes are mostly used to capture the surface of an object, including its details, therefore *fine meshes*, i.e. those composed of multiple small elements are studied the most. There is, however, the notion of a *coarse* mesh, that is one that uses larger elements, to capture just the shape of an object, ignoring its detail. Coarse quad meshes have been introduced in modern literature – mostly in the form of layouts, as in [39], [53] – but are generally not studied enough. Additionally, workflows as in [40] that we will be using, use a quad mesh of a model to define a detailed spline that closely follows it.

Traditionally, quad meshes have been modeled by hand by artists or engineers, as they must satisfy certain topological criteria to present good quality. Algorithms to automatically generate a quad mesh (from a point cloud or a triangle mesh or a set of images etc..) have been studied in the last decades, but the results still fall short in comparison to meshes modeled by hand, and such algorithms have yet to become the industry standard [8]. This thesis contributes, in part, to addressing this challenge.

Our main contribution is the automated conversion of a triangle mesh to a coarse quad mesh and the parametrization on u-v coordinates of points of the original triangle mesh on the faces of the coarse quad. This is part of a larger pipeline that uses established algorithms to convert a point cloud to a G^1 surface B-Spline. The point cloud is first converted to a triangle mesh, using the Poisson algorithm [35], then our method - which is a modification of the algorithm of [46] - creates the coarse quad mesh, the parametrization follows, and in the end we can create a B-Spline using [40] that closely follows the original point cloud.

1.1 DEFINITIONS

In this section certain useful concepts are defined. For a more complete list see [8].

A mesh is composed of *vertices*, *edges* and *facets* or *faces*. In computer graphics, a vertex is a point in (our case) 3D space, an edge is a straight line segment connecting two vertices and a face is a polygon bounded by cycles of edges. We will symbolize a mesh as $M = \{V, E, F\}$, where V is the set of vertices, E the set of edges and F the set of faces. The number of vertices, edges and faces are, then, $|V|, |E|, |F|$.

In the broader terminology of computational geometry, the term face may also refer to lower-dimensional components, such as vertices and edges, while a facet denotes a face of dimension (n-1) of an (n)-dimensional polytope.

The *valence* of a vertex is the number of its incident edges, while the valence of a face is the number of vertices that bound it.

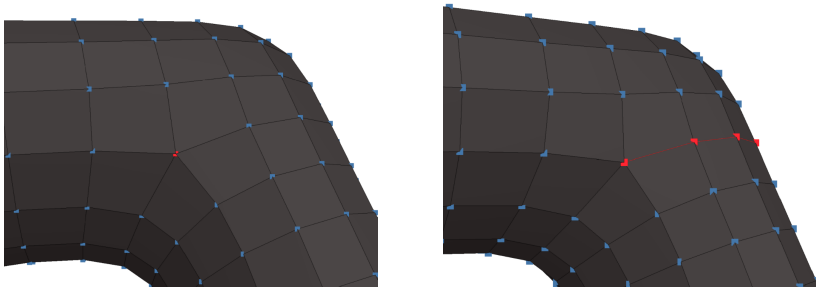
For our purposes, the mesh should be a *2-manifold*, as explained in [8]. Informally, no self intersections are allowed. For a formal definition, see [21]. If an edge has

one incident face it is called a *boundary edge*, otherwise it is called an *internal edge*. If all edges of a mesh are internal then the mesh is called *watertight*.

A *regular* vertex is one that has valence 4, otherwise it's called *irregular* or *extraordinary* (*ev* for short). For regular vertices, a *traversal* is defined as a path of adjacent edges such that no two successive edges are incident to the same face. When a traversal reaches an irregular vertex, it has to stop. On figure 1.1 an irregular vertex and edge traversals can be seen.

A *triangle mesh* is one that consists of only triangles (every face has valence 3), a *quad mesh* is one the consists only of *quadrilaterals* or *quads* (every face has valence 4) and a *quad-dominant mesh* is one where the majority, but not all, of the faces are quads.

Lastly, as stated in [8], we call *T-mesh* a mesh where an edge e of a face f can coincide with a chain of edges of one or more faces glued on to f along e. *T-junction* or *T-vertex* is an internal vertex of such a chain, as seen in figure 3.3.



(a) Vertices are colored as blue and one as red, edges as black and faces as yellow. All blue vertices are regular while the red one is an ev with valence 5.

(b) A traversal is colored in red. Notice how it stops on the ev.

Figure 1.1: Vertex/edge examples.

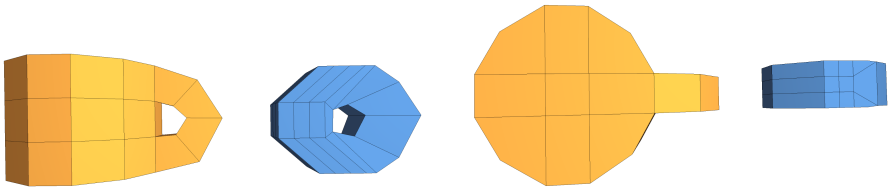
1.2 TOPOLOGY IN MESHES

In this thesis, we are not concerned in formally defining topological equivalencies,but the word *topology* will often appear when talking about meshes and mesh quality. There is a clarification to be made here as topology can refer to different concepts: In formal mathematical approach topology is the study of equivalencies of objects, say for example comparing the amount of holes two different objects have. In computer graphics, topology refers to a related but different concept, the arrangement of polygons, how they are connected, the valence of the vertices etc. The position of the vertices in this case should not matter for two meshes to have the same topology, only the relations between them. An example can be seen in figure 1.2. Informally, in this case, two meshes with the same vertex connections are *topologically equivalent*, which is a concept similar to topological equivalencies in the formal mathematical case.

In this thesis, we will mainly use the second meaning of the word. For example, in Section 1.3, when talking about *capturing the topology of a model*, we refer to placing

the regular vertices, evs and faces in such a way that the mesh can be as close to the model as possible.

For a discussion on mathematical topology the reader is redirected to [21].



(a) Side view. (b) Top view.

Figure 1.2: The two meshes are topologically equivalent: they have the same vertex connections. Only vertex translations have been made.

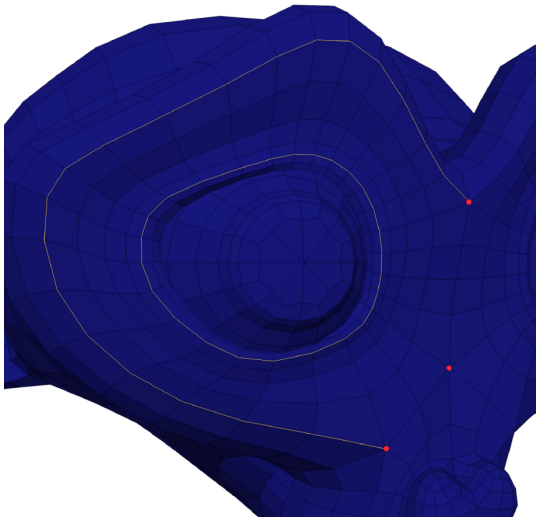


Figure 1.3: Highlighted traversals to show that the edges flow around the eye so that they capture (are parallel with) the eyebrow and eyelid. Notice that the irregular vertices, colored in red, are needed to create this edge flow. [Model: Suzanne, from blender]

1.2.1 *What makes a good quad topology*

The quality of a quad mesh in general can be assessed on its effectiveness on certain factors. Some of them are, as described in [20], the following:

SURFACE APPROXIMATION The approximation error should be as small as possible. It is typically measured as the Hausdorff distance of the quad mesh and the input surface (be it in the form of a function, point cloud or a different mesh).

TARGET EDGE LENGTH A specified length that the lengths of the mesh's edges should be as close to as possible. This helps the user control the density of the meshing.

ELEMENT QUALITY It is usually preferable to have faces as close to rectangular as possible, i.e. the the inner angles should not deviate much from 90°.

EDGE FLOW Edge flow refers to the traversals of adjacent edges, as well as the orientation of each individual edge on the mesh. A "good" edge flow is not easily defined, however artists and modelers pay a great deal of attention on it (capturing natural flows of the objects they are modelling, or possible directions of deformation during animation or soft body simulation).

A reasonable placement for edge flows are the principal curvatures of the surface.

IRREGULAR VERTICES Regular vertices are preferred in general, because they behave well with edge traversals, as well as with subdivision surfaces - a widely used technique in computer graphics where the mesh is refined and smoothed to gain resolution (see section 2.2). Irregular vertices can create hard lines when used in a subdivision surface. However, they are generally needed, as a mesh consisting of only regular vertices is suited for disk or toroidal topology only [8], and so is not versatile. Irregular vertices can be necessary for the topology of the desired surface (see the colored vertex in figure 1.1a) or for the modelling of the desired edge flow (see figure 1.3). It is still, however, preferred to have as few irregular vertices as possible, as they lead to artifacts.

In different applications these criteria may change. For example one may want larger detail in a certain region of the mesh so they can ignore the target edge length for that region. Or if one is not using the model for simulation or animation, they may not be interested in having square faces, they may prefer elongated rectangles to save on detail.

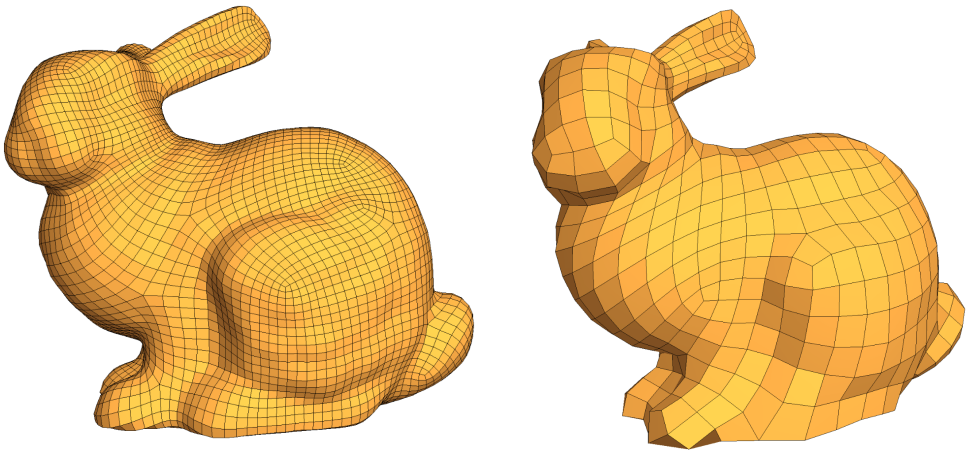
In our case, we are interested in making the mesh as *coarse* as possible, i.e. having the least amount of faces, while not losing the topological information or quality. In this case we want to abide to the above criteria, with a target edge being as large as possible without damaging the rest. An example of good quality fine and coarse meshes for comparison can be seen in figure 1.4, of models that have been generated from our code.

1.3 WHY A COARSE MESH?

A coarse mesh can be more useful than a fine one in certain occasions:

A. If a coarse mesh manages to capture well the information of a model, then with refinement (as in 2.2) it can approach the fine mesh. In this case it is less computationally and spatially expensive to store and perform actions on the coarse mesh rather than a fine one, and refine as a final stage.

B. Certain applications are only interested in the *topology* of a model and not the actual positions of its vertices, neither capturing details. A very good example is the fitting algorithm by Marsala et al. [40] which uses the quad mesh to generate a set of basis functions that will be used to fit a B-Spline on point data. We will use this in Chapter 6 to tests our meshes.



(a) (b)

Figure 1.4: Generated meshes from a point cloud of the Stanford bunny. Left: a fine mesh using the quadwild [46] code, right: a coarse mesh using our edited version.

We are thus interested, in this thesis, in capturing just the topology of a model without additional detail.

1.4 IMPLEMENTATION

Our pipeline in its entirety takes a point cloud as input and creates a coarse quad mesh and later a spline surface that closely resembles the original point cloud. For the implementation we have used:

1. The program MeshLab [13] for the Poisson reconstruction method 3.2, that produces a triangle mesh from a point cloud.
2. The code of Nico Pietroni et al. *quadwild* [46], that produces a quad mesh from a triangle mesh. The code was modified by us to generate a coarse rather than fine quad mesh, and to provide sharp edges support.
3. The G+Smo package in C++ for working with meshes and splines: we wrote programs to create the parametrization 5 on the quad mesh and the projection 5.5 of the parameters on the accurate triangle mesh, to prepare for the fitting. The fitting is also a script provided by the G+Smo package.

In the end we have a spline surface that closely follows the original point cloud, and can be used later for simulation.

The whole pipeline can be found on the github page:

`https://github.com/frilip/Pipeline-from-Point-Clouds-to-Coarse-Quad-Meshes-and-B-Splines`

Most meshes in this text are plotted using the CGAL library in C++, but some also using blender. The cross fields and splines and some demonstrations on meshes are plot with Paraview. The small point clouds have been visualized with CGAL and the large ones with Paraview.

Some of the models used are modelled by the author by hand and the others are taken from the below libraries:

1. Stanford Computer Graphics Laboratory, cited as Stanford.
2. Thingi10K, citied as Thingi10K.
3. The dataset composet of 300 meshes , citied as 300.
4. The dataset provided by CGAL, citied as CGAL.
5. The libigl-tutorial dataset, cited as libigl.

Whenever a model is used for the first time, it's source will be cited.

1.5 NOTE TO THE READER

The chapters 3–6 of this thesis represent a different stage of the pipeline. Since they are independent they can be read in any order, or according to ones interests, any one can be skipped without loosing much information for the next. They are presented in the order of operations in the pipeline.

2

PRELIMINARIES

This section is dedicated to introducing some preliminary concepts that will be used throughout the thesis. Doubly-connected edge lists are widely used in most mesh operations and are essential for the processes in chapters 4 and 5. The Catmull-Clark subdivision scheme is discussed in various parts of the text and is used in the process of generating the grid points in section 5.4.

Octrees are used once when they are needed (section 3.2) and can be skipped for a quick reading of the text.

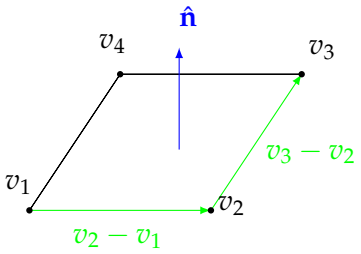
2.1 DATA STRUCTURES

2.1.1 *Doubly-connected edge lists*

When working with meshes on the computer, the first problem that arises is how to store them. Remember that a mesh is a collection of vertices, edges connecting two vertices and faces that are defined by a cycle of edges. A first idea might then be to distinguish vertices, edges and faces by giving an index to each, store a *vertex catalog* that will hold for every vertex index its coordinates in 3D space, an *edge catalog* storing for every edge index the indices of its two vertices and a *face catalog* storing for every face index the indices of the edges that define it.

It can be a sensible choice for very simple operations, but problems with its algorithmic efficiency arise very quickly:

Let's try to solve a simple problem that we will use later. Suppose that you want to calculate the dihedral angle of every edge, that is the angle between its two incident faces. It can be seen in figure 2.1. The vectors $\hat{n_1}$, $\hat{n_2}$ represent the normalized *normal vectors* of the faces f_1, f_2. The normal vector of a face is the vector that is perpendicular to the plain it defines. Since four points are not necessarily in the same plain, a better definition is the following (as seen in the inline):



$$\mathbf{n} = (v_2 - v_1) \times (v3 - v2), \qquad \hat{\mathbf{n}} = \frac{\mathbf{n}}{\|\mathbf{n}\|} \tag{2.1}$$

Thus for an edge e, with incident faces f_1, f_2, its dihedral angle α will be the angle between the normals of the two faces. The angle between two vectors can be calculated easily in radians with:

$$\alpha = \arccos\left(\frac{\mathbf{n_1} \cdot \mathbf{n_2}}{\|\mathbf{n_1}\|\|\mathbf{n_2}\|}\right) \tag{2.2}$$

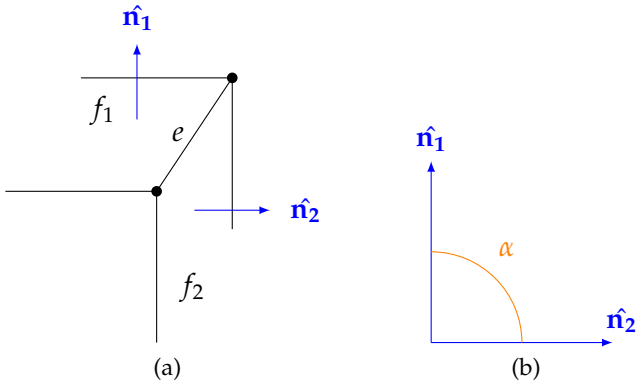


Figure 2.1: Dihedral angle.

Now, trying to calculate the dihedral angle using the vertex, edge and face catalogs we run into a problem: the normals are easy to calculate, but we don't know for a given edge its incident faces, the algorithm then would be to search every combination of faces, read their vertices from the catalog, see if they have a common pair and if they do, we now know that they are incident: we can calculate their normals and the dihedral angle.

So far, the algorithm already has $O(|F|^2)$ time complexity (assuming a constant number of vertices per face, which of course holds for quad meshes), but there is a second more subtle problem. We have to know the order with which the vertices appear in the face to compare the pairs accurately. If this information is not provided we will need a lot of unnecessary checks with the edges that will make the time complexity unhandlable.

To address these issues and many more, the *doubly-connected edge list* (*DCEL*) data structure is used almost exclusively to store mesh structures. It was first described by Muller and Preparata [41] to solve an intersection of polyhedra problem. A textbook description can be found in [3].

A general implementation of the doubly-connected edge list is discussed below, an amount of deviations exist for different applications. For our case, since we are only interested in triangle and quad meshes, cases where a face can have two boundaries (inside and outside) are not allowed, so we will not deal with them.

As we discussed above, we want to be able to find all edges of a face with the correct order, the idea then is to store for every edge another edge that it is connected to. Since we want to be able to keep visiting the next edge so that we loop around the face, the stored next edges should all point in the same direction inside the face - say counter-clockwise. Of course in a neighboring face this direction will be opposite so that the traversal stays counter-clockwise. This introduced the notion of *half-edges*: an edge is split to two half-edges, one for each one of its incident faces, that point to a *next half-edge* in the correct direction.

A second problem that we had before was that we didn't know the incident faces of an edge. But if we know for a given half-edge its incident face, we know already one of them. Then its natural to store for a half-edge its *opposite half-edge*, that is the other half-edge that corresponds to the same edge. After traveling to the opposite

half-edge and reading its incident face, we have found both incident faces of the edge.

The half-edges have a notion of *origin* and *destination*, the vertices where it starts from and points to.

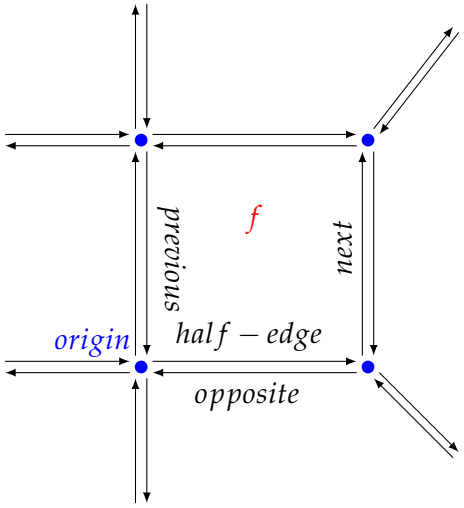
Thus we keep:

A. A vertex record, storing for each vertex its *position* and an arbitrary half-edge that has it as an origin.

B. A face record, storing an arbitrary half-edge of its boundary.

C. A half-edge record, storing its origin vertex, its next half-edge (traveling counter-clockwise in the same face), its *previous half-edge* and its incident face.

This is visualized in figure 2.2, where an arbitrary half-edge of the face f, with its opposite, next, previous half-edges and its origin are shown. The graph is visualized as flat to be clearer, but in our case it would lie in 3D space.



(a)

Figure 2.2: A doubly-connected edge list example.

We do not need to store the destination vertex of a half-edge, since it is easily found as the origin of the next half-edge. Also it may be useful, though not necessary, to keep an *edge record* pointing to one of its half-edges. This can help for looping over all edges as discussed below.

This data structure can now reduce the unnecessary complexity of the previous dihedral angle algorithm quite substantially. The process is illustrated in the algorithm 1. Now we see that the time complexity is linear to the number of edges, or constant per edge.

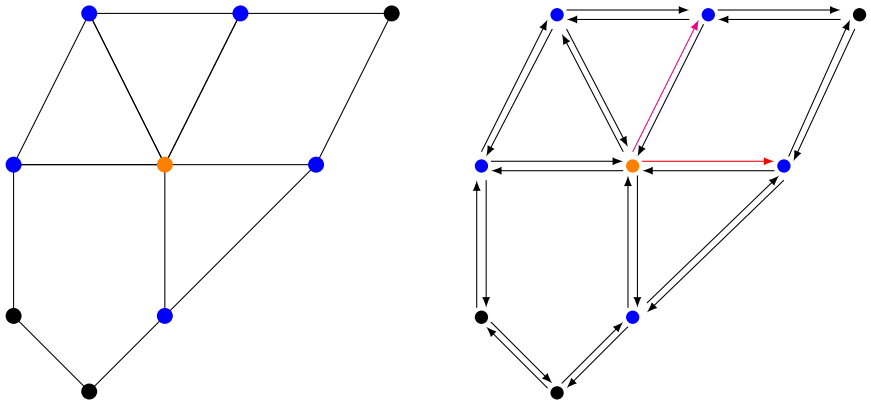
It is very common to need additional properties defined on a mesh, say for example that you need to use the dihedral angle many times, so you don't want to be calculating it every time but want to store it on every edge. For such reasons, every record in the doubly-connected edge list can hold a *property* or *attribute*, that is any quality defined for every vertex, or every face, half-edge or edge.

Algorithm 1 Calculate dihedral angle

for edge e **do**
  $he \leftarrow halfedge(e)$
  $ohe \leftarrow opposite(he)$
  $f_1 \leftarrow incident_face(he)$
  $f_2 \leftarrow incident_face(ohe)$
  $n_1 = normal(f_1)$ ▷ the normals are calculated with equation 2.1
  $n_2 = normal(f_2)$
  $\alpha = angle(n_1, n_2)$ ▷ the angle is calculated with equation 2.2
end for

Lastly, even if we don't store unnecessary information - such as the destination vertex of a half-edge - we would want to find it easily, so basic functions are used. For the destination vertex for example, the function would return the origin of the next half-edge. Or we could implement a function that returns the right incident face of a half-edge, i.e. returning the incident face of its opposite half-edge. Such trivial cases will not be written in detail in the rest of this thesis.

Let's look at an example that we will use later, which involves counter-clockwise rotation of a half-edge: that of finding the neighbors of a vertex. The *neighbors* of a vertex are the vertices that it is connected to immediately with an edge.



(a) The neighbors (blue), of a vertex (orange).

(b) Counter-clockwise rotation (magenta) of a starting half-edge (red).

Figure 2.3: Vertex neighbors and counter-clockwise rotation of a half-edge.

The counter-clockwise rotation of a half-edge can be seen in figure 2.3b, it is simply the opposite of its previous half-edge. With this in mind, finding all neighbors of a vertex can be done in linear time to the number of neighbors by starting with the half-edge that is referenced by this vertex (it has this vertex as an origin), and doing a counter-clockwise rotation until a loop has happened, storing in every case the destination vertex of the half-edge. They can be stored in a property so that they can be used later.

2.1.2 *Octrees*

Many times in arithmetic methods the discretization of space is needed, usually to approximate a quantity in every discrete cell. In one dimensional space (i.e. a line) this can be done by partitioning it in finite line segments. A start and end of the computational space will also be needed as a computer does not have infinite space, nor can it compute for infinite time. For example this process could be used to approximate the definite integral of a one variable scalar function $\int_a^b f(x)$ by partitioning the line segment from a to b in smaller segments, say N in number, $(b-a)/N$ in length, and then summing for every segment $i \in [0, N-1]$ with center $(2i+1) \cdot (b-a)/(2N)$, the product of f at its center with its length:

$$\sum_{i \in [0,N-1]} f\left(\frac{(2i+1)(b-a)}{2N}\right) \cdot \frac{b-a}{N}$$

This is a Riemann sum - as the number of segments N grows larger, the sum approximates better the integral.

In similar sense, one may want to discretize three dimensional space. Suppose then that the computational region is a box (we call this a *bounding box*, as it bounds all objects we are computing on). We now want to partition it in smaller boxes, we call these *voxels*. In a, similar to the prior example, approximation of a triple integral of a scalar function, we would sum the product of the function in the center of each voxel with the volume of the voxel. As the number of voxels would grow, the approximation would get better.

A problem arises when voxels are used in this manner on calculations that require a large amount of detail on some parts, but smaller on others. If the space is partitioned in constant density then this density would need to be the appropriate for the most detailed part, thus wasting calculations on places that need not that much detail.

The idea, then, is to subdivide voxels that need more detail in half repeatedly, until the needed density is acquired. The ones that don't need more detail need not be subdivided.

A cube is subdivided in half when it is partitioned in cubes that have half its length on all sides. There need to be 8 such cubes in every subdivision. An example can be seen in figures 2.4a and 2.4b where the original cube has been partitioned in 8 smaller cubes with half its length.

An *octree* data structure can be used to store and manipulate the partition:

An octree is a tree were every node - if it has children - has to have exactly 8 children.

An example can be seen in figure 2.5.

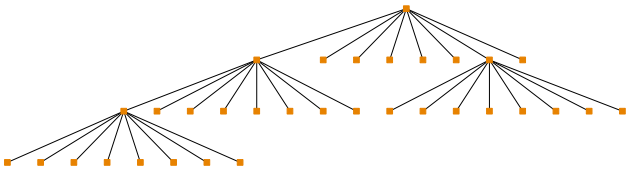
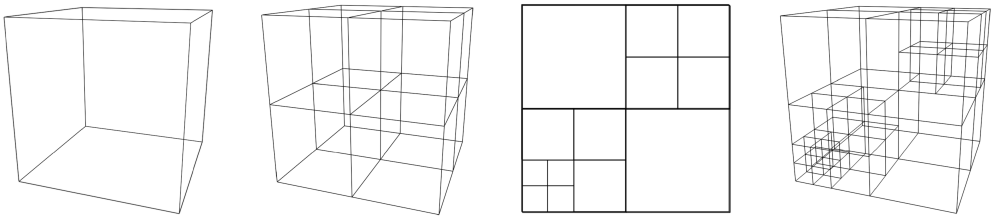
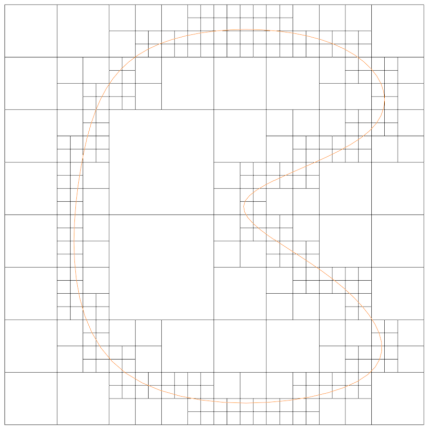


Figure 2.5: The octree that represents the partition of figure 2.4d.



(a) (b) (c) (d)

Figure 2.4: Representation of the process. (a) Shows the bounding box, (b) the outcome of one subdivision. (c) and (d) are the side and free views of a series of subdivision that correspond to the tree seen in figure 2.5.



The number 8 comes from the fact that every subdivided cube creates 8 smaller cubes. The root node of the tree, then, represents the bounding box, each other node represents a box down the line of subdivisions.

In out case, octrees will be used so that we can have more detail close to a surface in 3D space and less away from it, as outlined in the inline figure.

The *depth* of an octree is the maximum depth of the tree (for example in the tree of figure 2.5 it would be 3).

Let's now suppose that we know the value of a function at the centers of every cell of an octree, and we want to find it at an arbitrary point p. Let us also suppose that this point lies inside a cell at maximum depth D and that it has been cared for that neighboring cells are also at maximum depth (we will be taking care of this in the Poisson algorithm in section 3.2 where this process is used). The value of the function at the point p can be approximated using the trilinear interpolation: Since the cell p is in and the neighboring cells are of the same depth, their centers create a cube as the one seen in figure 2.6. We interpolate linearly the values at the 8 corners of this cube to approximate the value at p. In the figure, the corners have each been given a 3 bit binary number as name and the known value of the function at the corners is symbolized as C_{***} where $***$ is the bit representation of the corner. Also the function approximation at the point p is symbolized as P. If the point p has coordinates (x, y, z) and the corner 000 has coordinates (x_0, y_0, z_0), the length of every side of the cube is $1/2^D$ (if the length of the bounding box is 1) and the fractional distances of p from 000 are:

$$\begin{aligned} \lambda_x &= \frac{x - x_0}{1/2^D} = 2^D(x - x_0) \\ \lambda_y &= 2^D(y - y_0) \\ \lambda_z &= 2^D(z - z_0) \end{aligned} \tag{2.3}$$

Here fractional distance is meant in the sense that it is 0 at $x = x_0$ and 1 at $x = 1/2^D + x_0$ and in between it provides a ratio of which side it is closest to.

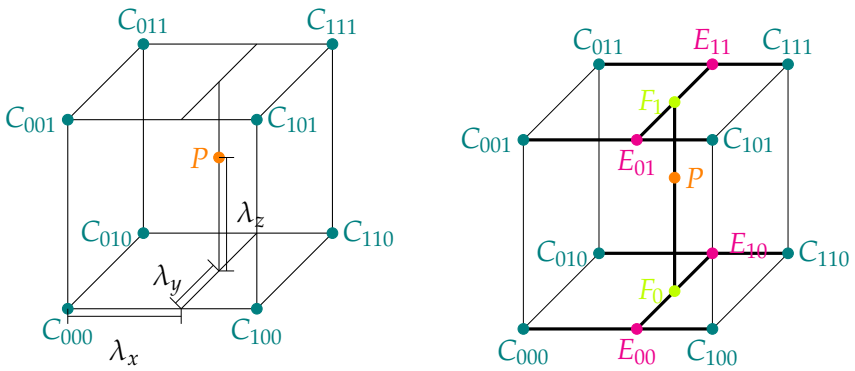


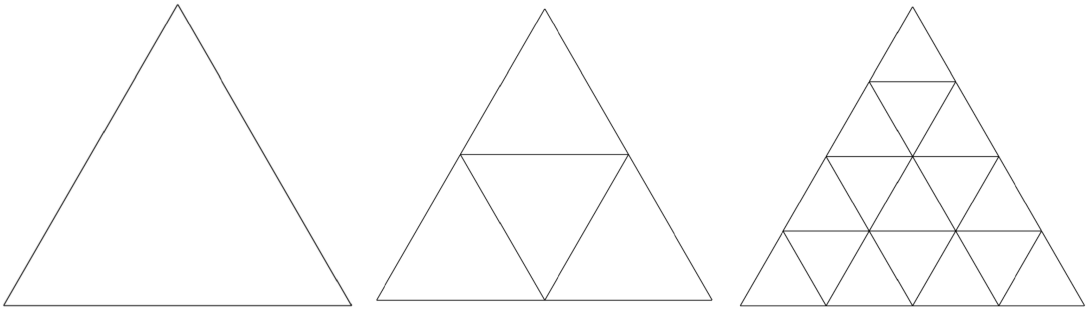
Figure 2.6: Visualization of the trilinear interpolation.

Defining P to be $(1-\lambda_x)C_{000} + \lambda_x C_{100}$ would mean that it is linearly interpolated in the x axis between the corners 000 and 100. To use all 8 corners we first interpolate them on pairs to find the x axis interpolations $(E_{00}, E_{01}, E_{10}, E_{11})$, then interpolate those one pairs to find the y axis interpolations (F_0, F_1) and lastly the last two for the final value P. This process in seen in figure 2.6 (right), the thick lines are the axis we interpolate on. The full set of equations is:

$$\begin{aligned}
E_{00} &= (1-\lambda_x)C_{000} + \lambda_x C_{100} \\
E_{01} &= (1-\lambda_x)C_{001} + \lambda_x C_{101} \\
E_{10} &= (1-\lambda_x)C_{010} + \lambda_x C_{110} \\
E_{11} &= (1-\lambda_x)C_{011} + \lambda_x C_{111} \\
F_0 &= (1-\lambda_y)E_{00} + \lambda_y E_{10} \\
F_1 &= (1-\lambda_y)E_{01} + \lambda_y E_{11} \\
P &= (1-\lambda_z)F_0 + \lambda_z F_1
\end{aligned} \tag{2.4}$$

2.2 CATMULL-CLARK SUBDIVISION SCHEME

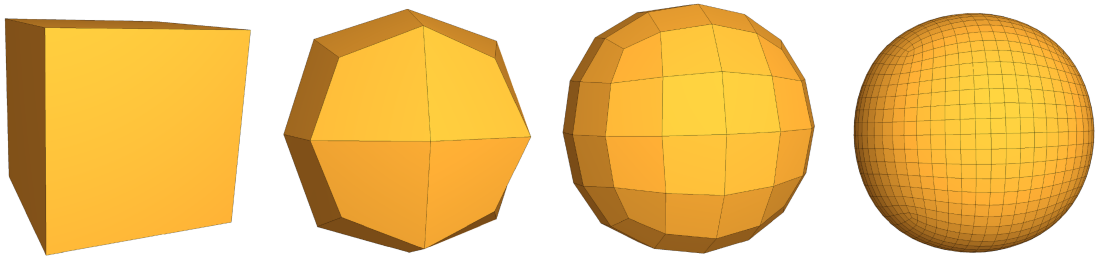
A *refinement scheme* is a set of rules on how to subdivide elements to create more detail. In the case of computer graphics they can also be called *subdivision methods* or schemes. An example, where triangles are split in four each to allow for more detail is seen on a 2D mesh in figure 2.7, the midpoint subdivision scheme [12]. By far the most popular subdivision scheme for quad meshes is *Catmull-Clark* [10], ofter written simply as *CC*.



(a) Base model. (b) After 1 refinement. (c) After 2 refinements.

Figure 2.7: Sequential steps of the triangular midpoint subdivision scheme.

This subdivision scheme was developed by E. Catmull and J. Clark to allow for the rendering of smooth surfaces. The idea is to not just introduce more detail - as seen in the example of figure 2.7, where the shape stays the same - but also move the position of the vertices so that the result is smoothed out on each step. An example of how the cube changes with recursive Catmull-Clark subdivisions can be seen in figure 2.8.



(a) Base model. (b) After 1 subdivision. (c) After 2 subdivisions. (d) After 4 subdivisions.

Figure 2.8: Catmull-Clark showcase.

This scheme can work for arbitrary models, with faces of any valence. In short, the rules of the refinements are as follows:

A. For every original face, add a vertex that has the position of the average of the vertices of the face, let's call it a face point.

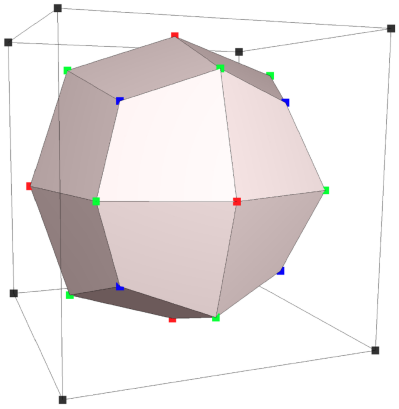
B. For every original edge, add a vertex that has the position of the average of the end points of the edge and the newly added face points of the adjacent faces of that edge, let's call them edge points.

C. For every original vertex, say v, with valence n, calculate the average f of all newly created face points from faces adjacent to v, the average e of all midpoints of its incident edges, and set a new vertex with position $\frac{f+2e+(n-3)v}{n}$, let's call those vertex points.

D. Connect with an edge the newly created edge points with the newly created face points of their adjacent faces.

E. Connect with an edge the newly created vertex points with the newly created edge points of their incident edges.

F. Define the new faces as enclosed by the new edges.



The process is visualized in the inline for the case of a cube. The original cube is plotted in *wireframe*, meaning that the faces are not visible, with its vertices black. The subdivided once mesh in grey, the red vertices are the face points, the green are the edge points and the blue the vertex points.

The edge connections form new faces as: face point - edge point - vertex point - face point, therefore the resulting mesh will always be quad, and will have as many new quads per face as its valence. The new points are chosen as such so that the new mesh will appear smoother. The authors of the original paper show that this scheme converges to bicubic B-spline surfaces [10].

2.2.1 *Boundaries and sharp edges support*

From the above rules it is not clear what happens on boundary edges, as they have only one incident face. However, before continuing let us talk about sharp edges.

The Catmull-Clark subdivision scheme can be extended to support *infinitely sharp edges* [15], that is, edges that remain C^0 in the limit, first introduced in [29]. Here C^0 refers to the smoothness of the surface, explained in more detail in section 6.3. Basically, on sharp edges, the limit surface should remain non-differentiable.

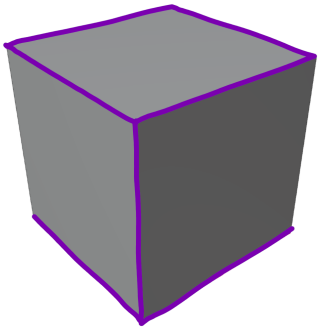
The new set of rules for the sharp edges, or simply *sharps* is:

A. Face points stay the same: the average of all of its vertices.

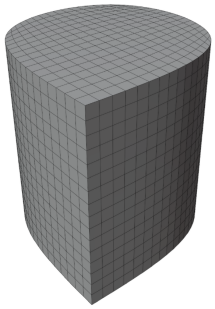
B. Edge points will simply be the midpoint of the edge: the average of the positions of its two vertices.

C. For vertex points the rules are different depending on the amount of sharp edges incident to the vertex:

   a) If the vertex has one incident sharp edge, called a *dart*, the rules are the same as in the non sharp case.



(a) Base model, the purple edges are marked as sharp. It has both crease vertices (top and bottom loops) and corner vertices (on the side sharp edge).



(b) After 4 Catmull-Clark subdivisions. The creases stay sharp only on the direction of the sharp edge, while the corners are fixed in space.

Figure 2.9: Catmull-Clark with sharps.

b) If it has two incident sharp edges, called a *crease*. Let's suppose the vertex v is a crease, with two incident sharp edges with endpoints (e_i^0, v) and (v, e_j^1), then the new vertex point will be placed at $\frac{e_i^0+6v+e_j^1}{8}$.
c) If the vertex has three or more incident sharp edges, called a *corner*, the vertex point will be placed exactly where the vertex was.

In figure 2.9 the cube example as in figure 2.8 can be seen, but with marked sharp edges now.

And to return to the boundaries, they are simply treated as sharp edges. We see from the above rules that incident faces are not used, so they are valid also for boundaries.

Sharp edges can also be extended to semi-sharp [15], but we do not deal with this case in this thesis.

Part II

PIPELINE

This part of the thesis concerns itself with every step of the pipeline. Chapter 3 deals with the reconstruction of the surface from a point cloud to a triangle mesh. Chapter 4 with converting said triangle mesh to a coarse quad one. Chapter 5 deals with the definition of a parameter space on each face of the coarse mesh and the creation of a fixed amount of parametrized points from the surface of the triangle mesh on each face. Finally, chapter 6 will present how to create a B-Spline with the previous information.

Each chapter presents independently results from that step. Part iii summarizes the results from the entirety of the pipeline.

3 SURFACE RECONSTRUCTION

The first stage of our pipeline is transforming a point cloud to a triangle mesh. The reason we do not transform immediately to a quad mesh is that a lot of research has already been made [4] in transforming to tri meshes rather than quad, and there are more appealing options to choose from. There are some methods that would create the quad mesh immediately (for example [38]) but a point cloud to tri and tri to quad pipeline has proven more resilient and reliable. Also, in this way we can use our pipeline on CAD models instead of physical objects, skipping this step and starting immediately from a triangle mesh produced from the CAD model.

In this section certain existing algorithms of creating a triangle mesh that approximated a point cloud are discussed, and the very popular Poisson method that we chose to use is explained.

3.1 STATE OF THE ART

The process of automatically creating a surface mesh from a point cloud is most usually called *surface reconstruction*. This is a topic that is extensively researched as its use cases range from computer graphics to archeology, and a lot more [4]. Usually a LiDAR scan of an object of a scene is used to capture points in 3D space that lie on the surface the object(s). These points may or may not have normal direction information data. Some algorithms, such as the Poisson algorithm, require this information and if it wasn't captured when creating the data, it needs to be reconstructed artificially. Algorithms on this end exist, for example [30] or MeshLab's method of K-nearest neighbor plane fitting [13], but are not always accurate.

We can separate the surface reconstruction methods in categories, the main ones being presented here, though our list is not complete.

Combinatorial methods interpolate between points of the data to create the surface. Essentially, the resulted surface will be composed of all or some of the original points, and triangles that connect them somehow. A delaunay tetrahedralization is usually used and the returned surface is the union of a selection of its triangles. These methods are among the few to produce surfaces with precisely stated qualities [11].

There is a variety of combinatorial methods that vary at the surface produced and runtime. The Advancing Front algorithm [14], for example, provides reliability in results and speed. Another standard method is the Ball Pivoting algorithm [5]. A detailed survey of such methods can be found in [11].

These algorithms are generally very sensitive to input data and are mostly useful when the point cloud is of certain quality, for example noise is minimal, edges are clearly defined by points on them and the points are well spaced. In general they can result in jagged edges, poor element quality, noisy surface etc.

Shape detection and primitive fitting methods exploit knowledge that an object is consisted of certain shapes to produce the best possible mesh. For example if an object is known to consist of only planar shapes, fitting planes on the point cloud using for example RANSAC can result in a perfect reconstruction. The method of [48] can fit planes, cylinders, spheres, cones and torii using RANSAC.

For piecewise planar objects, the method of [1] provides very good results.

Implicit function methods prove more versatile. This category refers to methods that firstly calculate a function that clearly defines the surface, and then produce the mesh on that function. These methods work only for watertight models, the majority of the models studied on this field are, however, watertight. The function - called an indicator function - is usually defined to be positive on the inside of the model and negative on the outside, while interpolating in between, i.e. where the surface is supposed to be.

These methods are advantageous as they can smooth out possible noise of the data and generally form complex shapes that are not easily modeled with primitives. For models that are composed of primitives, of course they cannot provide nearly as good results are shape detection.

The most popular example is that of the *Poisson* algorithm [35], [36], which we will be using. It models the surface reconstruction as a Poisson equation and solves with the Finite Element method on an octree to approximate the indicator function. It is studied in detail in the rest of this section. One known problem that it produces is that due to the definition of the problem, the sharp edges are smoothed out.

Another category of methods that represent the surface as a function are the *Moving Least Squares* algorithms, with notable example that of [42] which promises to fix the smoothing of the sharp edges of the Poisson algorithm.

We are choosing in this thesis to use the Poisson algorithm, mainly for its versatility, as it is proven to be able to reliably reconstruct complex organic and mechanical shapes. Below the base of it is explained in detail. The authors made an improvement on how it handles sharp edges in [36] which is not explained here.

3.2 POISSON RECONSTRUCTION

Suppose we want to model a volume M with the surface being its boundary ∂M. We want to reconstruct a watertight mesh from points sampled on this surface.

The poisson method takes as input points in 3D space and for every point a corresponding inward-facing normal vector, that is perpendicular to the surface and points on its inside. These normals are easily calculated during the scanning process, and if they are not provided before hand they can be estimated algorithmically with some accuracy [4].

The objective is to find an implicit function χ_M, or *indicator function*, that best fits the points. It is defined as being 0 on the outside of the surface and 1 on the inside, as can be seen in figure 3.1.

As described in [36], the Poisson algorithm can be expressed in a Galerking formulation of the Poisson equation, thus we will need to define some concepts first. Namely the *Finite Element Method*.

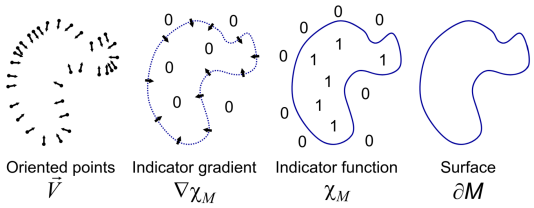


Figure 3.1: Showcase of the indicator function [35].

3.2.1 *The Finite Element Method*

Let's say that we want to solve a non trivial partial differential equation (PDE) on a domain. Later we will equate finding the implicit function of a point cloud to solving a Poisson equation in three dimensions. The Poisson equation is:

$$\Delta\phi = f \tag{3.1}$$

In general, such PDEs paired with boundary value conditions are hard to solve by hand so a computer is used to approximate their solution. A very common and versatile method is the Finite Element Method (FE method or FEM for short). In this section the FE method is introduced to solve boundary value problems. A textbook description can be found in [33].

Suppose we want to approximate the solution of the PDE:

$$L\phi = f \tag{3.2}$$

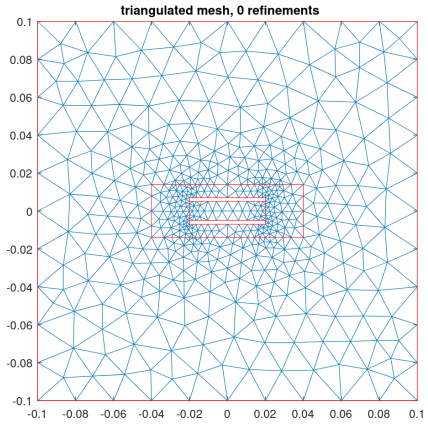
where L is the differential operator, ϕ is the unknown and f a known function. We will try to approximate ϕ as a linear combination of some basis functions:

$$\tilde{\phi} = \sum_{1}^{n} a_i B_i \tag{3.3}$$

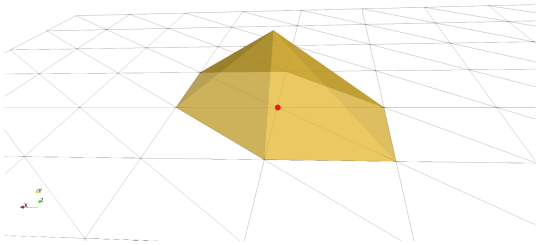
where $\tilde{\phi}$ is the approximation of ϕ, B_i the (finite) basis functions and a_i the corresponding weights, or *degrees of freedom (DOFs)*. Our goal is to estimate those DOFs.

The approach of the FE method is to partition the computational domain on *elements* and define the basis functions on those elements. For example in a 2D problem a mesh will be created on the computational domain consisting of vertices (nodes), edges and faces. The elements in this case are the faces. In a 3D problem the mesh will consist of vertices, edges, faces and *cells* (polyhedra bounded by a set of faces), the elements in this case being the cells. The mesh most usually consists of triangles in 2D, tetrahedra in 3D, but a lot more options exist. In the poisson algorithm we will use an octree in a similar sense. In figure 3.2a a 2D mesh for an electromagnetic problem is showcased.

In node based FEMs the basis functions are defined on the nodes, one basis function for every node. A very common selection is linear functions, defined as 1 on the corresponding node, zero on any element that the node is not part of, and



(a) A triangular mesh created for a 2D problem: approximating the electric potential of a parallel plaque capacitor.



(b) Black: a triangle mesh, Red: a node, Yellow: the linear basis function associated with the red node, the magnitude of the function is shown in the z axis. The basis function is 1 on the node, zero outsides the triangles the node is part of and interpolating in between.

Figure 3.2

linearly interpolating the values in the elements the node is in. An illustration can be seen in figure 3.2b.

With this choice of basis functions, the DOFs amount to the value of the function at the nodes. The basis functions then define a linear interpolation inside the elements of the values of the function at the nodes.

If we need more accuracy, quadratic or cubic basis functions can be chosen, in this case though more nodes will need to be introduced as, for example, the three nodes of a triangle are not enough for a quadratic interpolation inside it.

3.2.2 *Galerkin formulation*

The *Galerkin formulation* is specific case of the *Weighted Residual Method* that is often used in par with the FE method to approximate the solution of a PDE [22]. This method is explained shortly, the notation of [47] is used.

The *residual* R is the error of the approximation:

$$R = L\tilde{\phi} - f \tag{3.4}$$

We seek to make the residual as small as possible. The weighted residual method takes its name from the fact that some *weight functions* (or *test functions*) w_i are chosen and then the system defined by the dot products:

$$\langle R, w_i \rangle = 0 \tag{3.5}$$

is solved to find $\tilde{\phi}$. Here the dot product of functions f, g in $\mathbb{R}^n$ is defined as

$$\langle f, g \rangle = \int_{\mathbb{R}^n} f(\mathbf{x}) \cdot g(\mathbf{x}) d\mathbf{x} \tag{3.6}$$

The equations 3.5 are as many as the number of weight functions chosen.

The Galerkin formulation occurs when the weight functions are chosen as the basis functions

$$w_i = B_i$$

Then the system is

$$\langle L\tilde{\phi} - f, B_i \rangle = 0, \qquad \forall i \in 1, .., n \tag{3.7}$$

This system guarantees that the error (residual) is perpendicular to all the basis functions, thus providing the best possible approximation in the norm.

Substituting equation 3.3 in 3.7 we obtain:

$$\left\langle L \sum_i a_i B_i - f, B_j \right\rangle = 0, \qquad \forall j \in 1, .., n \tag{3.8}$$

Notice that different indexes i and j are used to differentiate the use of the basis functions in the approximation of ϕ and as weight functions.

We can cast the system in matrix form with the following operation:

$$\left\langle L \sum_i a_i B_i - f, B_j \right\rangle = 0 \Rightarrow$$

$$\left\langle L \sum_i a_i B_i, B_j \right\rangle = \langle f, B_j \rangle \Rightarrow$$

$$\sum_i a_i \langle L B_i, B_j \rangle = \langle f, B_j \rangle \Rightarrow$$

$$\begin{aligned} &\mathbf{S} \cdot \mathbf{F} = \mathbf{C} \\ &S_{ij} = \langle L B_i, B_j \rangle \\ &F_i = a_i \\ &C_i = \langle f, B_i \rangle \end{aligned} \tag{3.9}$$

$\mathbf{S}$ is an $n \times n$ matrix, $\mathbf{F}$ and $\mathbf{C}$ are $n \times 1$ matrices. The matrix $\mathbf{S}$ is called the *stiffness matrix*. The matrix $\mathbf{F}$ contains the degrees of freedom, i.e. the solution.

The basis functions B_i must be carefully chosen so that they provide a close enough approximation without overcomplicating the process. In a lot of applications linear functions suffice, however some applications call for quadratic or higher order functions.

In order for the computer to solve the system 3.9 the dot products must be converted beforehand to simple multiplicative and additive calculations. This process is illustrated with an example:

Suppose the Poisson equation 3.1 and suppose we are using linear basis functions as described in the above section. The computational domain is symbolized as Ω, its boundary being $\partial\Omega$. Since the differential operator L and the basis functions are known, the integrals in 3.9 can be simplified.

For S_{ij} let's substitute:

$$S_{ij} = \langle LB_i, B_j \rangle = \int_\Omega \Delta B_i B_j \, d\Omega = 0$$

We find a disappointing outcome: $\Delta B_i = \nabla \cdot \nabla B_i = 0$ as the basis functions are linear, i.e. their second derivative is zero. This means that the system 3.9 is not useful in this case! Some more definitions are needed, the plain Galerkin formulation we saw above, i.e.:

$$\langle L\tilde{\phi} - f, B_j \rangle = 0 \Rightarrow \int_\Omega (\Delta \sum_i a_i B_i - f) B_j \, d\Omega = 0 \tag{3.10}$$

is called the strong formulation. We can rewrite it using the integration by parts rule:

$$\int_\Omega \nabla \cdot \mathbf{V} u \, d\Omega = -\int_\Omega \nabla u \cdot \mathbf{V} \, d\Omega + \oint_{\partial\Omega} u \mathbf{V} \cdot \hat{\mathbf{n}} \, dl \tag{3.11}$$

as:

$$\begin{aligned} &\int_\Omega (\Delta \sum_i a_i B_i - f) B_j \, d\Omega = 0 \Rightarrow \\ &\sum_i a_i \int_\Omega \nabla \cdot (\nabla B_i) B_j \, d\Omega - \int_\Omega f B_j \, d\Omega = 0 \Rightarrow \\ &\sum_i a_i \left[-\int_\Omega \nabla B_i \cdot \nabla B_j \, d\Omega + \oint_{\partial\Omega} B_j \nabla B_i \cdot \hat{\mathbf{n}} \, dl \right] - \int_\Omega f B_j \, d\Omega = 0 \end{aligned} \tag{3.12}$$

This is called the weak formulation. The advantage now is that no double differentiation exist, thus linear basis functions can be used.

The line integral depends on the boundary conditions, as it is evaluated in the boundary of the domain. The most common choice is Dirichlet boundary conditions, i.e. a known value of the function at the boundary. This is also what we will use later. In this case the line integral of equation 3.12 is equal to zero, as the nodes at the boundary do not have a basis function (their values being known) and the rest of the basis functions will be zero on them. Thus the final form is:

$$\sum_i a_i \int_\Omega \nabla B_i \cdot \nabla B_j \, d\Omega + \int_\Omega f B_j \, d\Omega = 0 \tag{3.13}$$

Again we can cast it in matrix form:

$$\begin{aligned} &\mathbf{S} \cdot \mathbf{F} = \mathbf{C}, \\ &S_{ij} = \int_\Omega \nabla B_i \cdot \nabla B_j \, d\Omega \\ &F_i = a_i \\ &C_i = -\int_\Omega f B_i \, d\Omega \end{aligned} \tag{3.14}$$

It is easy now, to convert the integral of the stiffness matrix $\mathbf{S}$ in multiplicative and additive calculations: Let us call *natural support* the domain where a basis function

is non-zero. In a linear basis function it will consist of the elements adjacent to its node. The gradient of the basis functions B_i is non-zero only inside its natural support, where it is constant, and the dot product of the gradient of two different basis functions: $\nabla B_i \cdot \nabla B_j$, is non-zero for nodes i, j that are part of a common element. So the integral $\int_\Omega \nabla B_i \cdot \nabla B_j d\Omega$ is non-zero only for nodes i, j that are in a common element. When that is the case, the integral inside the common element is equal to the product of the area of the element with the two gradients of their basis functions inside the element. Thus in total, the integral is equal to the sum for all common elements of that product. An efficient algorithm is outlined in pseudocode in Algorithm 2, the exact formulas for $\nabla B_i \cdot \nabla B_j$ inside the elements are not written here, as they are of no use in the discussion, but they can easily be found by the reader.

Algorithm 2 Stiffness Matrix Calculation

```
function CALCULATE_STIFFNESS
    Initialize S with zeros
    for every element ε do
        A = Area(ε)
        for every node i in ε do
            G_i = ∇B_i inside ε
            S_ii = S_ii + A G_i · G_i
            for every node j ≠ i in ε do
                G_j = ∇B_j inside ε
                S_ij = S_ij + A G_i · G_j
                S_ji = S_ji + A G_j · G_i
            end for
        end for
    end for
    return S
end function
```

In a similar way the integral of matrix **C** can be calculated be hand beforehand and converted to performable by the computer computations.

The only thing left is to solve the system 3.14. Depending on its size, the direct method is very often not feasible and we must resolve to iterative ones [26]. It is easy to see from the algorithm that the stiffness matrix will be sparse and symmetric.

3.2.3 *Adaptive meshing - Basis function refinement*

The approximation is better the denser the elements are. We want to control the density to have a good enough approximation, however in a lot of cases we need a lot of detail in certain sections and little in others. In these cases, if we use constant density, we will have to accommodate the most important section and waste calculations on the least.

Adaptive meshing can be used [source], where the mesh is made denser on the parts that require it, while remaining less dense elsewhere. Figure 3.2a features an adaptive mesh: the triangles are denser towards the center.

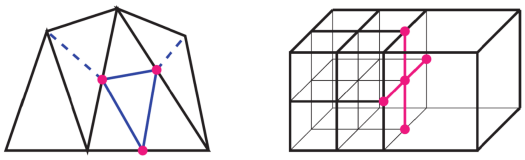


Figure 3.3: 2D Triangle and 3D cube meshes refinement, T-vertices with pink [27]. In triangle meshes they can be taken care of, by introducing the edges of the dashed line, but in cube meshes it is not possible without using non-cube elements.

The problem with adaptive meshing is that when refining an element T-vertices are introduced, as seen with pink in figure 3.3, and when T-vertices are introduced they have to be taken care of, otherwise discontinuities arise due to the interpolation inside the elements not using all the nodes [27]. However, we will use octrees, so the T-vertices cannot be handled.

For this reason we will be using instead a different method, that of *Basis Function Refinement* [27]. In basis function refinement we shift focus from the elements to the basis functions: the error can be reduced by refining the elements, but it can also be reduced by introducing more basis functions. The approximation $\tilde{\phi}$ is just the linear combination of more basis functions, this way, we need not worry about taking care of T-vertices.

The idea is to start with one basis function, and keep introducing more where needed using a *subdivision scheme*. The subdivision scheme refers to a guide on how basis functions are subdivided to introduce more detail. It is a pairing of *topological refinement* and *coefficient refinement* operators. A topological refinement operator describes how topological entities -such as elements- are split to introduce more detail in the mesh. We have already encountered one when talking about how to subdivide a cube of an octree to 8 smaller cubes.

Coefficients carried by the mesh are qualities defined on it, usually on its elements. Here the basis functions are used as coefficients as one is defined per element. A coefficient refinement operator paired with a topological refinement operator describes how the coefficient are refined when refining the topological entities.

So, suppose we are using an octree to generate basis functions. The topological refinement operator is as before, splitting a cube to 8 smaller cubes. The coefficient of an element -here one of the cubes- is a basis function B of unit integral. We can center it on each cube with center o.c and width o.w and stretch it accordingly to retain its unit integral, with:

$$B_o(q) := B\left(\frac{q - o.c}{o.w}\right)\frac{1}{o.w^3} \tag{3.15}$$

Here, o refers to the node of the octree corresponding to the cube (see figure 2.5). Hereafter nodes will refer to the octrees, and vertices to meshes. The coefficient refinement operator in this case is equation (3.15) that describes how new basis functions will be placed on the new nodes. When new basis functions are introduced the old ones are not (generally) deactivated, so in the case of the octree. when a cube is subdivided in 8 smaller cubes, there will be a basis function for the starting cube and one for every one of the smaller cubes. We can denote with $B^s(\varepsilon)$ the basis

functions that overlap the element ε and are on the same level in the octree, and with $B^a(\varepsilon)$ those from ancestor levels.

After the basis functions have been refined sufficiently, all that is left is to compute the stiffness matrix. We need to keep track for every element which basis functions are overlapping, and perform the weak-form integrals. In general the basis functions can span on multiple elements.

Algorithm 3 Stiffness Matrix Calculation

function CALCULATE_STIFFNESS
  Initialize **S** with zeros
  for every element ε **do**
    for every basis function $B_i \in B^s(\varepsilon)$ **do**
      $\mathbf{S}_{ii} = \mathbf{S}_{ii} +$ **Integrate**(B_i, B_i, ε)
      for every basis function $B_j \in B^s(\varepsilon) \setminus \{B_i\}$ **do**
        $\mathbf{S}_{ij} = \mathbf{S}_{ij} +$ **Integrate**(B_i, B_j, ε)
        $\mathbf{S}_{ji} = \mathbf{S}_{ji} +$ **Integrate**(B_j, B_i, ε)
      end for
      for every basis function $B_j \in B^a(\varepsilon)$ **do**
        $\mathbf{S}_{ij} = \mathbf{S}_{ij} +$ **Integrate**(B_i, B_j, ε)
        $\mathbf{S}_{ji} = \mathbf{S}_{ji} +$ **Integrate**(B_j, B_i, ε)
      end for
    end for
  end for
  return S
end function

Notice that now the stiffness matrix entries are the basis functions instead of the nodes. The function **Integrate** refers to the step of calculating the weak-form integral of two basis functions inside one element, as illustrated before. This integral is evaluated at the coarsest level that captures both the basis functions - as only integrations with same level or ancestral basis functions are evaluated.

3.2.4 *The Poisson algorithm*

We have now enough to introduce the Poisson algorithm. Remember that the goal is to approximate the indicator function χ_M of a model M:

$$\chi_M = \begin{cases} 1, & \text{inside } M \\ 0, & \text{outside } M \end{cases} \tag{3.16}$$

using a point set S of points $s \in S$ consisting of coordinate $s.p$ and inward-facing normal vector $s.\mathbf{N}$ data. The algorithm stems from the idea that information about the indicator function can be drawn from the normals of the points. Specifically, the gradient of χ_M should be zero everywhere except the surface ∂M, where it should be equal to the values of the normal vectors. So, if a vector field $\mathbf{V}$ is inferred from the normals of the points, then we want:

$$\begin{aligned} &\nabla\chi_M = \mathbf{V} \Rightarrow \\ &\Delta\chi_M = \nabla \cdot \mathbf{V} \end{aligned} \tag{3.17}$$

There is a problem here: the gradient of χ_M cannot actually be defined on the surface ∂M, since it is non-continuous on it. To solve this we convolve χ_M with a smoothing filter $\tilde{F}$ as $\chi_M * \tilde{F}$, making it smooth, thus a gradient can be defined. So the goal now is to solve for $\chi_M * \tilde{F}$ instead of χ_M.

A lemma proven in [35] helps here:

Lemma 3.2.1. *Given a solid M with boundary ∂M, let χ_M denote the indicator function of M, $\mathbf{N}_{\partial M}(p)$ be the inward surface normal at $p \in \partial M$, $\tilde{F}(q)$ be a smoothing filter, and $\tilde{F}_p(q) = \tilde{F}(q - p)$ its translation to the point p. The gradient of the smoothed indicator function is equal to the vector field obtained by smoothing the surface normal field:*

$$\nabla(\chi_M * \tilde{F})(q_0) = \oint_{\partial M} \tilde{F}_p(q_0)\mathbf{N}_{\partial M}(p)\, dp \tag{3.18}$$

Proof. We prove only for the x component. Similar arguments work for the y and z components.

Taking the partial derivative over x of $\chi_M * \tilde{F}$ that is a function of q:

$$\begin{aligned} \left.\frac{\partial}{\partial x}\right|_{q=q_0} (\chi_M * \tilde{F}) &= \left.\frac{\partial}{\partial x}\right|_{q=q_0} \int_{\mathbb{R}^3} \chi_M(p)\tilde{F}(q-p)\, dp \\ &= \left.\frac{\partial}{\partial x}\right|_{q=q_0} \int_M \tilde{F}(q-p)\, dp \end{aligned}$$

because of the definition of χ_M in equation (3.16). Interchanging the derivative and the integral [why can we], we get:

$$\int_M \left.\frac{\partial}{\partial x}\right|_{q=q_0} \tilde{F}(q-p)\, dp$$

and changing the differentiation from q to p:

$$\int_M \left.\frac{\partial}{\partial x}\right|_{q=q_0} \tilde{F}(q-p)\, dp = \int_M -\frac{\partial}{\partial x}\tilde{F}(q_0-p)\, dp$$

where the first derivative is on the x component of q and the second on the x component of p. Now, using the Divergence Theorem:

$$\int_M \nabla \cdot \mathbf{F}\, dV = \oint_{\partial M} \mathbf{F} \cdot \hat{\mathbf{n}}\, dS \tag{3.19}$$

we get:

$$\begin{aligned}\int_M -\frac{\partial}{\partial x}\tilde{F}(q_0 - p)\, dp &= -\int_M \nabla \cdot (\tilde{F}(q_0 - p), 0, 0)\, dp \\ &= -\oint_{\partial M} (\tilde{F}(q_0 - p), 0, 0) \cdot \hat{\mathbf{n}}\, dp\end{aligned}$$

Notice that by definition of the divergence theorem, the normal $\hat{\mathbf{n}}$ is outward-facing, while we defined $\mathbf{N}_{\partial M}$ to be inward-facing, thus:

$$\begin{aligned}-\oint_{\partial M} (\tilde{F}(q_0 - p), 0, 0) \cdot \hat{\mathbf{n}}\, dp &= \oint_{\partial M} \left\langle (\tilde{F}(q_0 - p), 0, 0), \mathbf{N}_{\partial M}(p) \right\rangle\, dp \\ &= \oint_{\partial M} \tilde{F}_p(q_0)\mathbf{N}_{\partial M}(p)\, dp\end{aligned}$$

We have thus shown that

$$\nabla(\chi_M * \tilde{F})(q_0) = \oint_{\partial M} \tilde{F}_p(q_0)\mathbf{N}_{\partial M}(p)\, dp$$

□

We can now use the above lemma to create a Poisson formulation: notice that we don't know the vector field $\mathbf{N}_{\partial M}$, but we can approximate it. Partitioning the surface on patches $\mathcal{P}_s$ of area $|\mathcal{P}_s|$ around every point $s \in S$, we can approximate the integral as:

$$\begin{aligned}\oint_{\partial M} \tilde{F}_p(q_0)\mathbf{N}_{\partial M}(p)\, dp &= \sum_{s \in S} \int_{\mathcal{P}_s} \tilde{F}_p(q_0)\mathbf{N}_{\partial M}(p)\, dp \\ &\approx \sum_{s \in S} |\mathcal{P}_s| \tilde{F}_{s.p}(q_0) s.\mathbf{N} \\ &=: \mathbf{V}(q_0)\end{aligned} \tag{3.20}$$

Then we have $\nabla(\chi_M * \tilde{F})(q) \approx \mathbf{V}(q)$, or $\Delta(\chi_M * \tilde{F})(q) \approx \nabla \cdot \mathbf{V}(q)$. In simpler terms, we want to solve the Poisson problem:

$$\Delta\chi = \nabla \cdot \mathbf{V} \tag{3.21}$$

for an implicit function $\tilde{\chi}$.

The smoothed indicator function χ is approximated as linear combination of a set of basis functions B_i:

$$\chi \approx \tilde{\chi} := \sum_i x_i B_i \tag{3.22}$$

The definition of the vector field $\mathbf{V}$ in equation (3.20) is not really usable by the computer, as $|\mathcal{P}_S|$ is not really defined and the smoothing filters are not discretized. It will be more accurately defined at last.

3.2.4.1 *Implementation*

First, important is the choice of the smoothing filter $\tilde{F}$. Kazhdan et al. chose a Gaussian with variance in the same order of magnitude as the sampling resolution, in order to make the approximation (3.20) accurate but the outcome not oversmoothed [35].

Now let's see how the poisson problem is solved: We use a basis refinement approach on an octree. Given the point set S and a maximum depth D, the octree $\mathcal{O}$ is defined as the minimal octree where every point $s \in S$ falls into a leaf node of depth D. It can also be refined during the algorithm as we will see.

We have already seen how to cast the Poisson equation in matrix form. Using equation (3.14) to solve problem 3.22 we get the linear system:

$$\begin{aligned} &\mathbf{S} \cdot \mathbf{X} = \mathbf{C} \\ &S_{ij} = \int_\Omega \nabla B_i \cdot \nabla B_j \, d\Omega \\ &X_i = x_i \\ &C_i = -\int_\Omega \nabla \cdot \mathbf{V} B_i \, d\Omega \end{aligned} \tag{3.23}$$

Since $\mathbf{V}$ is zero on the boundary, similarly as before, using the Divergence theorem, the matrix $\mathbf{C}$ can be rewritten as $C_i = \int_\Omega \nabla B_i \cdot \mathbf{V} \, d\Omega$.

For the choice of the basis functions B_i, we would need *compactly supported* functions, meaning that they are non-zero for a small amount of nodes, so that the matrix is sparse and thus faster to solve, that approximate well the function.

Also, it is useful to use a functions that approximates the smoothing filter (gaussian) $\hat{F}$ while being easier to calculate and compactly supported, Kazhdan et al. chose:

$$F(x,y,z) = (\Pi(x)\Pi(y)\Pi(z))^{*3}, \qquad \text{with} \quad \Pi(t) = \begin{cases} 1, & |t| < 0.5 \\ 0, & \text{otherwise} \end{cases} \tag{3.24}$$

where $(f)^{*n}$ denotes n repeated convolutions of f. Π is the square pulse, thus this function is that of figure A.3.

Now we can look at the definition of the vector field $\mathbf{V}$ again. Let's ignore the area $|\mathcal{P}_S|$ for now. First, we need to discretize it: The smoothing filters F are only evaluated in nodes of the octree, where they have to be appropriately scaled and transformed to F_o for node o, using equation (3.15). Then $\mathbf{V}$ becomes:

$$\mathbf{V}(q) = \sum_{s \in S} F_o(q) s.\mathbf{N}$$

This way, the smoothing filter is applied on the center of the node o containing each sample. We can allow for sub-node accuracy by using trilinear interpolation:

$$\mathbf{V}(q) = \sum_{s \in S} \sum_{o \in \text{Ngbr}_D(s)} a_{o,s} F_o(q) s.\mathbf{N}$$

where $\text{Ngbr}_D(s)$ are the 8 closest neighboring nodes at depth D of point s, forming a grid as shown in figure 2.6 - if they are not on the octree, it is refined to include them - and $a_{o,s}$ are the interpolation weights of equation (2.4).

Secondly, we need to take care of the $|\mathcal{P}_S|$ term. If the samples S are uniform, then the area $|\mathcal{P}_S|$ will be constant and so $\mathbf{V}$ will only differ by a multiplicative order from the actual normal vector field. We will see in the isosurface extraction that a multiplicative difference does not affect the outcome. In general, though, the samples are not uniform, so a density estimator function will be defined, simply as:

$$W(q) = \sum_{s \in S} \hat{F}_{s.p}(q)$$

Equation (3.25) sums gaussians centered at every point, thus providing a density estimation where the variance of the gaussian gives the area of effect. We however have to discretize this as well, similarly as $\mathbf{V}$, resulting in

$$W_{\hat{D}}(q) = \sum_{s \in S} \sum_{o \in \text{Ngbr}_{\hat{D}}(s)} a_{o,s} F_o(q) \tag{3.25}$$

Depth $\hat{D} \leq D$ provides a measure - similar to the variance - of the area of effect of the density estimation.

Using the density estimation we can retouch $\mathbf{V}$ for non-constant density:

$$\mathbf{V}(q) = \sum_{s \in S} \frac{1}{W_{\hat{D}}(q)} \sum_{o \in \text{Ngbr}_D(s)} a_{o,s} F_o(q)$$

Here, only the magnitude is changed for different densities. However we would want the width of the smoothing filters to also change, to be smaller for denser areas and wider otherwise, so that areas with a lot of detail are not oversmoothed. We can use the depth of the node o where F_o is evaluated to this cause, the deeper o is, the smoothing filter is less wide. We want the width to be proportional to the radius of the area $|\mathcal{P}_S|$:

$$o.w \propto radius(|\mathcal{P}_S|) \qquad \Rightarrow \qquad o.w^2 \propto |\mathcal{P}_S|$$

We also know that the area $|(P)_S|$ is inversely proportional to the density:

$$|(P)_S| \propto \frac{1}{W_{\hat{D}}(s.p)}$$

We can also define W to be the average density of all points, and rewrite as:

$$|(P)_S| \propto \frac{W}{W_{\hat{D}}(s.p)}$$

When the depth increases by one, the width $o.w$ is divided by two, thus:

$$o.w \propto \frac{1}{2^{depth}}$$

In the end we have:

$$o.w^2 \propto |\mathcal{P}_S| \qquad \Rightarrow \qquad \frac{1}{4^{depth}} \propto \frac{W}{W_{\hat{D}}(s.p)} \qquad \Rightarrow \qquad depth \propto \log_4 \left(\frac{W_{\hat{D}}(s.p)}{W} \right)$$

The logarithm is negative for densities less that average. We want the average density to correspond to the maximum discretization depth D, so we chose the depth to be:

$$depth(s.p) = \min \left(D, D + \log_4 \left(\frac{W_{\hat{D}}(s.p)}{W} \right) \right) \tag{3.26}$$

The vector field $\mathbf{V}$, at last, is:

$$\mathbf{V}(q) = \sum_{s \in S} \frac{1}{W_{\hat{D}}(q)} \sum_{o \in \text{Ngbr}_{depth(s.p)}(s)} a_{o,s} F_o(q) \tag{3.27}$$

where what has changed is the depth the smoothing filters are evaluated at.

$\mathbf{V}$ is now accurately defined. The input points are only used to calculate the vector field $\mathbf{V}$.

3.2.4.2 *Isosurface extraction*

We have now an approximated smoothed indicator function $\tilde{\chi}$. Recall that is is 1 inside the model and 0 outside, with in-between values close to the surface. Theoretically we expect the value to be 0.5 on the surface. We can chose 0.5 as the *isovalue*, acquiring the corresponding *isosurface*, the surface with constant value equal to that isovalue.

However, we can do better. The best isovalue is the average of $\tilde{\chi}$ on all points $s \in S$:

$$\partial \tilde{M} = \{ q \in \mathbb{R}^3 \mid \tilde{\chi}(q) = \gamma \} \qquad \text{with} \qquad \gamma = \frac{\sum_s \frac{1}{W_{\hat{D}}(s.p)} \tilde{\chi}(s.p)}{\sum_s \frac{1}{W_{\hat{D}}(s.p)}} \tag{3.28}$$

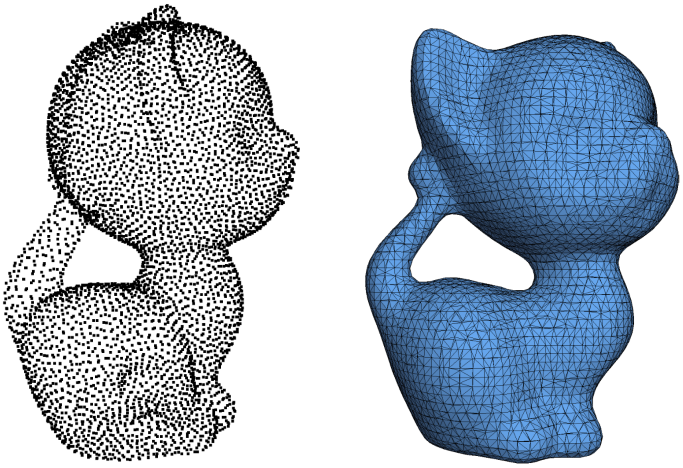
Here, $\partial \tilde{M}$ is the chosen isosurface, γ is the isovalue, chosen as the weighted by the densities average of the indicator function at the sample points.

3.3 MAKING THE TRIANGLE MESH

So far, we have managed to represent (approximate) the implied surface of the point cloud as a function in the form of

$$\hat{\chi} = \sum_i x_i B_i$$

What we want as an output, however, is a triangle mesh. A first thought would be to scatter points on the aforementioned isosurface and connect them to create the mesh. The connections could be done with some Delauney based 3d surface triangulation algorithm as in [14]. More sophisticated and faster algorithms that tackle this issue can be found in [56], [49], [55].



(a) Kitten [CGAL] point cloud. (b) The reconstructed triangle mesh.

Figure 3.4: Organic model 1 (kitten).

The topological quality of the triangle mesh in this step is of little concert to us, because we will pass it through a remeshing step later on. As is visible on the results below, the algorithm used in [35] creates very thin triangles when following contour lines of the object to place the vertices. This results in very poor quality of topology, but the important thing is that it captures all the curvature and the detail of the object.

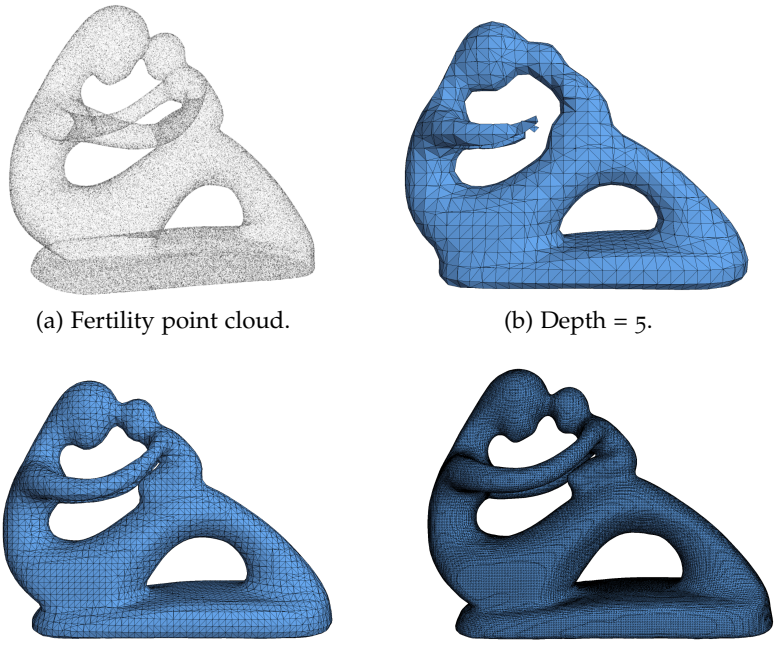
3.4 RESULTS

Let's first focus on organic models. In figure 3.4 a evenly spaced point cloud with no noise is used of an organic kitten model. We can see that the Poisson algorithm has provided a perfect reconstruction, following the natural curvatures of the model and providing a smooth result with all the details.

What has been talked about in section 3.3 is that the triangles provide poor topological quality. As seen in the figure, the placement of the vertices follows some contour lines and is not evenly spaced out. Therefore, there are triangles with very small angles, which is considered poor quality. This however is of little concern to us on this step as explained.

The parameter that proves most crucial is that of the *maximum depth D* of the octree. The larger the depth, the smaller the voxels and therefore the more detail can be captured. This also affects the size of the triangles on the output mesh. A comparison on a simple mesh can be seen in figure 3.5 and on a complex model in 3.6. It is clear that larger depths result in more detailed and more accurate meshes. In 3.6b the depth was so small that the narrow parts of the hands could not even be captured!

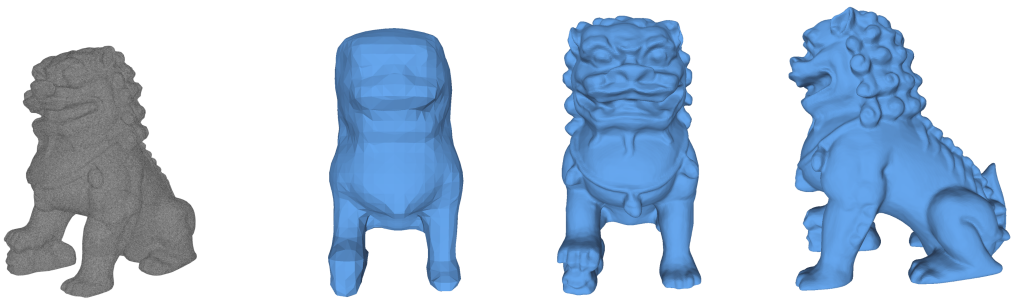
Of course larger depth values result in longer execution times, however a depth of 8 is not significantly slow (some seconds depending on the computer) and seems more than enough for the detailed model of the dragon and the simple model of fertility provides good results even with depth 6.



(a) Fertility point cloud. (b) Depth = 5.

(c) Depth = 6. (d) Depth = 8.

Figure 3.5: Organic model 2 (fertility [libigl]), depth comparison.



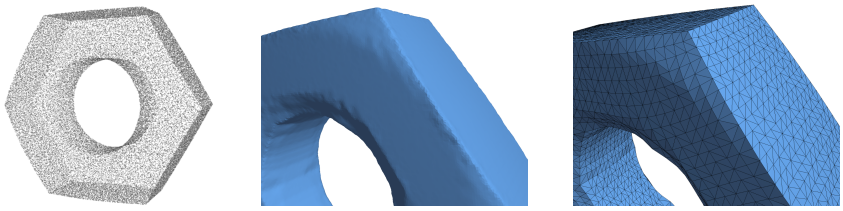
(a) Generated point cloud from the Chinese-Dragon model [CGAL]. (b) Depth = 5. (c) Depth = 8, front. (d) Depth = 8, side.

Figure 3.6: Organic model 3 (dragon), depth comparison.

There is a known problem of this algorithm on mechanical models that has been mentioned before: due to the smoothing filter of equation 3.18 that was used to make the indicator function differentiable, any sudden changes in the values are smoothed out, therefore *sharp edges* tend to be smoothed out.

Sharp edges here refers to edges where the *original model* is continuous but not differentiable, i.e. of C^0 continuity as explained in section 6.3. Since an edge of a *mesh* is always sharp, in the context of meshes a sharp edge would be one that has dihedral angle sharper than some threshold.

We would want a sharp edge of the model to create a sharp edge on the mesh, however this cannot happen in the poisson algorithm. An example is shown in figure 3.7.



(a) Bolt mesh [300] point cloud.

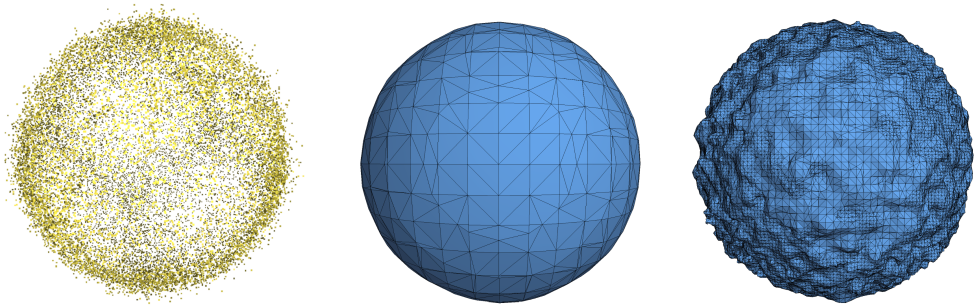
(b) Poisson reconstruction. The sharp edge has been smoothed out.

(c) Ideal sharp edge.

Figure 3.7

The implicit function methods can prove resilient to noise, but only up to a degree. In the previous examples we used point clouds that have minimal noise and no outliers. In figure 3.8 we examine the algorithms ability to handle noisy data. It is clear that it can infer pretty well the information of the surface - when it's clear - even with significant noise, however with larger octree depths overfitting is certainly possible.

Lastly, the reader can guess that this method does not handle outliers well. No example is provided as the results are not good.



(a) Sphere pointcloud with intense noise [CGAL], visualized with MeshLab.

(b) Reconstructed mesh with depth = 4. The result is very close to a perfect sphere.

(c) Reconstructed mesh with depth = 8. The surface is significantly overfitted on the noise.

Figure 3.8: Poisson reconstruction of a noisy sphere point cloud.

4

TRIANGLE TO COARSE QUAD MESH

The second step of our pipeline is the most crucial one, where we will be creating the coarse quad mesh. We want to convert the triangle mesh provided by the Poisson algorithm into one. There has been extensive research into converting tri meshes to - most usually fine - quad ones, some notable works are referenced here.

Most methods rely on a *cross field* (section 4.1.1) defined on the triangle mesh that follows the natural flow of the model to define a quad mesh of which the edges are aligned with the field, and therefore have the desired flow. Methods where the placement of the quad edges is based on this field are called *field-aligned* methods.

Another large class of methods are those that use a layout of the triangle mesh first and on a second step quadrangulate this layout. The layout can be *conforming*: not having T-junction, or otherwise *non-conforming*.

Some notable milestones of quad generating algorithms have been those of [52], [34], [7], [9].

The survey of [8] takes a thorough look on the state of art of automatic quad mesh generation.

We will base our work on Nico Pietroni's quadwild [46], as it provides very good results: retaining the sharp edges, the characteristics' and curvature's edge flow, good topology etc., but also because it is open source and we can modify it.

This method produces fine quad meshes, but we will modify it to create coarse ones.

4.1 FINE QUAD MESH USING QUADWILD

Pietroni's method creates a quad mesh that maintains features of the original model while providing good topology. An overview of the steps can be seen in figure 4.1.

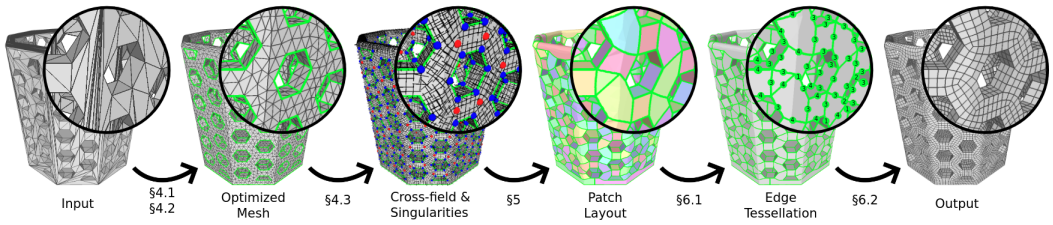
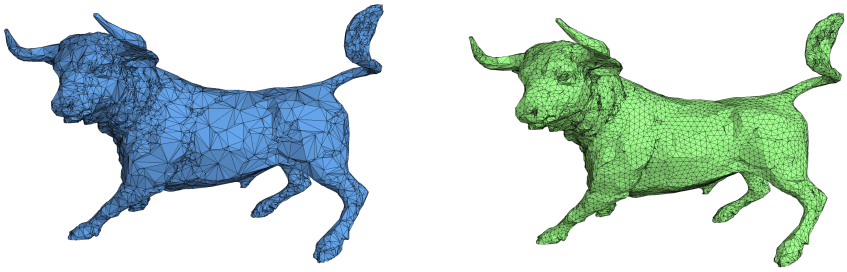


Figure 4.1: Quadwild steps overview [46].

First a remeshing step is applied. This refers to a step where the triangle mesh is transformed again to a triangle mesh, but which would have better topology – in the case of triangle meshes better topology is simply as close to constant size of elements and as close to equilateral shape as possible. We can expect the original mesh to have very bad topology, as seen in the figure, thus a better topology triangle mesh will allow the later steps to run much smoother.

Not much discussion will be made on this step, the algorithm of [31] is used, with a second adaptive pass [46]. An example is seen in figure 4.2.



(a) Input bull mesh [CGAL], the triangle topology is bad.

(b) After the remeshing step.

Figure 4.2: The remeshing step.

The second step is to create a cross field that respects the features (4-RoSy vectors defined on every face or vertex of a mesh - see section 4.1.1). Most quad meshing algorithms rely on cross fields to generate the quads on the original mesh.

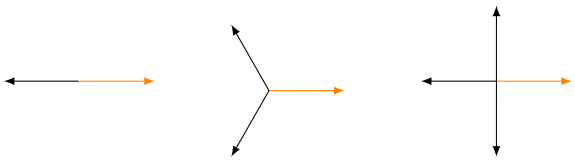
The feature lines are selected simply as the sharp edges of the original triangle mesh, where sharpness is evaluated by looking if the dihedral angle is larger than some threshold - by default 35° but it is a parameter. For a discussion on the dihedral angle see section 2.1.1.

On the third step, a non-conforming patch layout is created using the cross field. That is decimating the mesh in convex patches of 3 to 6 sides, which are aligned with the field and where T-junctions are allowed.

On the fourth step an optimizer is defined that decides on how many edges to subdivide every patch arch, respecting some topology criteria, and finally the output is the connection of those subdivided edges.

4.1.1 *Cross field*

It is useful in many design applications to define a vector field with *rotational symmetry* (*N-RoSy field*) on a mesh [43]. Generally an N-RoSy represents invariability to rotation by $\frac{2\pi}{N}$, so 2-RoSy vectors would be two opposite vectors and 4-RoSy vectors would form a cross.



(a) 2-RoSy (b) 3-RoSy (c) 4-RoSy

Figure 4.3: N-RoSy vector examples. One is colored in orange to highlight that only one needs to be defined explicitly, as the rest can be deduced from the symmetry.

The usefulness comes from the fact that the symmetry would be desired on the product of the design. For example, quad meshes benefit greatly by following a smooth 4-RoSy field - usually referred to as a *cross field* - structure, because of the

grid-like structure that is inherently desired for them to have. The idea is to define a cross field that follows the features of the model that we want to translate to a quad mesh, and follow the directions of the field for the edge flow of the output mesh.

4.1.1.1 *N-RoSy field definitions*

An N-RoSy field s consists of N directions for every point $\mathbf{p}$ of space, invariable to rotation by $\frac{2\pi}{N}$. These directions are called *member vectors* of s. For example, a constant 2-RoSy field is $\left\{\pm\begin{pmatrix}1\\0\end{pmatrix}\right\}$. Generally, for a vector $\begin{pmatrix}R\cos(\theta)\\R\sin(\theta)\end{pmatrix}$, where (R,θ) its polar coordinates, all N-RoSy vectors are:

$$\left\{\begin{pmatrix}R\cos\left(\theta+\frac{2k\pi}{N}\right)\\R\sin\left(\theta+\frac{2k\pi}{N}\right)\end{pmatrix} \mid k\in\{0,1,..,N-1\}\right\} \tag{4.1}$$

or if we allow R and θ to be functions of the spacial variable $\mathbf{p}$, then the N-RoSy field is:

$$s=\left\{\begin{pmatrix}R(\mathbf{p})\cos\left(\theta(\mathbf{p})+\frac{2k\pi}{N}\right)\\R(\mathbf{p})\sin\left(\theta(\mathbf{p})+\frac{2k\pi}{N}\right)\end{pmatrix} \mid k\in\{0,1,..,N-1\}\right\} \tag{4.2}$$

Notice that we need only one vector and the number N to find the rest using formula 4.2. The vector $\begin{pmatrix}R\cos(N\theta)\\R\sin(N\theta)\end{pmatrix}$ is defined as the *representation vector* and is the only one that is needed to perform operations on the field. We can define also the map γ that maps the field s to its representation vector, and consequently its inverse γ^{-1}. This allows us to define addition of N-RoSy fields s_1, s_2 as

$$s_1+s_2=\gamma^{-1}(\gamma(s_1)+\gamma(s_2)) \tag{4.3}$$

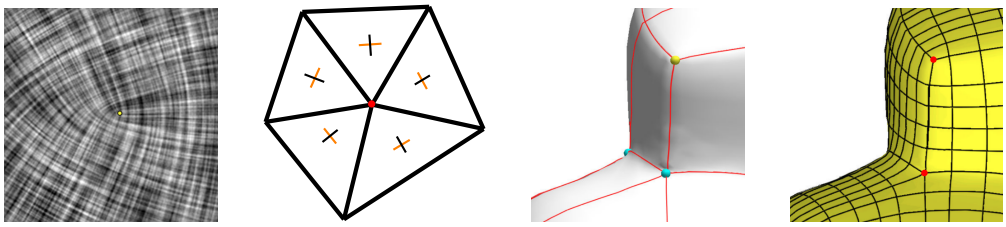
and so on other operations [43]. In figure 4.3 the representation vector is colored as orange.

A cross field can be visualized with streamlines following the directions of the vectors, as in figure 4.4a, or with crosses as in figures 4.4b, 4.7.

Suppose a vector field $V(\mathbf{p})=\begin{pmatrix}F(\mathbf{p})\\G(\mathbf{p})\end{pmatrix}$. A point $\mathbf{p_0}$ is called a *singularity* if $V(\mathbf{p_0})=\mathbf{0}$. If there exists a region containing $\mathbf{p_0}$ such that it is the only singularity inside it, then it is called an *isolated singularity*.

A point $\mathbf{p_0}$ of an N-RoSy field s is called a(n isolated) singularity if it is a(n isolated) singularity of its representation vector field.

In figures 4.4a, 4.4b examples of singularities can be seen. In 4.4a of a planar continuous and smooth cross field, and in 4.4b on a discretized cross field defined on a mesh, around an ev. Notice how the field "circles" the ev and as such averages to zero on it. There is a natural correspondence of field singularities and mesh evs in this way when edge flow is modelled to follow the field directions, or conversely,



(a) A singularity (yellow dot) of a cross field [43].

(b) Extraordinary vertex (red dot) and the cross field around it.

(c) Singularities of a cross field on the surface of a model (spheres) [43].

(d) The quad mesh structure from the cross field of 4.4c. The singularities correspond to extraordinary vertices (red dots). [43]

Figure 4.4: Cross field singularities / singularities to evs correspondence [43].

the field directions following the edge flow of a mesh. This is illustrated in figures 4.4c, 4.4.

We are mainly concerned on cross (4-RoSy) fields defined on surfaces. In particular, as we are converting a triangle mesh to quad, we need to define a cross field on the surface of a triangle mesh: The vectors need to be tangential to the surface, and are to be constant inside every triangle. In some literature [16] a group of N-RoSy vectors tangent to the surface of a facet are called an *N-PolyVector*, with the correspondent piecewise constant field being an *N-PolyVector field*.

As mentioned earlier we need only to store the representation vector of the field on every face.

Following [16], a vector on the tangential space of a face can be represented as a complex number $u \in \mathbb{C}$ as such: An orthonormal basis of the tangential plane is chosen, say $(\mathbf{b_1}, \mathbf{b_2})$, then the vector, say $\mathbf{v}$, can be expressed as a linear combination $\mathbf{v} = a\mathbf{b_1} + c\mathbf{b_2}$. We can create the complex number simply as $u = a + ic$, with i the imaginary unit. An N-RoSy vector field would then be represented with the complex numbers:

$$\left\{ u_f e^{i\frac{2k\pi}{N}} \mid k \in \{0, 1, .., N-1\} \right\} \tag{4.4}$$

where u_f is a complex number defined for every face. The rotational symmetry is provided by the term $e^{i\frac{2k\pi}{N}}$ and u_f corresponds to the representation vector.

If for every two neighboring faces f, g the edge vectors of their common edge are represented by the normalized complex numbers e_f, e_g, as seen in figure 4.5, we can compute a change in basis of the vector u_f to the basis of g with:

$$u'_f = u_f \bar{e_f} e_g \tag{4.5}$$

where $\bar{z}$ is the complex conjugate of z. Then the two vectors u_f, u_g are said to be *LC-parallel* [16] if

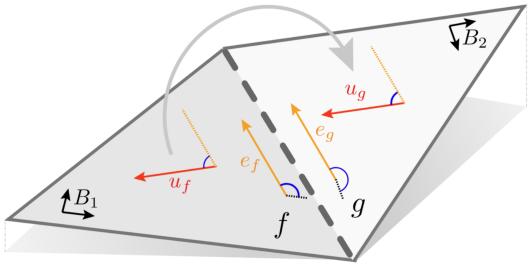


Figure 4.5: LC-parallel vectors [16].

$$u_g = u_f \bar{e_f} e_g \tag{4.6}$$

that is, if flattening both of them, or equivalently mapping the one to the other, they are the same vector. This notion or parallelism is useful in keeping the directions of the field close to constant in neighboring faces. Smoothness will later be defined to formalize this.

We can also represent an N-PolyVector as a polynomial: suppose an N-PolyVector consists of the vectors $u_0, u_1, .., u_{N-1}$, then the polynomial $P(z) = (z - u_0)(z - u_1)...(z - u_{N-1})$ represents this N-PolyVector as its roots are those vectors. If it is also N-RoSy, the vectors will be those of equation 4.4, and the polynomial will be

$$P_f(z) = \prod_k (z - u_f e^{i\frac{2\pi k}{N}}) \tag{4.7}$$

or simply

$$P_f(z) = z^N - (u_f)^N \tag{4.8}$$

The concept of LC-transport of equation 4.5 on polynomials is equivalently:

$$P'_f(z) = (z - u_0 \bar{e_f} e_g)(z - u_1 \bar{e_f} e_g)...(z - u_{N-1} \bar{e_f} e_g) \tag{4.9}$$

i.e. transporting the roots. In terms of the coefficients of the polynomial, for an N-RoSy field this would result in transporting every coefficient by the corresponding power of the edge vectors: $(\bar{e_f})^m (e_g)^m$ [16].

Now we have enough tools to define the *smoothness* of an N-PolyVector field. Ideally every N-PolyVector would be LC-parallel to the ones of neighboring faces. Then we would have a *smooth* field. Since those fields are discrete, a magnitude of smoothness cannot be defined using derivatives and thus a field either is or is not smooth. To extend this notion we can say that a field is "more" smooth when it approaches the perfect smoothness (since it is not always possible). This would amount to minimizing the deviation of neighboring N-PolyVectors from being LC-parallel. We can model this as a least squares equation of the difference of the transport of each N-PolyVector to each of its neighboring faces:

$$\sum_{m=0}^{N} \sum_{(f,g)\equiv e\in E} |x_{f,m}(\bar{e_f})^m(e_g)^m - x_{g,m}|^2 \tag{4.10}$$

Here $\{x_{f,m}\}$ are the coefficients of the polynomial of the face f. E is the set of edges, thus the second summation models summing on neighboring faces f, g. Intuitively, each coefficient is mapped accordingly to each neighboring face and their difference is punished by the least squares.

Notice that minimizing the above equation is equivalent to minimizing

$$E_{smooth} = \sum_{m=0}^{N} \sum_{(f,g)\equiv e\in E} |x_{f,m}(\bar{e_f})^m - x_{g,m}(\bar{e_g})^m|^2 \tag{4.11}$$

This form is more useful in formalizing the algorithm that will minimize it.

4.1.1.2 *Propagating a cross field on a triangle mesh*

In converting a triangle mesh to quad we want to define a cross field on the triangle mesh, whose directions will be followed to model the edge flow. The field should conform to the feature lines of the model and also its principal curvatures. Many approaches are possible on this end [16], Pietroni uses that of *constrained N-PolyVector field design*, where the field is known on a sparse number of faces, and then propagated to create the optimally smooth field that interpolates them on the entirety of the mesh.

The constrained values come from our need to follow the feature lines: If a triangle touches a feature line, the field is initialized on it so that one of its directions is parallel to that line. If it touches more than one feature lines then the triangle is split as seen in figure 4.6.

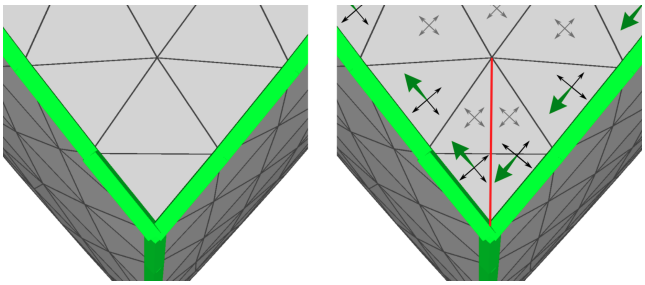


Figure 4.6: Triangle splitting [46]. Two triangles are touching two feature lines (green), and are therefore split (red cut). The initialized cross field is shown on the right.

After the constraints are set, we are minimizing the energy function of 4.11 to obtain the smoothest possible field maintaining these constraints. To minimize this term we will equate its derivative to zero. A modified version of the Laplacian as defined on meshes will be used:

Generally the Laplacian operator $L : \mathbb{R}^{|F|} \to \mathbb{R}^{|F|}$ on meshes calculates the discrete derivative of some scalar function u defined on the set of faces F, as Lu. Let's look

at one face f. Supposing it has a neighboring face g, the contribution on the total derivative is $u_g - u_f$, or

$$\begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} u_g \\ u_f \end{bmatrix}$$

The local operator $L_{(f,g)}$ defined as

$$L_{(f,g)}[f,g] = 1, \quad L_{(f,g)}[f,f] = -1 \tag{4.12}$$

models this exact behavior. Accounting for all contributions the Laplacian operator is

$$L = \sum_{(f,g)\in E} L_{(f,g)} \tag{4.13}$$

In our case we want the derivative of E_{smooth}. Notice how it is independent of the coefficient, thus we can minimize every coefficient separately, as:

$$\min_{x_{f,m}, x_{g,m}} \sum_{(f,g)\equiv e\in E} |x_{f,m}(\bar{e_f})^m - x_{g,m}(\bar{e_g})^m|^2 \tag{4.14}$$

The discrete derivative is simply $x_{f,m}(\bar{e_f})^m - x_{g,m}(\bar{e_g})^m$, or

$$\begin{bmatrix} (\bar{e_f})^m & -(\bar{e_g})^m \end{bmatrix} \begin{bmatrix} x_{f,m} \\ x_{g,m} \end{bmatrix}$$

. Then defining the local operator $L_{m,(f,g)}$ as

$$L_{m,(f,g)}[f,f] = (\bar{e_f})^m, \quad L_{m,(f,g)}[f,g] = -(\bar{e_g})^m \tag{4.15}$$

the global operator will be:

$$L_m = \sum_{(f,g)\in E} L_{m,(f,g)} \tag{4.16}$$

Solving

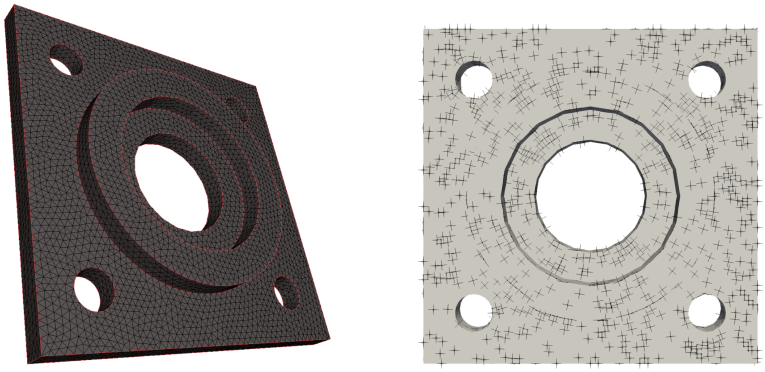
$$L_m X_m = 0, \quad \forall m \tag{4.17}$$

subject to the coefficients of the constraints be fixed, provides the optimal coefficients / the smoothest possible field. X_m are of course all the coefficients of degree m.

After the coefficients have been found, the polynomials can be factored and after we take their roots, which are the complex representation of the vectors, they can be converted back to a vector form from their orthonormal basis [16].

Two examples of the feature line identification and the propagated smooth cross field on mechanical meshes can be seen in figures 4.7 and 4.8.

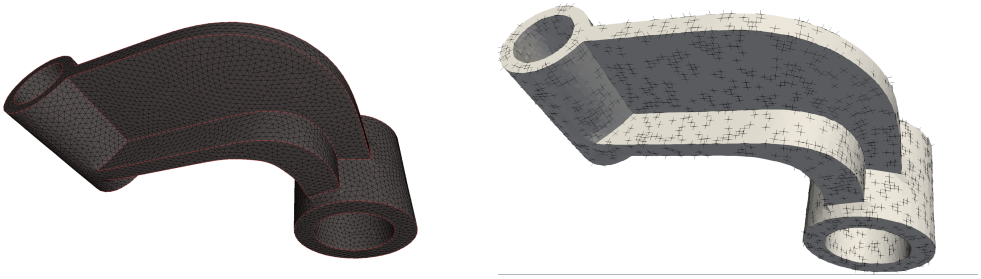
In the work of Pietroni, some additional soft constraints are added to align the field better on the principal curvatures of the model [46], [44]. As such, if a model does not have sharp features that will be identified as feature lines, the cross field can still be aligned to follow its curvature. An example is seen with a torus mesh in figure 4.9, where no feature lines have been identified, but still the field follows the natural curvatures of the model.



(a) Identified feature lines (red) of a mechanical object bearing-plate [300].

(b) The calculated cross field.

Figure 4.7: Cross field calculation, first mechanical example.



(a) Identified feature lines (red) of a mechanical object mech09 [300].

(b) The calculated cross field.

Figure 4.8: Cross field calculation, second mechanical example.

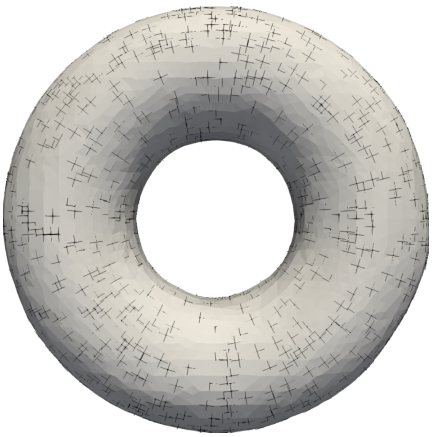
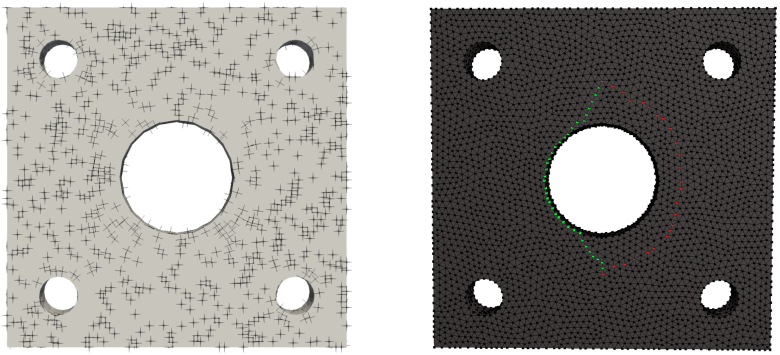


Figure 4.9: Calculated cross field on a torus model.



(a) The cross field.

(b) Paths connecting a start and edge vertex. One is the shortest path (green) and one is conforming to the cross field (red).

Figure 4.10: Tracing paths conforming to a cross field.

4.1.2 *Tracing lines*

To utilize the cross field we will be tracing lines following its directions on the mesh, say starting from a vertex s and ending in a vertex f. We will use this by tracing paths from candidate vertices to form convex patches. We want the paths of course to conform to the cross field so that the edge flow conforms with the features of the model.

The naive and easiest approach to form the paths would be a Dijkstra shortest path algorithm [17] from s to f. Of course this path would not generally conform with the features of the model, as illustrated in figure 4.10. Thus a better approach is needed.

An idea would be to modify Dijkstra's algorithm to minimize the deviation from the field instead of the distance. A middle ground of minimizing some combinations of the distance and the deviation from the field can be created also. Both those approaches were tried by the authors of the thesis, but Pietroni's method as implemented in quadwild seems to provide the best results. The difference is that instead of changing the distance metric in the Dijkstra algorithm, they change the *connections* allowed. Specifically a new graph is created from the mesh such that following the connections would result in following the field directions, the a regular Dijkstra can be performed on it.

Firstly, the field vectors need to be assigned to the vertices of the mesh - as now they are defined on the faces. For any vertex, the vectors of its adjacent faces can be averaged out, of course keeping in mind to average the same direction of the cross field and not one of the 90° symmetric ones. For vertices that are on field singularities the field should be zero. In reality the process of averaging will result to a very close to but not exactly zero vector, then we can use some threshold to identify those and make their vectors exactly zero. Separating singularities from regular vertices is easy as the directions on the adjacent faces of a regular vertex will be close enough that the average has a large magnitude, while around the singularity (as seen in figure 4.4b) the directions are circling and cancelling each other out.

After assigning a field direction on the vertices, we can add the vertices on the graph: For every vertex four nodes are added, one for each direction of the field. For vertices that correspond to field singularities no node is added. A connection is added on two nodes of the graph corresponding to vertices v_i, v_j when the are (1) connected with an edge e, (2) correspond to the same field direction and (3) the angle between the edge connecting them and their averaged field directions $\mathbf{f}$ is less than 45°. As weight of their connection a measure of the deviation of e and $\mathbf{f}$ is implemented as:

$$d_{ij} = (v_i - v_j) \cdot (\mathbf{f} \times \mathbf{n})$$

where $\mathbf{n}$ is the normal vector of the surface (following [52]).

If only the above connections are used, a problem will be introduced. The paths will of course be jagged as seen on the inline. This going back and forth is penalized as the edges will deviate from the field even though the path is generally on the field. The chosen path then is not the optimal, as the edges will minimal deviation on each path are chosen, and not on the whole line. To solve this additional connections are introduced between nodes previously connected with an intermediate node. This is the reason that vertices are seemingly missing in the red path of figure 4.10b.

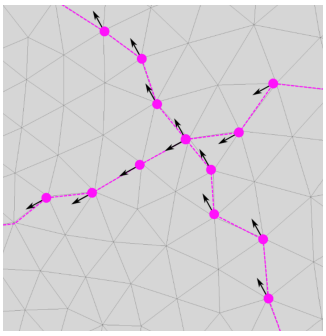


Figure 4.11: Jagged paths [46].

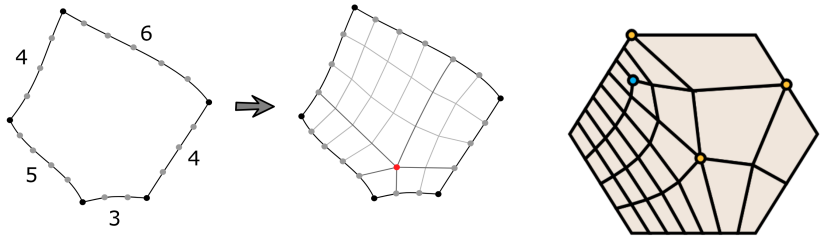
4.1.3 *Creating the patches*

The next phase of Pietroni's method is creating the convex patches. They will be created by tracing lines repeatedly on a set of candidate vertices until the criteria described below are satisfied. The patches should have between 3 and 6 sides, and be convex. A side in this case refers to a traced path on the same direction of the cross field. The feature lines have to be included on the sides of the patches.

The shape and size of the patches is important to produce a quality quadrangulation later, each side will be subdivided some amount of times and the *sub-sides* will be connected with edges. Research has been done in this topic, the ones used here are the algorithms of Tarini [51] and Takayama [50]. The former provides a good quality quadrangulation but will not always work, while the later can always compute a result (for patches with valence between 3 and 6) but is not always of good quality. We try to create patches that satisfy the conditions of Tarini, and when that is not possible we use the general strategy of Takayama.

Examples of what those quadrangulations look like are shown in figure 4.12. The quadrangulations for a general patch of n sides and some amount of subdivisions on each side is provided by the algorithms, and then it is mapped back on the actual patch using LSCM parametrization [37].

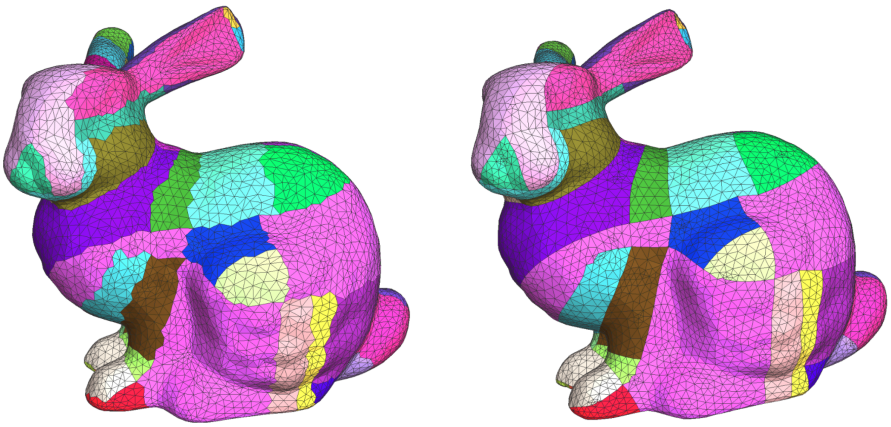
For patches p of 3 to 6 sides e_i with ordered indices clockwise or counter-clockwise, the condition of Tarini for a good triangulation are [51]:



(a) Tarini's quadrangulation example [51].

(b) Takayama's quadrangulation example [50].

Figure 4.12: The two patch quadrangulation methods.



(a) Calculated patches.

(b) After the boundaries have been straightened.

Figure 4.13: Straightening the borders of the patches.

$$
\begin{aligned}
&p \text{ has 3 sides:} && e_i \leq e_{i+1} + e_{i+2} \\
&p \text{ has 4 sides:} && e_i = e_{i+2} \\
&p \text{ has 5 sides:} && e_i + e_{i+1} + e_{i+4} \geq e_{i+2} + e_{i+3} \\
&p \text{ has 6 sides:} && e_i \leq e_{i+2} + e_{i+4}
\end{aligned}
\tag{4.18}
$$

The amount of subdivisions per side is of course not known yet, but it can be estimated using the length of the side - as the edges are expected to be roughly equal.

The patches are created with rounds of inserting traced paths only when they benefit the above criteria, and after the constraints have been satisfied a round of cleanup redundant paths is used and the patches are ready. The steps are explained in more detail in [46].

Of course, the sides of a patch, because they are traced paths, are not generally straight. After they have been calculated, any border of patches is straightened as seen in figure 4.13.

Depending on how many sub-sides the patch sides are subdivided to in the next step, they will conform better or worse to the curvature of the model they will map to.

Lastly, the feature lines are used as sides and also a rule is enforced so that the points will remain exactly on them. Thus any sharp edges on the original mesh will remain sharp on the final mesh.

4.1.4 *Optimizer*

Now that the patches are created, the last remaining step is to calculate the amount of subdivisions per side of the patch. This is cast as an integer linear optimization problem (ILP) [54] minimizing an objective function defined on integer variables, subject to some number of constraints. The integer variables that constitute the solution are the amounts of subdivisions per side of the patches s_i. (For simplification T junctions and right turns are not taken into account in this thesis). The ILP is modeled so that the subdivisions provide a good quality topology after using the quadrangulation technics of Tarini and Takayama.

The objective function is complicated to write in a single equation, so its terms and constraints are described below. Essentially every term introduced is summed in the total function.

PARITY Firstly, a patch can be quadrangulated only if its total sub-sides are even in number. As proof, suppose a patch p that is quadrangulated with the set of quads Q. Suppose then summing the summation $\sum_{q \in Q} 4 = 4|Q|$ where we some the amount of edges (4) for each quad. In this sum, internal edges are counted twice and external ones only once, naming the set of the external ones as O and the set of the internals as I: $4|Q| = |E| + 2|I|$. Then $|E| = 4|Q| + 2|I|$ which is an even number. Of course, the amount of external edges is also the number of sub-sides of the patch, thus proving the statement.

With this in mind, an auxiliary variable k is added to the problem and the constraint $\sum_{i \in p} s_i = 2k$ is used for every patch p.

ISOMETRY As discussed in section 1.2.1, we would want the edges to have as close to a constant length as possible. In Pietroni's algorithm this is enforced using a user defined parameter that sets the ideal edge length. Then dividing the side length by the ideal length, the ideal subdivisions per side $\hat{s_i}$ can be found for every side.

The isometry term penalizes the deviation from this ideal length, including the term $\sum_i (s_i - \hat{s_i})^2$ in the objective function.

REGULARITY FOR RECTANGULAR PATCHES A rectangular patch is quadrangulated best as a grid, i.e. when the opposite sides match: $s_i = s_{i+2}, s_{i+1} = s_{i+3}$. This could be added as a constraint, but a problem would arise as it is not always possible to enforce. Therefore, in Pietroni's algorithm it is *strongly favored*, which means that the constraint is added as $s_i = s_{i+2} + K$ and the new variable $K \geq 0$ is added to the objective function, with some weight that is fine tuned.

GENERAL REGULARITY The same process as above is used to strongly favor the inequalities of 4.18 for a good quadrangulation of non rectangular patches.

SINGULARITY ALIGNMENT A more complicated term, exploiting implications of the ev placement in [51] is used to align the evs of the resulting mesh, as explained in [46].

All the above terms are weighted with respect to each other so that the result does not overwhelmingly favor one over the others.

4.2 MESSING WITH THE OPTIMIZER TO GET A COARSE RESULT

In our work we want to obtain a coarse quad mesh, while Pietroni's algorithm provides a fine one. We would want the coarse mesh to capture just the topology of the model, without additional detail (as explained in section 1.3). It can be though as minimizing the number of edges without loosing from the quality of the topology, which is what we are trying to model below.

As described in section 4.1.4, there is a user defined parameter for the desired edge length. However this is only a part of the optimizer and no matter how low it is set on most models we cannot get a coarse enough result. Also we would like for the method to always produce the coarsest possible result, without the user having to result on trial and error to fine tune a parameter.

Our idea is, then, to change the objective function of the optimization step, remove the user specified parameter and penalize the amount of sub-sides while not loosing topology quality.

Therefore, the isometry term is changed, while keeping the rest of the term identical. As we want to limit the amount of quads, thus limiting the amount of edges, or equivalently limiting the amount of subdivisions per patch side, the term

$$\sum_i s_i^2$$

is added on the objective function.

But since the isometry term is removed and there is nothing favoring the quads to have constant edge length, the quality will not be generally good. Thus a modified version of the isometry term is added:

$$\sum_i (s_i - k\hat{s}_i)^2$$

Here k is a free variable, meaning no constraint is added on it and it can get whatever value is best for the system. What it accomplices is that the sub-sides have constant size (k times the previously ideal) but not a pre-specified one. The only reason $\hat{s}_i$ is used instead of only k, is to retain the scale-invariability on the models, since s_i need to be compared to some integer value of subdivisions. The ideal length is not taken into account in this case.

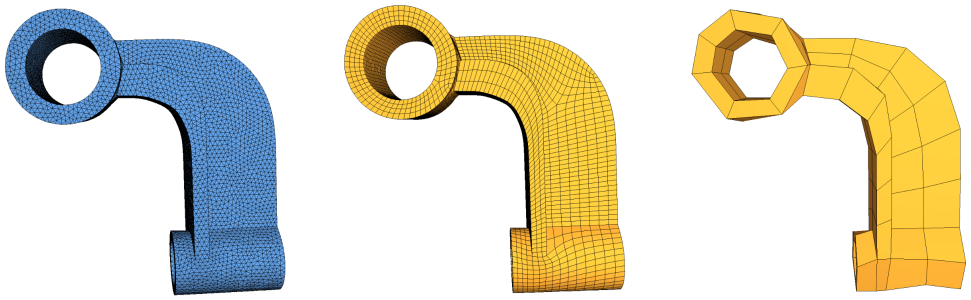
The isometry term is now changed from

$$\min \sum_i (s_i - \hat{s}_i)^2$$

to

$$\min \sum_i (s_i - k\hat{s}_i)^2 + a \sum_i s_i^2 \tag{4.19}$$

with a being a constant weight to balance the terms, set to $4 \cdot 10^{-4}$.



(a) Original triangle mesh (mecho9).

(b) Fine quad reconstructed mesh.

(c) Coarse quad reconstructed mesh.

Figure 4.14: The reconstruction of a mechanical mesh. The cross field can be seen in figure 4.8.

4.3 RESULTS

The algorithm is tested here and some examples are shown with mechanical and organic meshes, featuring the triangle mesh, the fine quad mesh of Pietroni's quadwild and the coarse quad mesh of our modified version for comparison.

One thing to notice is that, though normally quad meshing algorithms place extraordinary vertices (evs) *on* the edges of the patches, therefore they would be placed on the feature lines in the final mesh, Pietroni's quadwild explicitly avoids them, placing them in the interior. It is visible in 4.14b, with the ev being in the interior of the flat part. Our coarse method (4.14c) manages to avoid placing an ev entirely, as the faces are large enough that it is not needed.

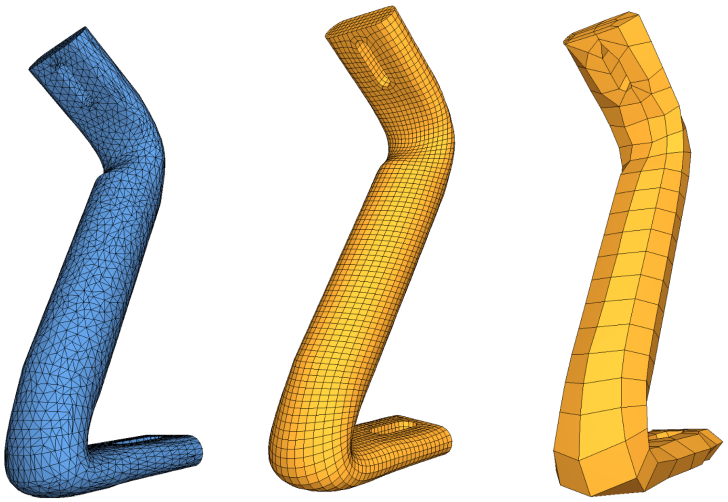
In the example of 4.15, one thing to notice is that even though the long part of the stem could be represented by long quads, making the mesh coarser without loosing the shape, our algorithm decided to go form smaller square faces. This is a consequence of our topological criterion of keeping the quads as close to squares as possible.

The organic mesh of the Stanford bunny (figure 4.16) results on a very good mesh, showcasing exactly what we need of the coarse algorithm, capturing the shape without the details, and with a good topology.

4.4 FINDING THE SHARPS ON THE QUAD MESH

A last step we want to take care of is keeping track of the sharp edge information on the coarse quad mesh. The sharp edges (i.e. the feature lines in our case) are first identified from the dihedral angle on the remeshed triangle mesh. Those edges correspond to some edges on the final coarse quad mesh, but it is not obvious to which.

First, notice that we can easily map faces that correspond to the same patch, as the patches are first defined on the triangles - so for every triangle we know its patch - and then the quads are generated inside the patches, providing the information also.

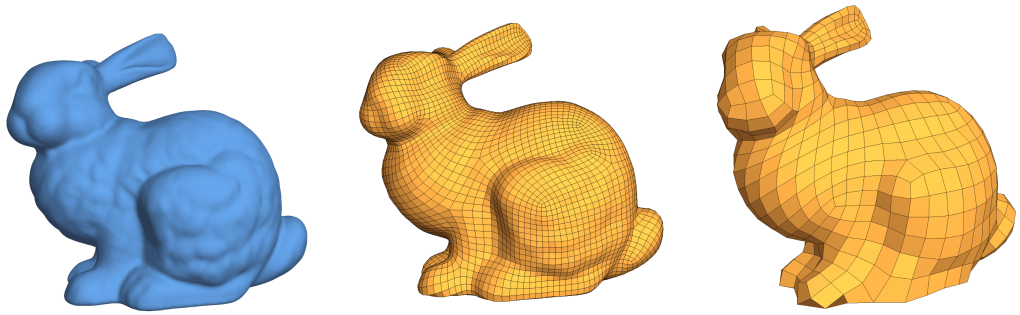


(a) Original triangle mesh BendTube [300].

(b) Fine quad reconstructed mesh.

(c) Coarse quad reconstructed mesh.

Figure 4.15: The reconstruction of a second mechanical mesh.

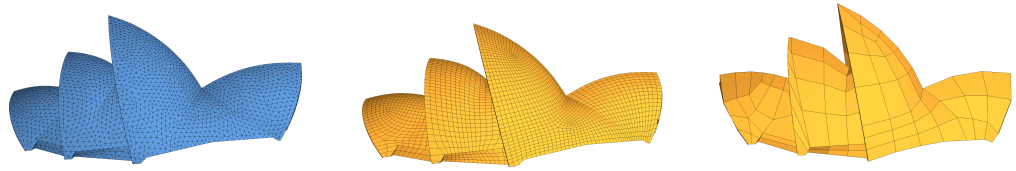


(a) Original Stanford bunny triangle mesh.

(b) Fine quad reconstructed mesh.

(c) Coarse quad reconstructed mesh.

Figure 4.16: The Stanford bunny reconstruction.



(a) Sydney [300] triangle mesh.

(b) Fine quad reconstructed mesh.

(c) Coarse quad reconstructed mesh.

Figure 4.17: Opera of Sydney reconstruction.

To do the mapping, first the sharp edges on the triangle mesh are looped, for each one the two patches on its left and on its right are stored. Then the quad edges are looped, if the corresponding patches on the left and right are the same, then it is a candidate sharp edge. It is not marked immediately as sharp because it is not certain that the entirety of the border of those patches is sharp. To mark it, the distance of both endpoints (vertices) of the edge to the closest sharp triangle edge is calculated. If both are closer than the average edge length divided by 2, then the edge is marked as sharp.

Also, since any sharp triangle edge is marked as such because it is part of a feature line, we know for each one the feature line its part of. Thus we can create a map, mapping a sharp quad edge, through the sharp triangle edge that was used to identify it, to the feature line it stems from. This map will prove useful in the projection of sharp edges in section 5.6.1.

4.5 CRITIQUE

There is a problem with our approach that rises from the identification of the feature lines: a simple dihedral angle threshold is used, which means that only *sharp* edges are identified and *almost sharp* lines are lost. Almost sharp here refers to lines or features in a model around which the surface deforms greatly, but over a *series* of edges and not on a single one. The angles of each one of the edges will be small, therefore not marking them as sharp, but accumulatively they create a corner. An example is visualized in figure 4.18.

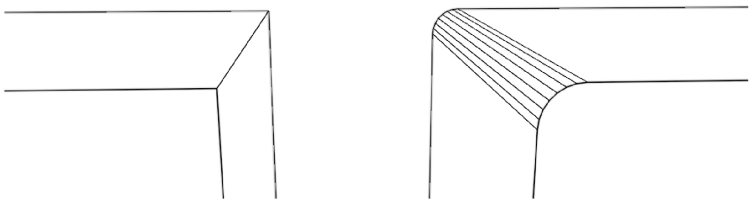
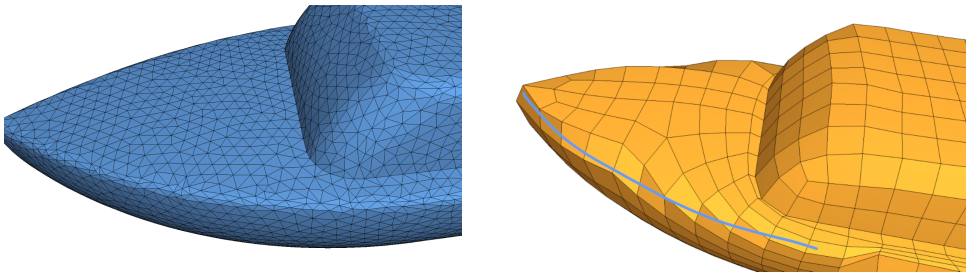


Figure 4.18: A sharp and an almost sharp edge.

Since our dihedral test cannot identify such edges, they are absent from feature lines and are, therefore, not necessarily kept. This can be seen in the opera of Sydney example (figure 4.17) where the interior almost sharp edges of the triangle mesh have not been kept as features and therefore as edges on the quad mesh. On our spline application in chapter 6, this will be fixed through the fitting, but more extreme cases are not always fixable. As seen in figure 4.19, since the almost sharp feature was not identified, the edge flow ended up being in 45° to the feature and the faces *cannot* conform to the flow. This will present problems later on the projection. Keep in mind that this problem exists also in Pietroni's original method.

The solution for is an immediate one: more sophisticated approaches to finding feature lines than that of the dihedral threshold exist, which could identify almost sharp edges, such as [24], [6] and many more. Using these would result immediately in noticeably better results. Another approach is to allow the user to hand draw (select) the features they would like to be identified.

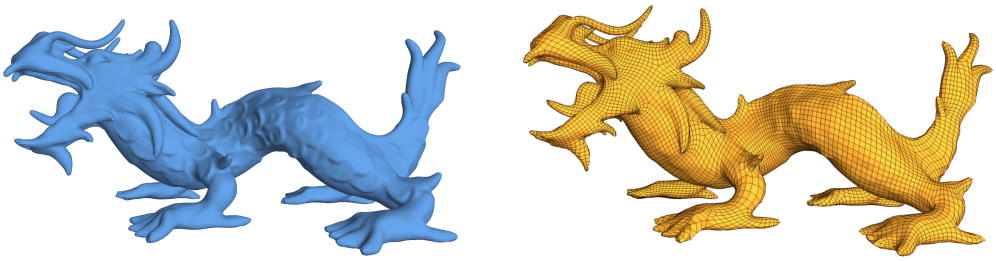


(a) An almost sharp edge of a boat model [300].

(b) The coarse quad mesh. With blue the ideal flow on the almost sharp.

Figure 4.19: Almost sharp edges present a problem.

Finally, on meshes with densely detailed shape, the isometry term of equation 4.19, doesn't allow for very coarse meshes: the faces need to be small enough to capture the small details – here detail refers to the shape, for example the horns of the dragon in figure 4.20 – but since they are encouraged to have close to constant size, they will remain small everywhere, even if there is no detail there. An example is illustrated below.



(a) A detailed dragon [300] triangle mesh.

(b) The coarse quad reconstruction.

Figure 4.20: A detailed mesh will not provide a very coarse result.

5 PARAMETRIZATION / PROJECTION

5.1 WHY PARAMETRIZE

Parametrization of a mesh is informally defining a parameter space on the surface of the mesh, so that this information can be used in mapping information of a different form on it, or mapping information from the mesh on some different representation [32], [23].

The most common example is that of texture maps in CGI, where an image has to be mapped on the surface of a mesh [28], [18]. The surface needs to be "flattened", the image is placed on the flat mesh, and then the surface is returned to 3D space. To do that, what is called a *u-v map* is used, which essentially is defining the space of two orthogonal parameters ($u, v \in [0, 1]$) on the surface. Every point of the 3D surface is represented by a pair of (u, v) coordinates. Then, since those are 2D, an image can be overlaid on the u-v space thus mapping it on the surface.

In this case one parameter space is defined over the entirety of the mesh, however in the case of [40], individual parameter spaces are defined on every face of a quad mesh, as visualized in figure 5.1.

As such, there are different kinds of parametrization and different reasons one may want one. In our case, we are interested in creating a spline from the quad mesh, using the algorithm of Marsala et al. [40], therefore we will define a u-v parameter space on every face of our coarse quad mesh. Later, we will need to find points that lie on the original model and pair their 3D coordinates with their u-v parameters on the corresponding face so that the fitting (in chapter 6) may happen. The process is explained below.

5.2 GENERAL VIEW

So far we have reconstructed a triangle mesh from a point cloud, and a coarse quad mesh from the triangle mesh. Now we want to create a spline that closely follows the original point cloud, using the coarse quad mesh. The coarse mesh, of course,

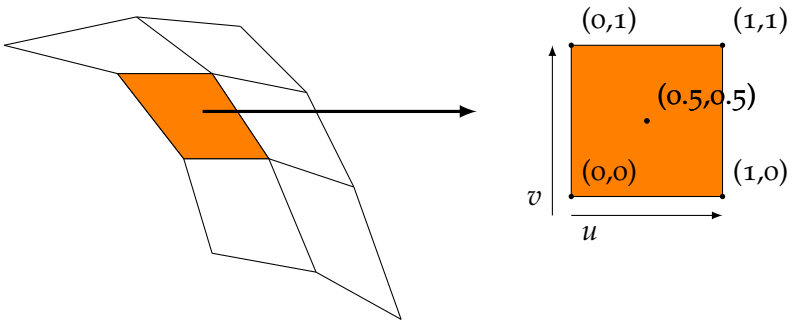


Figure 5.1: A u-v parameter space defined on a face of a quad mesh.

does not have enough information to capture the details of the point cloud, so a fitting step is needed in between.

An amount of data points will be used for every face of the coarse quad mesh, and each face will deform as needed to follow those data points. The point of the parametrization becomes clear here: while the face itself will have warped from flat to some curved surface, the parametrization is the same and can provide a link between the curved surface and the original flat face.

In preparation of the fitting stage, we need to create the dataset of the original information in its proper form. What is needed in Marsala's algorithm are:

1. A definition of a u-v parameter space on every face of the quad mesh.
2. The 3D coordinates of some points (at least 36) for every face of the quad mesh.
3. The u-v parameters those points should have on the deformed surface of the face.

Our approach to creating these is some form of projection of points from the coarse quad to the detailed triangle mesh. The triangle mesh is supposed to have enough detail to be almost identical to the original point cloud. Specifically the remeshed triangle mesh of the first step of the conversion to quad is used (section 4.1).

One approach would be to project every vertex of the triangle mesh onto the quad. From the exact point of the projection - that lies inside some face - its u-v parameters could be found inside this face. Those could then be used as the data points.

There are a couple of problems here. First, there is no guarantee that enough points will be projected in each face of the quad mesh - remember we need at least 36 - or that *too many* points will be projected into a face, rendering the fitting very slow. Secondly, on curved surfaces (see figure 5.4) the projection would not reach the edges of a quad face, resulting in deformed data.

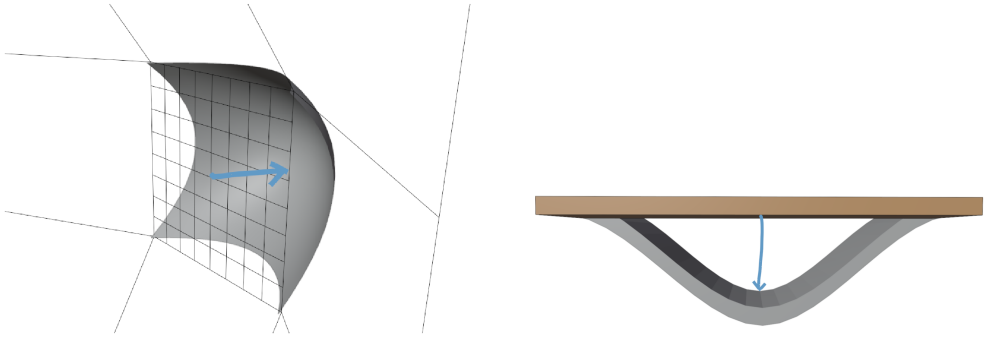
Our idea is to perform the approach in the opposite: A grid of points is defined on every face of the quad mesh. The u-v parameters of each point is easy to find, depending on how the grid has been created. Then these points are projected onto the detailed triangle mesh, finding the closest point of that surface provides the 3D coordinates.

There is another issue with this approach: the closest point would not necessarily be the correct correspondence, as visualized in 5.2b. But for our work, we assume that the curvature has been captured well enough (by our isometry term in the optimizer - section 4.2) that this is not a problem.

5.3 DEFINING THE U-V SPACE

A u-v parameter space need to be defined on every quad on a consistent manner, so that every part of the program that uses it, uses the same space.

Since we are defining a new one on every quad face, and since the parameters should range form 0 to 1, it is as simple as choosing a vertex on which assigning $(0, 0)$ and one of its edges where u will be increasing.



(a) Grid points projection.

(b) Due to high curvature on the original mesh, a grid point was projected on the wrong side.

Figure 5.2: Grid points projection.

Remember (from section 2.1.1) that the mesh is stored as a doubly-connected edge list data structure, and that for every face instance, a half-edge is referenced. We can use this half-edge to consistently define the u-v directions:

The origin vertex of this half-edge is labeled as v_1. Then, traversing in every step the next half-edge, and getting their origin vertex we label the other 3 vertices, counter-clockwise, as v_2, v_3, v_4. The direction $v_1 \rightarrow v_2$ is used as the u direction and $v_1 \rightarrow v_4$ as the parameter v direction. Thus, v_1 has $(0,0)$ coordinates, v_2 has $(1,0)$ coordinates, v_3 has $(1,1)$ and so on. The in-face points get linearly interpolated u-v coordinates from these vertices. The process is visualized in figure 5.3, where the orange half-edge labeled as *he* is the one referenced by the face in the DCEL data structure.

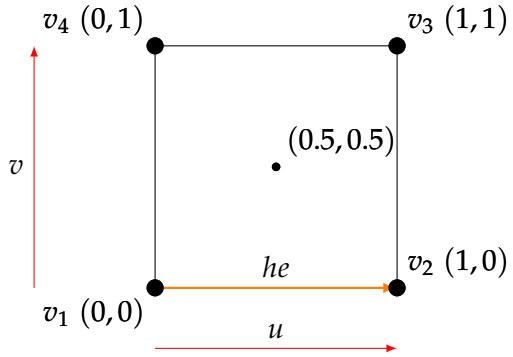
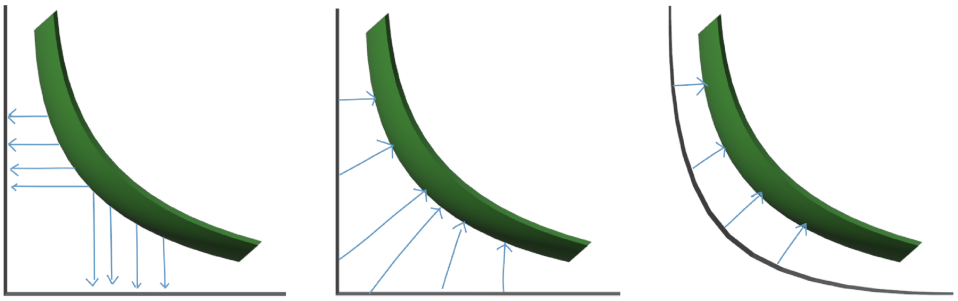


Figure 5.3: u-v space definition.

5.4 CREATING THE GRID POINTS

The grid could be created simply flat on each mesh, as $n \times n$ points. However in this case the projection introduces some deformation on high curvature surfaces (different densities as seen in figure 5.4).

Our idea is to perform some Catmull-Clark subdivisions on the quad mesh before the projection, so that it follows the curvature better and the projection runs



(a) Projecting from the original surface to the quad, not all u-vs will be used.

(b) Projecting from the quad sampling uniformly the u-vs.

(c) Projecting from a subdivided with CC mesh.

Figure 5.4: Different options for the projection. The grey lines represent two faces of the quad mesh (in left and center) and the 3 times subdivided with Catmull-Clark quad mesh on the right. The green surface represents the original mesh we want to capture information from.

smoother (see figure 5.4). Remember that we have kept track of the sharp edge information (section 4.4) and that CC supports sharp edges, thus when the surface is not actually curved the subdivided surface will not curve without reason.

Then we don't need to define a grid on each face, we can use the new vertices of the subdivided mesh. To obtain more than 36 points per face, as needed we need at least 3 CC subdivisions, obtaining 81 vertices for every face (counting the common vertices on both incident faces) as seen in figure 5.5.

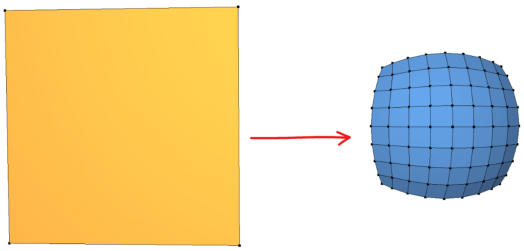


Figure 5.5: A face after 3 Catmull-Clark subdivisions.

Those vertices are used as the grid points, their u-v coordinates are kept track when they are produced:

Suppose an initial face f, the u and v values at its vertices are either 0 or 1. When it is subdivided, new points are placed on the midpoints of the edges, taking u, v values 0.5 accordingly, and one vertex is added on the middle of the face, with u-v coordinates $(0.5, 0.5)$. There are now 4 sub-faces. With the same logic, when those are subdivided, the new vertices get the middle value of of the uvs at the vertices of each sub-face. This is illustrated in figure 5.6, where a generic face whose u ranges from u_s to u_e and v from v_s to v_e, is subdivided, the new midpoints being $u_s + (u_e - u_s)/2$ and $v_s + (v_e - v_s)/2$.

In the code, care has to be taken on which order the new vertices are generated, so that the correct vertex index gets the correct u-v pair. In G+Smo, first a loop over all edges of the mesh creates the edge points and then a loop over all faces creates the face points. Thus, the index of the edge point of the edge with index e_i will

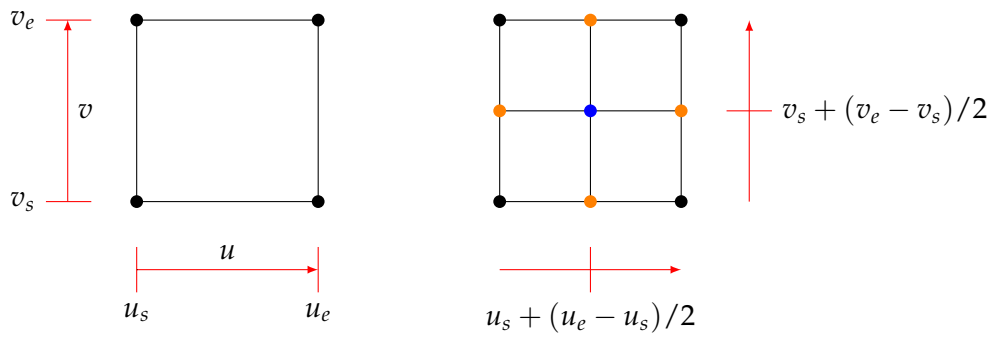


Figure 5.6: Updating the u-v values after one CC subdivision.

have index $|V| + e_i$, and the face point of the face f_i, $|V| + |E| + f_i$, with $|V|, |E|$ the total number of vertices and edges. This equation holds for each subdivision step separately.

5.5 PROJECTING A POINT ON A MESH ACCURATELY

We have created 81 grid points for every face of the coarse quad mesh. Now we need to project each one to the triangle mesh, i.e. find the closest point of the triangle surface to it. It doesn't have to be a vertex, we want the 3D coordinates of the surface that are the closest.

Since projecting immediately using a mathematical formula is not possible, let's first solve the easier problem, finding the closest vertex of the triangle mesh to any point $p \in \mathbb{R}^3$. The naive approach would be to loop over all vertices, calculate the distance to p and keep the closest one, but remember, this will be calculated for 81 grid points for every face of the quad mesh, with a total of $81 \cdot |F|^{coarse} \cdot |V|^{triangle}$ distance calculations. We can expect the triangle mesh to be very dense, making this approach extremely slow.

We can do better using a k-d tree [2]. A k-d tree is a data structure that allows for k-nearest neighbor searches in logarithmic time (for a textbook description see [3]). After the tree has been initialized on the remeshed triangle mesh, i.e. the 3D coordinates of the vertices of the mesh inserted as its nodes, finding the closest vertex of p is simply a logarithmic call.

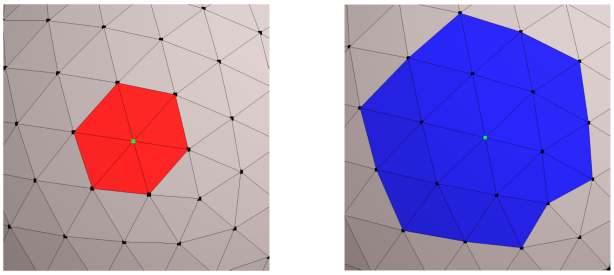
The C++ script knn-cpp is used for the k-d tree implementation.

5.5.1 *Triangle projection - T-neighborhoods*

The next step is to find the accurate projection of $p \in \mathbb{R}^3$ in the surface of the triangle mesh.

We can assume that the actual projection of p is near the closest vertex cv. We can also project a point to the plain defined by a triangle. So, we define the below notions to tackle the projection:

Definition 5.5.1. *Suppose a mesh and two of its vertices v_1, v_2. We define their distance to be the length of the shortest path connecting them (the number of traversed edges).*



(a) T_1 neighborhood. (b) T_2 neighborhood.

Figure 5.7: T_1 and T_2 neighborhoods of the green vertex.

Definition 5.5.2. *Suppose a triangle mesh $M = \{V, E, F\}$ and one of its vertices $v \in V$. We define the T_i neighborhood of v, with $T_i(v) \subseteq F$, as the set of triangles whose vertices have at most i distance with v.*

Examples for T_1 and T_2 neighborhoods are visualized in figure 5.7.

In our projection algorithm, we assume that the closest point of p lies somewhere inside the T_2 neighborhood of the closest vertex cv. This can be false for very stretched out triangles, but since we are using a remeshed mesh (section 4.1), the triangles are close to equilateral and this shouldn't be a problem.

We store the T_2 neighborhood of every vertex as a property on the doubly-connected edge list data structure, so that we calculate it once in the beginning of the script and not waste time on repeated calculations. A fast calculation of the T_1 neighborhood is given in algorithm 4 as a preliminary step.

Algorithm 4 Store T_1 neighborhood of every vertex

Input Mesh $M = \{V, E, F\}$
for $t \in F$ **do**
  $he \leftarrow halfedge(t)$
  $v_1 \leftarrow origin(he)$
  $v_2 \leftarrow destination(he)$
  $v_3 \leftarrow destination(next_halfedge(he))$
  $T_1(v_1).pushback(t)$
  $T_1(v_2).pushback(t)$
  $T_1(v_3).pushback(t)$
end for

Instead of traversing paths from each vertex to create triangles, we use the DCEL to find the three vertices of each triangle and append the triangle to their T_1 neighborhood. This works as a triangle is in the T_1 neighborhood of a vertex if and only if that vertex is part of the triangle. The algorithm has linear time complexity $O(|F|)$.

Going forth to storing the T_2 neighborhood, we introduce an extra limitation: The propagation should stop on sharp edges. We do this to fix degeneracies when projecting on sharp corners, an example is seen in figure 5.8. More explanation is given in section 5.6.2, for now the full algorithm is provided. Therefore algorithm 5 does not store the actual T_2 neighborhood, but this modified version.

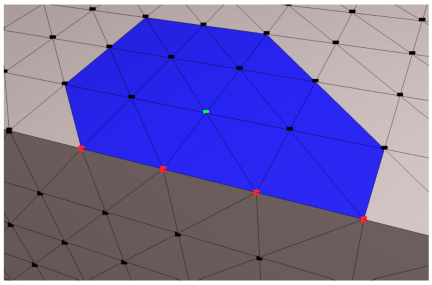


Figure 5.8: Modified T_2 neighborhood of the green vertex, stopping at sharp (red) vertices.

We loop through every triangle, and append it to the neighborhood of its adjacent vertices as well as their neighbors. A property is used to keep track of neighbors that have already been accounted. If any vertex is sharp, the process stops for this vertex, so their neighbors (after the sharp edge) will not be traversed. The T_2 neighborhood of sharp vertices is empty.

A vertex is sharp when it is adjacent to a sharp edge. A property is stored on the vertices in the DCEL keeping track of which are sharp. The neighbors of the vertices are calculated once and also stored in a property, to save from repeated calculations.

If we assume that every vertex can be connected to every other vertex, then the worst case time complexity is $O(|F| \cdot |V|)$, because of searching the neighbors of the vertices of each triangle. However this would result in a non-manifold surface with very bad topology. In general, in a well remeshed mesh, each vertex will have a close to constant and bounded number of neighbors, rendering the algorithm of linear $O(|F|)$ time complexity.

Algorithm 5 Store T_2 neighborhood of every vertex

```
Input Mesh M = {V, E, F}
for t ∈ F do
    initialize is visited property as false ∀v ∈ V
    for v ∈ vertices(t) do                    ▷ Traversed as in algorithm 4
        if v is sharp then
            continue
        end if
        T2(v).pushback(t)
        for n ∈ neighbors(v) do
            if n is visited or is sharp then
                continue
            end if
            T2(n).pushback(t)
            Mark n as visited
        end for
    end for
end for
```

5.5.2 *Projecting a point on a triangle*

First, let's project a vector $\mathbf{a}$ on another non-zero vector $\mathbf{b}$, producing $proj_{\mathbf{b}}\mathbf{a}$, as in figure 5.9. $proj_{\mathbf{b}}\mathbf{a}$ will share the direction of $\mathbf{b}$, so it will be a scalar multiplicative of $\hat{\mathbf{b}} = \mathbf{b}/\|\mathbf{b}\|$. Its magnitude will be

$$\|proj_{\mathbf{b}}\mathbf{a}\| = \cos\theta\|\mathbf{a}\| = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}\|\mathbf{a}\| = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{b}\|}$$

Then:

$$proj_{\mathbf{b}}\mathbf{a} = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{b}\|}\hat{\mathbf{b}} = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{b}\|^2}\mathbf{b}$$

So it can easily be calculated with vector operations as:

$$proj_{\mathbf{b}}\mathbf{a} = \frac{\mathbf{a}\cdot\mathbf{b}}{\mathbf{b}\cdot\mathbf{b}}\mathbf{b} \tag{5.1}$$

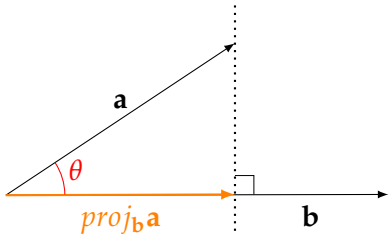


Figure 5.9: Vector projection.

Now let's project a point $p \in \mathbb{R}^3$ on the surface of a triangle. Suppose a triangle defined by 3 points $t_1, t_2, t_3 \in \mathbb{R}^3$, we can project p on the plain of the triangle as seen in figure 5.10.

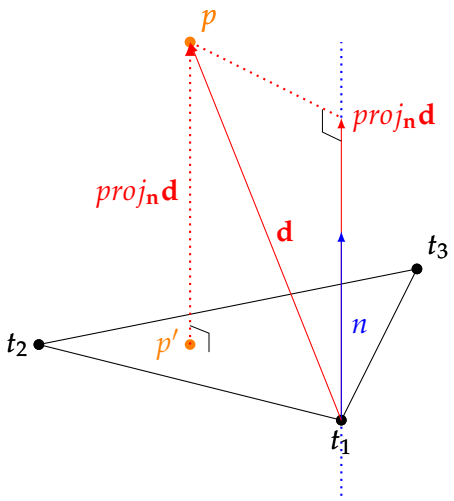


Figure 5.10: Projection of a point on the surface of a triangle.

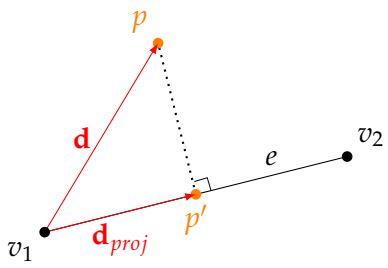


Figure 5.11: Projection of a point on an edge.

$$\begin{aligned} \mathbf{n} &= (t_3 - t_1) \times (t_2 - t_1) \\ \mathbf{d} &= p - t_1 \\ proj_{\mathbf{n}}\mathbf{d} &= \frac{\mathbf{d} \cdot \mathbf{n}}{\mathbf{n} \cdot \mathbf{n}} \mathbf{n} \\ p' &= p - proj_{\mathbf{n}}\mathbf{d} \end{aligned} \tag{5.2}$$

The operations of equations 5.2 are easy to follow, keeping in mind the vector projection of equation 5.1. The projected point p' will lie on the plain defined by the triangle, but will not necessarily be *inside* the triangle. We can check whether it is inside by converting it to barycentric coordinates in the triangle [19].

$$\begin{aligned} \gamma &= \frac{(t_3 - t_1) \times \mathbf{d} \cdot \mathbf{n}}{\mathbf{n} \cdot \mathbf{n}} \\ \beta &= \frac{\mathbf{d} \times (t_2 - t_1) \cdot \mathbf{n}}{\mathbf{n} \cdot \mathbf{n}} \\ \alpha &= 1 - \gamma - \beta \end{aligned} \tag{5.3}$$

p' is inside the triangle if and only if $\alpha, \beta, \gamma \in [0, 1]$.

5.5.3 *Projecting a point on an edge*

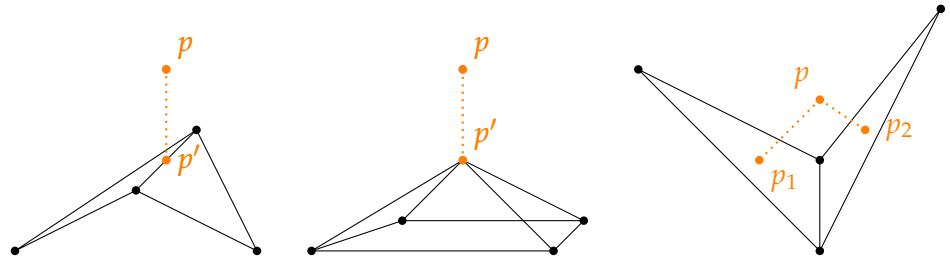
We can easily project a point $p \in \mathbb{R}^3$ on an edge e with endpoints v_1, v_2, following the previous logic, as:

$$\begin{aligned} \mathbf{d} &= p - v_1 \\ \mathbf{w} &= v_2 - v_1 \\ \mathbf{d}_{proj} &= proj_{\mathbf{w}}\mathbf{d} = \frac{\mathbf{d} \cdot \mathbf{w}}{\mathbf{w} \cdot \mathbf{w}} \mathbf{w} \\ p' &= v_1 + \mathbf{d}_{proj} \end{aligned} \tag{5.4}$$

(see also figure 5.11).

If $\mathbf{d}_{proj}$ points in the direction of $\mathbf{w}$ and its magnitude is less than or equal to the magnitude of $\mathbf{w}$, then p' is inside the edge. We can check whether those conditions hold with:

$$\mathbf{d}_{proj} \cdot \mathbf{w} > 0 \quad \text{and} \quad \|\mathbf{d}_{proj}\| \leq \|\mathbf{w}\| \tag{5.5}$$



(a) Projecting on both triangles is not possible, so we project on their common edge

(b) Projecting neither on triangles nor on edges is possible, so we just keep the closest vertex.

(c) On concave surfaces projection can be possible on multiple triangles, then the closer one (in this case p_2) is chosen.

Figure 5.12: Examples of when projecting on edges or keeping the closest vertex will be necessary.

5.5.4 *Combined projection*

When projecting a point $p \in \mathbb{R}^3$ on the surface of a mesh $M = \{V, E, F\}$, our fast and reliable algorithm first finds the closest vertex $cv \in V$ using a k-d tree as explained before. Then using equations 5.2 we project on all triangles of $T_2(cv)$, if multiple projections are possible (the projected points are inside the triangles) we choose the closest one to p. If none are possible, it is still possible that projecting on an edge is. We project on all edges of the $T_2(cv)$, if multiple projections here are possible, again we choose the closest one. If no projections on edges are possible then we just choose cv as the closest point on M.

5.6 SHARP RETENTION

Looking at figure 5.13, we realize that more care has to be taken in the case of sharp edges:

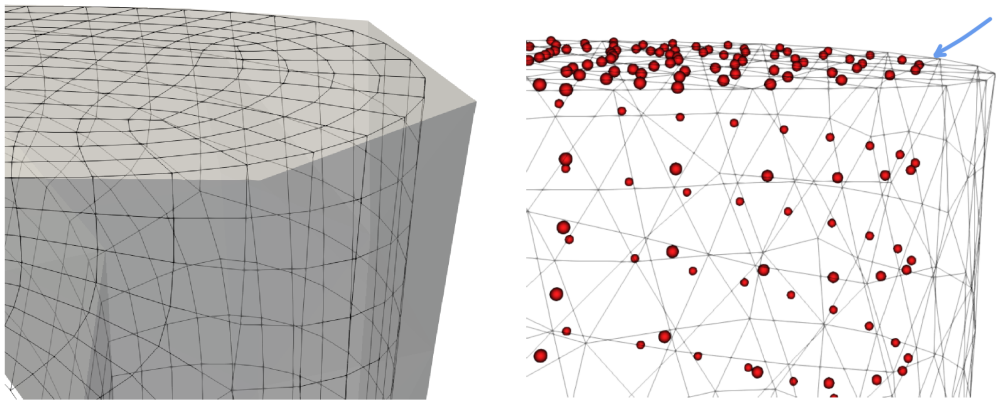
In 5.13a we see how a sharp edge deforms after 3 Catmull-Clark subdivisions. Even though it is marked as sharp and CC takes that into account and keeps it sharp, through the smoothing out it is pushed inside the model. In 5.13b we compare the generated grid points from the CC surface to the remeshed triangle mesh. We see that those grid points are a bit inside the triangle surface also. Of course this happens because the coarse quad mesh does not have enough information.

Two problems arise here:

A. Look at the grid point pointed by the arrow. It is sharp (it stems from a sharp edge), so we would want its projection to be on the sharp edge. However the closest point from its position is the flat surface on the top.

B. We can easily imagine a grid point that should be projected on the surface on the right, but happens to be closer to the surface on the top and gets projected there.

These issues are taken care of in the next sections.



(a) Grey: the coarse quad mesh, Wire: the 3 times cc subdivided mesh.

(b) Wire: the remeshed triangle mesh, Points: the grid points, Arrow: a sharp grid point.

Figure 5.13: The grid points on sharp edges. The sharp edge of this model is the loop around the top. Model: BendTube.

5.6.1 *Projecting on sharp edges*

When creating the grid points (though the 3 CC subdivisions) all edge points keep an inheritance map to the original edge that they are on. If that edge is sharp the grid point is marked as sharp. Sharp grid points should be projected only on sharp edges.

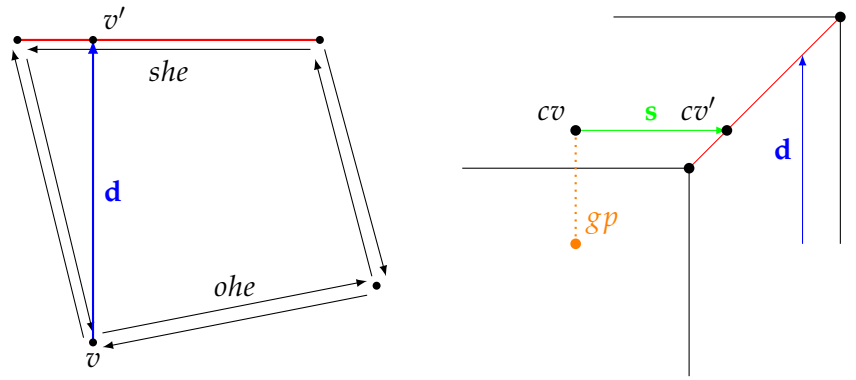
Through the map of section 4.4, mapping a sharp quad edge to the feature line it stems from, we know where we should project those sharp grid points. We project them on all edges of their feature line, if multiple projections are possible, we choose the closest one to the grid point. If none are possible, we cannot just keep the closest vertex as before, as that one is not necessarily on the feature line. What we do instead is keep asking from the k-d tree to give the next closest point, until the one we get is part of the feature line. Then we chose that one as the projection.

In reality a k-d tree supports returning multiple points at once, so when we call it, we ask for the (in our case) 40 closest vertices at once and store them. This way we avoid multiple calls. Those closest vertices will be useful also in the next section.

5.6.2 *Fixing degeneracies*

The degeneracy we are concerned in fixing is that of a grid point near a sharp edge being projected on the wrong side because it's closer.

Let's look at it carefully: The projection on the wrong side happens when the grid points are placed inside the original surface, due to the Catmull-Clark subdivision step - usually below a loop of sharp edges - as illustrated in figure 5.13 - otherwise it is not an issue. Now imagine a quad face, if it has no sharp edges this problem cannot happen. If it has two adjacent sharp edges, the loop is sharpened (see figure 2.9) and the grid points will stay close to the model surface. In general the problem arises *mostly* when there is exactly one sharp edge or two opposite sharp edges. In our testings we encountered problems only with the first case so it is the one



(a) A quad face with exactly one sharp edge. We store the direction **d**.

(b) The grid point gp should project on the right side, but is closer to the top. The two directions **s**, **d** have a large angle which helps us identify the the vertex cv is on the wrong side.

Figure 5.14: Fixing wrong side projection. The red edges are sharp.

we are proposing a solution on. Our solution can be extended on the two opposite sharps case.

The first step is to store some directional information on every quad face.

A. We check for every quad face if it has exactly one sharp edge. This is done easily by traversing the half edges in the DCEL. For quads with exactly one sharp edge, we store its index on a property.

B. If v is the origin of the opposite half edge of the sharp edge (see figure 5.14a) and v' its projection on the sharp edge (using equation 5.1), we store the vector $\mathbf{d} = v' - v$ that points perpendicular to the sharp edge on the plain of the quad.

Also, when creating the grid points we keep track of which quad each one stems from.

Now, when projecting a grid point gp on the triangle mesh, the first thing is to see if the quad it stems from has exactly one sharp. If it does, we read the direction **d**, we find its closest vertex cv from the k-d tree, then we project cv on the sharp edge of the quad, finding cv' and calculate the directional vector $\mathbf{s} = cv' - cv$. For clarity look at figure 5.14b.

If the closest vertex we have found is on the correct side, **s** and **d** should have approximately the same direction, i.e. have a small angle. Otherwise we have projected on the wrong side.

So what we do is, as explained in section 5.6, is instead of 1 we call for the 40 closest vertices. We calculate the vector **s** as explained above and if $30° < angle(\mathbf{s}, \mathbf{d}) < 330°$ - which are some threshold angles signifying that the angle is large - that means that cv is on the wrong size so we try the same process for the next closest vertex. When the condition doesn't hold we stop the process as the vertex is now on the correct side. The angle is calculated with equation 2.2. We also check if cv is a sharp vertex and if it is discarded, as in this case we know we are not projecting on a sharp.

Now is where the modified T_2 neighborhood that stops the propagation on sharp edges is important. Since we found the correct closest vertex, we proceed with the projection of section 5.5.4. But supposing the T_2 neighborhood didn't stop on sharps, it would "spill" again on the wrong side, which is closer to the grid point, therefore projecting on the wrong side.

6 FITTING OF A B-SPLINE

The last part of our pipeline consists of the generation of a spline surface that closely follows the original point cloud. To that end we use the algorithm of [40] that uses the coarse quad mesh that we generated to create the basis functions, and the parametrized data points (see chapter 5) to fit them.

6.1 WHAT IS A SPLINE

So far we have used meshes to represent surfaces. However, the reader has probably noticed that there is a fatal flaw when using them for curved surfaces: they are always piecewise planar, therefore the curve is discretized and not entirely accurate. A mesh can never be smooth. In the case where a simulation is done on a curved object using a mesh representation - imagine for example an aerodynamic analysis - the corners that the edges create skew the result from accurate outcome of the smooth object. We would need, then, a surface representation that can be *actually* smooth.

Another example where a mesh is not useful is if we want to show an object, for example on a CAD software, and be able to zoom in as much as needed, without running on the issue of low resolution. Then a representation that can keep the details of the curve is needed.

Hopefully, mathematics allows us to do this. A sphere for example, instead of using a quad mesh which would only approximate it, can be represented exactly with the equation

$$(x, y, z) \in \mathbb{R}^3 \quad | \quad (x - x_0)^2 + (y - y_0)^2 + (z - z_0)^2 = r^2$$

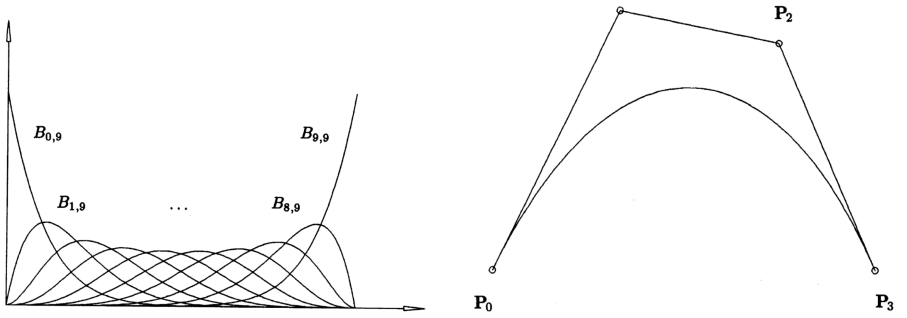
Then, only 4 variables (x_0, y_0, z_0, r) need to be stored, and the fact that we are talking about a sphere, to have perfect detail. This equation of corse works only for a sphere and we cannot use it for an arbitrary shape. The idea then is to use a set of basis functions and some constraints, to fit those on any arbitrary shape.

6.2 BASIS FUNCTIONS

Some basic concepts of the basis functions generally used in splines are showcased here, for more detailed information the reader is redirected to [45], [25].

Let us first look at curves instead of surfaces, as they are easier to showcase. The concepts will then generalize on to surfaces.

First and foremost, to represent a curve using basis functions, a degree n has to be decided. The degree correlates to the amount of basis functions that will be used, and therefore to the detail that they can provide.



(a) The 9th degree Bernstein polynomials [45].

(b) An example of a cubic Bézier curve [45].

Figure 6.1

An easy first choice of basis functions for a curve are those of the Bernstein polynomials, defined as

$$B_{i,n}(u) = \binom{n}{i} u^i (1-u)^{n-i}, \quad \text{for } i = 0, .., n \tag{6.1}$$

Using these, a curve $\mathbf{C}$ can be represented as

$$\mathbf{C}(u) = \sum_{i=0}^{n} B_{i,n}(u)\mathbf{P}_i \qquad 0 \leq u \leq 1 \tag{6.2}$$

This is called a *Bézier curve*. Since the control points $\mathbf{P}_i \in \mathbb{R}^3$ are placed in three dimensions, $\mathbf{C}$ is a curve in three dimensions. The control points $\mathbf{P}_i$ also provide a geometric interpretation, as the curves tries to closely follow them, as seen in figure 6.1.

The Bernstein polynomials can be extended easily to 2 dimensions with variables u, v as:

$$B_{i_1,n_1}(u)B_{i_2,n_2}(v) = \binom{n_1}{i_1}\binom{n_2}{i_2} u^{i_1}(1-u)^{n_1-i_1} v^{i_2}(1-v)^{n_2-i_2} \tag{6.3}$$

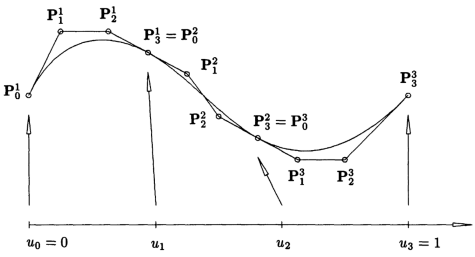
where the degree on the direction of u is n_1 and on the direction of v, n_2. Then a surface would be represented as

$$\mathbf{S}(u,v) = \sum_{i_1=0}^{n_1}\sum_{i_2=0}^{n_2} B_{i_1,n_1}(u)B_{i_2,n_2}(v)\mathbf{P}_{i_1,i_2} \tag{6.4}$$

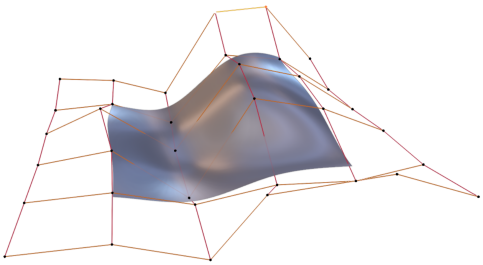
While the above representation of curves and surfaces works well and is used widely, there are a couple of problems with it:

A. A high degree is needed to fit a curve on multiple constraints, or to capture a complex curve/surface.

B. While the control points provide a geometric interpretation, their effect is not local, meaning that moving a control point $\mathbf{P}_i$ changes the entirety of the curve/surface, even very far from it.



(a) C^0 continuous B-spline curve, with piecewise polynomial segments of Bézier form [45].



(b) A (5,5) degree Bézier surface patch.

Figure 6.2

To address these problems, the B-Spline is introduced, which used widely in the industry. The curves/surfaces above are polynomial on their entirety, but in B-Splines we use ones that are *piecewise* polynomial. The notion of *breakpoints* is introduced, in curves they would be parameters $0 < u_1 < u_2 \ldots < u_m = 1$ that break the space $[0, 1]$ into m *segments*. The segments are denoted as $\mathbf{C}_i(u),\ i \in \{1, .., m\}$ and need to be joined with each other with some form of continuity.

Any set of basis functions can be used for every segment, and any form of continuity to define how they are connected. We can use, as in the example of [45], 3 breakpoints u_1, u_2, u_3 and a cubic Bézier form for every segment with control points $\mathbf{P}_i^j$ where j denotes the jth segment. If, then, we enforce that $\mathbf{P}_3^1 = \mathbf{P}_0^2$ and $\mathbf{P}_3^2 = \mathbf{P}_0^3$, we get curves that are C^0 continuous everywhere (see figure 6.2a).

Discussion on C^0, as also other forms of continuity is done in the next section.

A B-Spline surface consist of *segments* of the form

$$\mathbf{S}_j(u, v) = \sum_{i_1=0}^{n_1} \sum_{i_2=0}^{n_2} B_{i_1,n_1}(u) B_{i_2,n_2}(v) \mathbf{P}_{i_1,i_2} \tag{6.5}$$

where $B_{i,n}$ can be *any* basis function of degree n, but later we will use specifically the Bernstein polynomials of 6.1. Simply, it uses Bézier surfaces as segments.

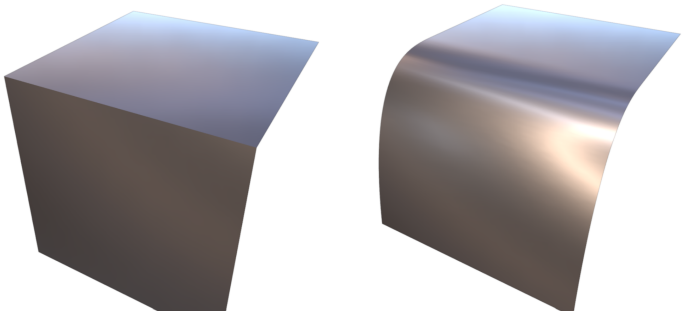
6.3 DIFFERENT LEVELS OF CONTINUITY

Here we deal with the different ways that the segments of a B-Spline can be joined together.

The curve is called C^k continuous at a breakpoint when the kth derivatives on its left and right side are equal. The mentioned above C^0 continuity then simply means that the curve is *continuous* everywhere. A C^1 continuity would imply that the curve is smooth and so on.

As we saw, enforcing C^0 continuity is as simple as equating some control points, but with higher degrees it gets increasingly difficult to enforce. We are not going to deal with those cases, as the algorithm that we use deals with geometric continuity G^1 instead.

Geometric continuity of kth order, symbolized as G^k, is a relaxation of the parametric continuity C^k. Formally, as stated in [25], G^k refers to curves/surfaces that are



(a) A G^0 continuous surface. (b) A G^1 continuous surface.

Figure 6.3: Geometric continuity examples.

k times differentiable with respect to their arc length but not necessarily to their current parametrization. It is more appropriate for dealing with shape rather than speed of traversal. Intuitively, G^1 continuity refers to curves/surfaces where the *tangent* (or normal) vectors are continuous, but the curve/surface is not necessarily smooth [40].

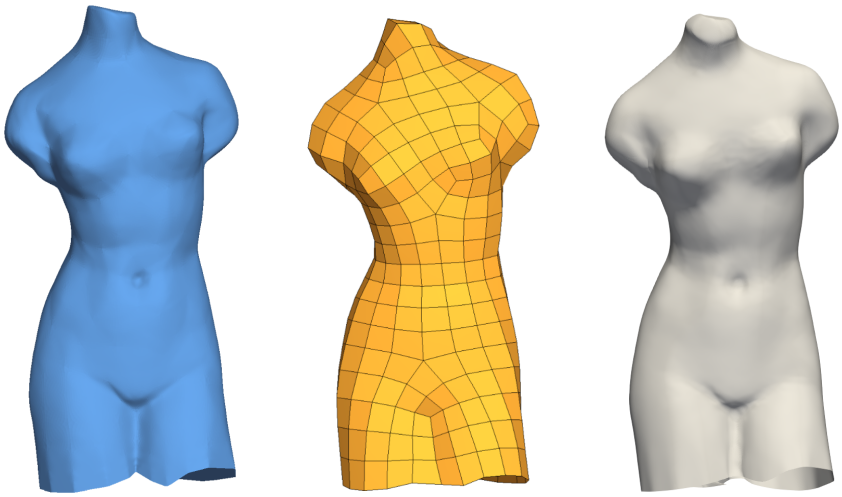
6.4 FITTING A POINT CLOUD ON A QUAD MESH

Marsala et al. [40] use a quad mesh to generate basis functions that are G^1 on the edges of the mesh. A segment corresponds to a face of the quad mesh and is of biquintic Bézier form, i.e. of degree (5,5) using the Bernstein basis functions as in 6.5. There are therefore $(5+1) \times (5+1) = 36$ control points for every face. The G^1 continuity is enforced through the use of masks, which we will not explain in this thesis as it gets highly technical and is of little importance in our work. The mesh has to be strictly quad for the method to work.

After the basis functions have been created on the quad mesh, the fitting of the data takes place. The data points have been already created in the appropriate form in chapter 5. As a recap: we have generated 81 points for every face, which hold u-v parameter information and which closely follow the original point cloud or triangle mesh.

Now for the fitting, for every data point D_i with u-v parameters (u_i, v_i), we want to minimize the distance of the B-Spline surface at (u_i, v_i) from D_i. This is modeled as a least squares problem. Additionally, in [40], a smoothing term with weight λ is introduced in the least squares minimization to deal with possible noise or error on the parametrization stage. The least squares problem can be cast into matrix form and solved explicitly. The solution are the control points of the B-Spline.

Since the segments of the generated B-Spline surface are of degree (5,5) (see figure 6.2b), more detail can be captured by the spline that what is captured by the coarse quad mesh. This is exactly the use of the fact that it is coarse here: no additional information need to captured from the quad mesh, since the spline will be fit on the point cloud. Thus, as we will see on the results, the final surface can



(a) Original triangle mesh. (b) Coarse quad mesh. (c) B-Spline surface.

Figure 6.4: Fitting a spline on the venus [300] sculpture model.

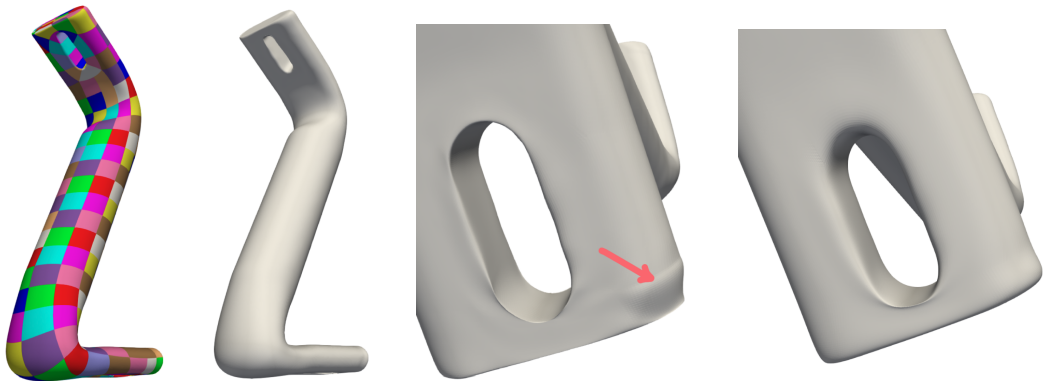
follow closely the original point cloud, even though the quad mesh had lost most detail.

6.5 RESULTS

We test how the above process works with the meshes our algorithm generates. Below an organic example of a statue and a mechanical object are shown, the resulting surfaces being very close to the surfaces of the originals. In 6.5, the degeneracy of projecting on the wrong edge and our proposed fix - section 5.6.2 - are showcased.

In 6.6, artifacts that are creating from the lack of identification of almost sharp edges are shown to carry over to the B-Spline surface.

One thing to note is that in its current form, the algorithm of [40] does not allow for sharp edges. That is because the joints of the spline segments are always of G^1 continuity and no option for allowing G^0 is allowed. Therefore the edges will always be smooth. This is reflected on our mechanical example of figure 6.5 and also in the example of 7.3.



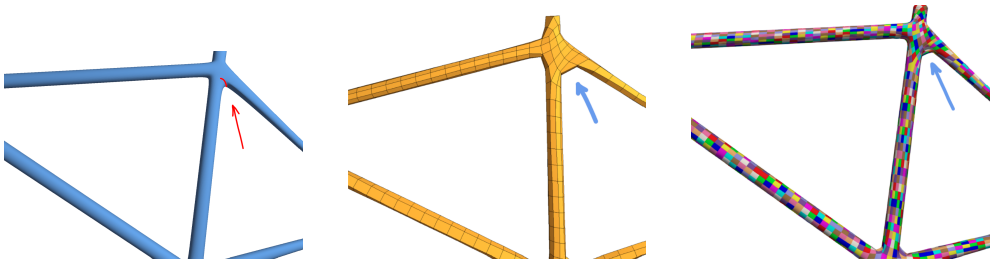
(a) B-Spline with patches.

(b) B-Spline solid color.

(c) The degeneracy that projecting a grid point on the wrong face produces.

(d) Our proposed fix of such degeneracies.

Figure 6.5: A mechanical spline example (BendTube). The quad meshes can be seen in figure 4.15.



(a) The original triangle mesh. The almost sharp line marked with blue was not identified.

(b) The quad mesh. Since the almost sharp was not identified, an edge was not used, thus an artifact appeared.

(c) The artifact carries over to the spline surface.

Figure 6.6: Model of the author's bike.

Part III

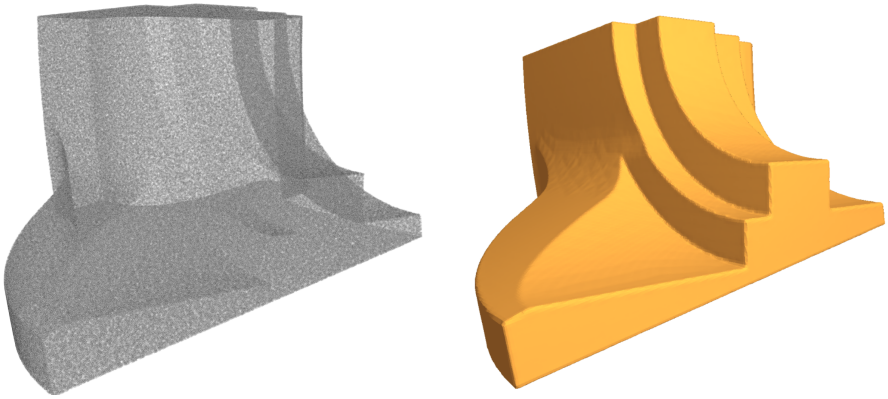
FULL SHOWCASE

Every step of the pipeline is showcased with a figure, using an organic and a mechanical object. In the end a collage of coarse quad meshes generated with our algorithms is presented.

7 FULL SHOWCASE

An organic and a mechanical object are used for the full showcase. The reader will notice that in the organic (Stanford bunny, figure 7.2) model no feature lines are plotted. This is because there are no sharp edges and no feature lines were identified. As explained in section 4.1.1.2 and in figure 4.9, in this case the cross field will align with the principal curvature of the mesh.

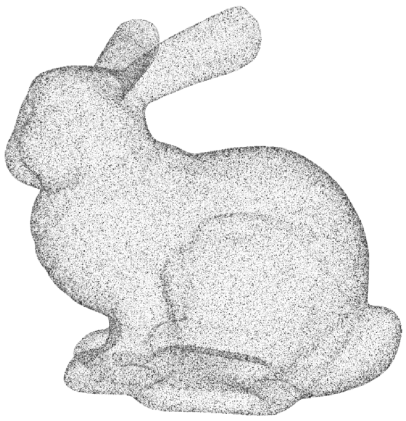
Additionally, the reader will notice that on the mechanical model (fandisk, figure 7.3) the pipeline starts immediately from a triangle mesh and not a pointcloud. That is because if we start from a point cloud (as shown in figure 7.1), the Poisson algorithm looses the sharp edges. Since we want to showcase our method's sharp edge support, we choose in this case to start from the triangle mesh.



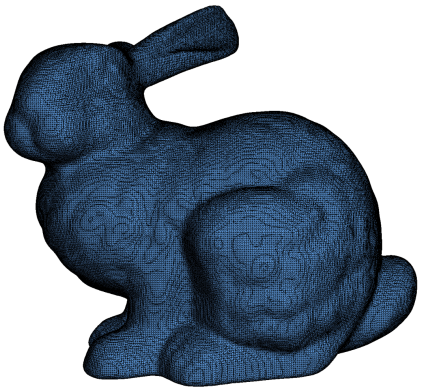
(a) Point cloud. (b) Poisson reconstruction.

Figure 7.1: The Poisson reconstruction smoothens the sharp edges, so we start from a triangle mesh to showcase this example. Fandisk [libigl] model.

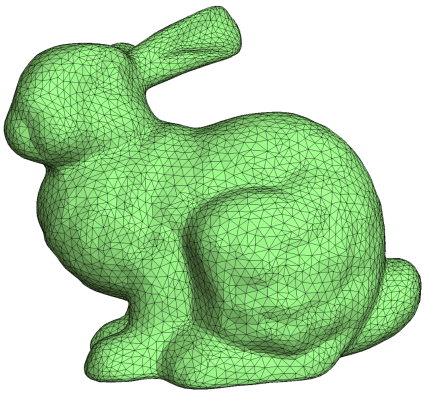
Lastly, as explained in section 6.5, the B-Spline surface itself currently does not support sharp edges, as G^0 joints on the basis functions are not an option. On future work these problems can be tackled: using a surface reconstruction method that would not smooth out the sharps - [42] seems promising - and allowing for G^0 on certain edges.



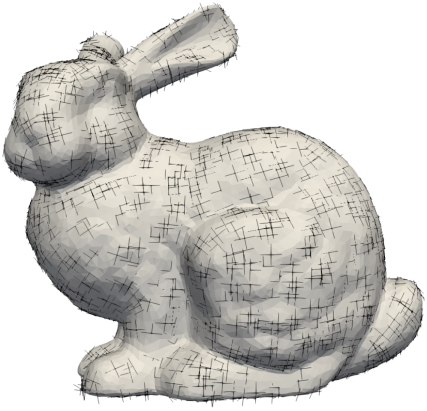
(a) Original point cloud.



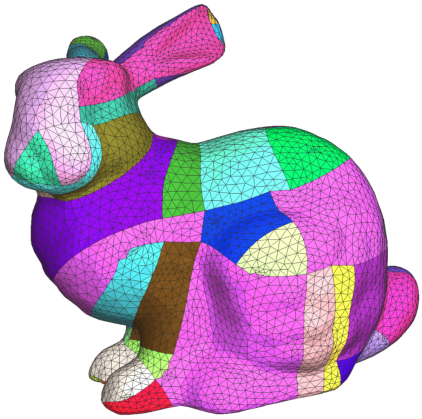
(b) Poisson reconstruction.



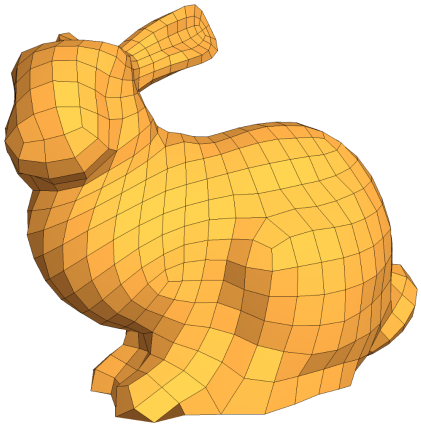
(c) Remeshing.



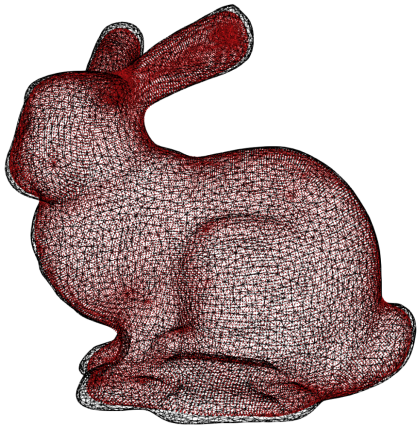
(d) Cross field.



(e) Patches.



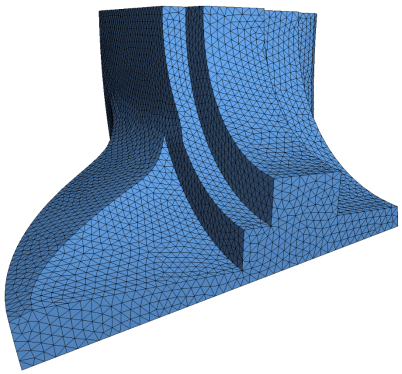
(f) Coarse quad mesh.



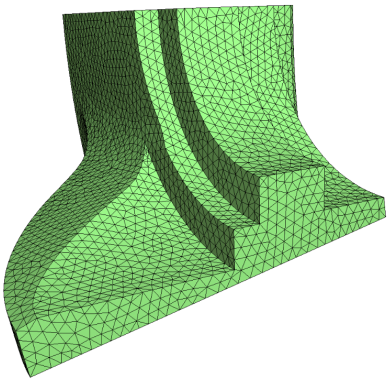
(g) Gridpoints and remeshed triangle mesh - before the projection.

(h) B-Spline surface.

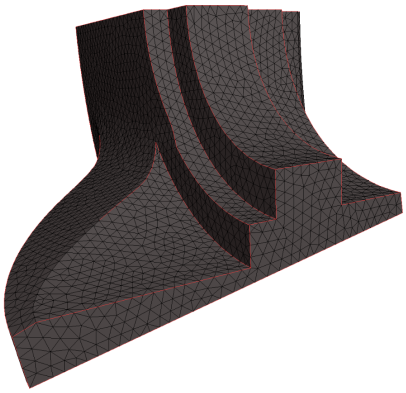
Figure 7.2: The Stanford bunny full pipeline.



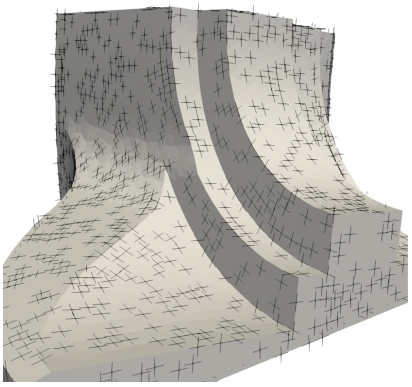
(a) Original triangle mesh.



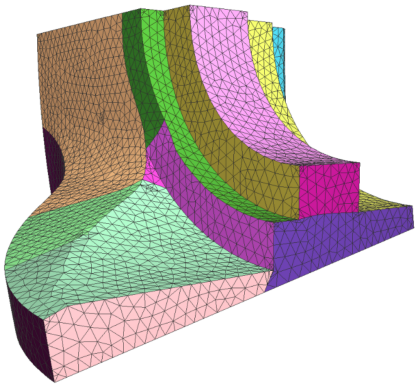
(b) Remeshing.



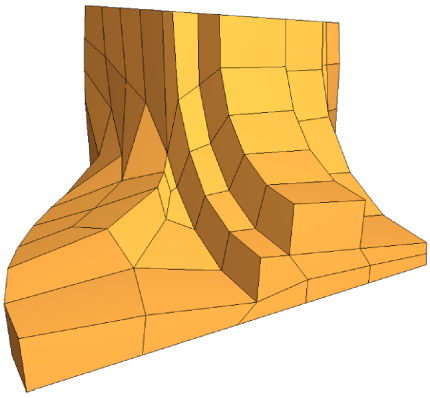
(c) Feature lines.



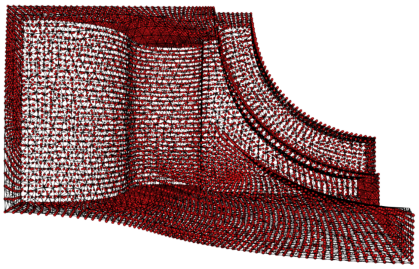
(d) Cross field.



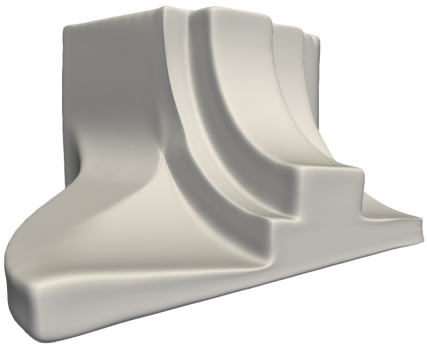
(e) Patches.



(f) Coarse quad mesh.



(g) Gridpoints and remeshed triangle mesh - before the projection.



(h) B-Spline surface.

Figure 7.3: Fandisk model pipeline from triangle mesh.

Figure 7.4: Coarse quad mesh collage.

Part IV

APPENDIX

A

APPENDIX

A.1 CONVOLUTION

Definition A.1.1. *Suppose two functions* $f : \mathbb{R} \to \mathbb{R}$ *and* $g : \mathbb{R} \to \mathbb{R}$*, their convolution is a function defined as:*

$$(f * g)(t) := \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$

Or equivalently $(f * g)(t) = \int_{-\infty}^{\infty} f(t-\tau)g(\tau)d\tau$.

Intuitively, one of the two functions can be seen as a filter applied to the second one: the filter is first flipped and then offset by t, then the area under their product is returned. Figure A.1 shows the process for two square functions.

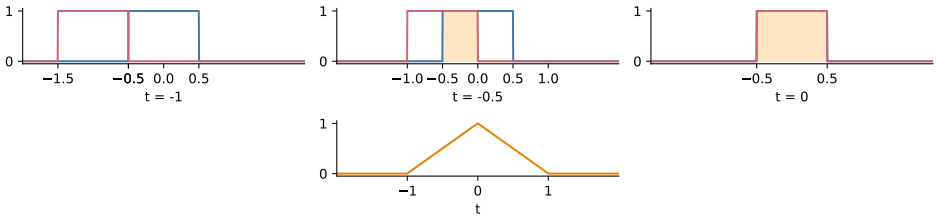


Figure A.1: Visualization of the convolution between two square functions. The blue function is fixed and the red is moving, on the top three timestamps are seen and the area of the overlap is shaded with orange. On the bottom the convolution function for all offsets is graphed.

A convolution may be used to smooth a function, e.g. using a gaussian as a filter. Every point t is influenced by its neighbors with diminishing amplitudes for bigger distances, thus smoothing.

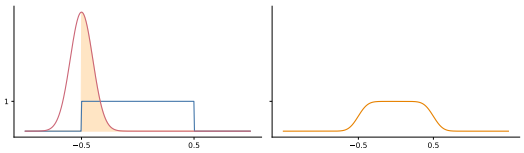


Figure A.2: The square function (left) is convolved with a gaussian filter to produce a smoothed outcome (right).

Convolution can be defined in three dimensions as:

Definition A.1.2. *Given two functions* $f : \mathbb{R}^3 \to \mathbb{R}$, $g : \mathbb{R}^3 \to \mathbb{R}$*, we call their convolution the function:*

$$(f * g)(\mathbf{p}) := \iiint_{\mathbb{R}^3} f(\mathbf{q})g(\mathbf{p}-\mathbf{q})$$

where $\mathbf{p}, \mathbf{q}$ *are points in* $\mathbb{R}^3$.

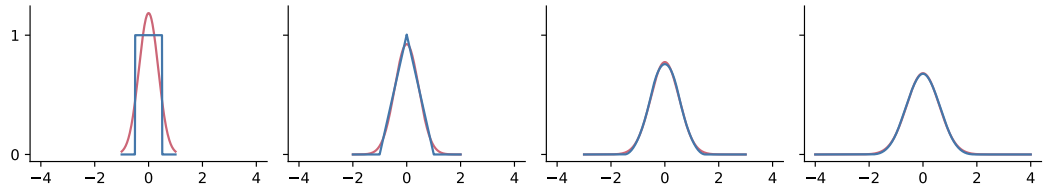


Figure A.3: With blue: zero to three repeated convolutions of a square pulse to itself. With red: the closest gaussian. At two convolutions the outcome already mimics closely the gaussian, but the approximation doesn't taper off, it is exactly zero after a point. With every step the variance increases.

Repeated convolutions of a square pulse with itself are used to approximate a gaussian: the limit of self convolutions tends to a gaussian, with growing variance. This is a consequence of the *Central Limit Theorem* and the fact that summing independent random variables corresponds convolving their density functions.

BIBLIOGRAPHY

[1] Jean-Philippe Bauchet and Florent Lafarge. "Kinetic Shape Reconstruction." In: *ACM Trans. Graph.* 39.5 (June 2020). ISSN: 0730-0301. DOI: 10.1145/3376918. URL: https://doi.org/10.1145/3376918.

[2] Jon Louis Bentley. "Multidimensional binary search trees used for associative searching." In: *Commun. ACM* 18.9 (Sept. 1975), pp. 509–517. ISSN: 0001-0782. DOI: 10.1145/361002.361007. URL: https://doi.org/10.1145/361002.361007.

[3] Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. "Computational Geometry." In: *Computational Geometry: Algorithms and Applications*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 1–17. ISBN: 978-3-540-77974-2. DOI: 10.1007/978-3-540-77974-2_1. URL: https://doi.org/10.1007/978-3-540-77974-2_1.

[4] Matthew Berger, Andrea Tagliasacchi, Lee M. Seversky, Pierre Alliez, Gaël Guennebaud, Joshua A. Levine, Andrei Sharf, and Claudio T. Silva. "A Survey of Surface Reconstruction from Point Clouds." In: *Computer Graphics Forum* 36.1 (Mar. 2016), pp. 301–329. DOI: 10.1111/cgf.12802. URL: https://doi.org/10.1111/cgf.12802.

[5] F. Bernardini, J. Mittleman, H. Rushmeier, C. Silva, and G. Taubin. "The ball-pivoting algorithm for surface reconstruction." In: *IEEE Transactions on Visualization and Computer Graphics* 5.4 (1999), pp. 349–359. DOI: 10.1109/2945.817351.

[6] Alberto Biliotti, Cesare Bracco, Carlotta Giannelli, and Krunal Raval. "A data-driven anisotropic refinement strategy for LR B-spline surface approximation of 3D scattered point clouds." In: *Computer-Aided Design* 199 (2026), p. 104134. ISSN: 0010-4485. DOI: https://doi.org/10.1016/j.cad.2026.104134. URL: https://www.sciencedirect.com/science/article/pii/S0010448526001041.

[7] David Bommes, Marcel Campen, Hans-Christian Ebke, Pierre Alliez, and Leif Kobbelt. "Integer-grid maps for reliable quad meshing." In: *ACM Trans. Graph.* 32.4 (July 2013). ISSN: 0730-0301. DOI: 10.1145/2461912.2462014. URL: https://doi.org/10.1145/2461912.2462014.

[8] David Bommes, Bruno Lévy, Nico Pietroni, Enrico Puppo, Claudio Silva, Marco Tarini, and Denis Zorin. "Quad-Mesh Generation and Processing: A Survey." In: *Comput. Graph. Forum* 32.6 (Sept. 2013), pp. 51–76. ISSN: 0167-7055. DOI: 10.1111/cgf.12014. URL: https://doi.org/10.1111/cgf.12014.

[9] David Bommes, Henrik Zimmer, and Leif Kobbelt. "Mixed-integer quadrangulation." In: *ACM SIGGRAPH 2009 Papers*. SIGGRAPH '09. New Orleans, Louisiana: Association for Computing Machinery, 2009. ISBN: 9781605587264. DOI: 10.1145/1576246.1531383. URL: https://doi.org/10.1145/1576246.1531383.

[10] E. Catmull and J. Clark. "Recursively generated B-spline surfaces on arbitrary topological meshes." In: *Computer-Aided Design* 10.6 (1978), pp. 350–355. ISSN: 0010-4485. DOI: https://doi.org/10.1016/0010-4485(78)90110-0. URL: https://www.sciencedirect.com/science/article/pii/0010448578901100.

[11] Frédéric Cazals and Joachim Giesen. "Delaunay Triangulation Based Surface Reconstruction." In: *Effective Computational Geometry for Curves and Surfaces*. Ed. by Jean-Daniel Boissonnat and Monique Teillaud. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 231–276. ISBN: 978-3-540-33259-6. DOI: 10.1007/978-3-540-33259-6_6. URL: https://doi.org/10.1007/978-3-540-33259-6_6.

[12] Qi Chen and Hartmut Prautzsch. "General triangular midpoint subdivision." In: *Computer Aided Geometric Design* 31.7 (2014). Recent Trends in Theoretical and Applied Geometry, pp. 475–485. ISSN: 0167-8396. DOI: https://doi.org/10.1016/j.cagd.2014.06.002. URL: https://www.sciencedirect.com/science/article/pii/S0167839614000600.

[13] Paolo Cignoni, Marco Callieri, Massimiliano Corsini, Matteo Dellepiane, Fabio Ganovelli, and Guido Ranzuglia. "MeshLab: an Open-Source Mesh Processing Tool." In: *Eurographics Italian Chapter Conference*. Ed. by Vittorio Scarano, Rosario De Chiara, and Ugo Erra. The Eurographics Association, 2008. ISBN: 978-3-905673-68-5. DOI: 10.2312/LocalChapterEvents/ItalChap/ItalianChapConf2008/129-136.

[14] David Cohen-Steiner and Frank Da. "A greedy Delaunay-based surface reconstruction algorithm." In: *The Visual Computer* 20.1 (Apr. 2004), pp. 4–16. DOI: 10.1007/s00371-003-0217-z. URL: https://doi.org/10.1007/s00371-003-0217-z.

[15] Tony DeRose, Michael Kass, and Tien Truong. "Subdivision Surfaces in Character Animation." In: *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*. 1st ed. New York, NY, USA: Association for Computing Machinery, 2023. ISBN: 9798400708978. URL: https://doi.org/10.1145/3596711.3596795.

[16] Olga Diamanti, Amir Vaxman, Daniele Panozzo, and Olga Sorkine-Hornung. "Designing N-PolyVector Fields with Complex Polynomials." In: *Computer Graphics Forum* 33.5 (2014), pp. 1–11. DOI: https://doi.org/10.1111/cgf.12426. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.12426. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.12426.

[17] E. W. Dijkstra. "A note on two problems in connexion with graphs." In: *Numerische Mathematik* 1.1 (Dec. 1959), pp. 269–271. DOI: 10.1007/bf01386390. URL: https://doi.org/10.1007/bf01386390.

[18] J.M. Dischler and D. Ghazanfarpour. "A survey of 3D texturing." In: *Computers & Graphics* 25.1 (2001). Shape Blending, pp. 135–151. ISSN: 0097-8493. DOI: https://doi.org/10.1016/S0097-8493(00)00113-8. URL: https://www.sciencedirect.com/science/article/pii/S0097849300001138.

[19] Leo Dorst, Daniel Fontijne, and Stephen Mann. "Chapter 11 - The Homogeneous Model." In: *Geometric Algebra for Computer Science (Revised Edition)*. The Morgan Kaufmann Series in Computer Graphics. Boston: Morgan Kaufmann, 2009, pp. 271–326. ISBN: 978-0-12-374942-0. DOI: https://doi.org/10.1016/B978-0-12-374942-0.00016-6. URL: https://www.sciencedirect.com/science/article/pii/B9780123749420000166.

[20] Hans-Christian Ebke. "Parametrization Based Quad Meshing in Practical Applications." Veröffentlicht auf dem Publikationsserver der RWTH Aachen University; Dissertation, RWTH Aachen University, 2017. Dissertation. Aachen: RWTH Aachen University, 2017, 1 Online–Ressource (xi, 187 Seiten) : Illustrationen, Diagramme. DOI: 10.18154/RWTH-2017-07479. URL: https://publications.rwth-aachen.de/record/698105.

[21] Herbert Edelsbrunner and J. Harer. *Computational topology: An introduction*. American Mathematical Society, 2022.

[22] C. a. J. Fletcher. *Computational Galerkin methods*. Jan. 1984. DOI: 10.1007/978-3-642-85949-6. URL: https://doi.org/10.1007/978-3-642-85949-6.

[23] Michael S. Floater and Kai Hormann. "Surface Parameterization: a Tutorial and Survey." In: *Advances in Multiresolution for Geometric Modelling*. Ed. by Neil A. Dodgson, Michael S. Floater, and Malcolm A. Sabin. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 157–186. ISBN: 978-3-540-26808-6.

[24] Qiqi Gao and Yasushi Yamaguchi. "Extraction of coherent and smooth feature lines from meshes with fine details." In: *Computers & Graphics* 82 (2019), pp. 222–231. ISSN: 0097-8493. DOI: https://doi.org/10.1016/j.cag.2019.05.020. URL: https://www.sciencedirect.com/science/article/pii/S0097849319300810.

[25] Farin Gerald. *Curves and surfaces for Computer-Aided geometric design*. Jan. 1993. DOI: 10.1016/c2009-0-22351-8. URL: https://doi.org/10.1016/c2009-0-22351-8.

[26] Anne Greenbaum. "2. Some Iteration Methods." In: *Iterative Methods for Solving Linear Systems*. Society for Industrial and Applied Mathematics, 1997, pp. 25–47. DOI: 10.1137/1.9781611970937.ch2. eprint: https://epubs.siam.org/doi/pdf/10.1137/1.9781611970937.ch2. URL: https://epubs.siam.org/doi/abs/10.1137/1.9781611970937.ch2.

[27] Eitan Grinspun, Petr Krysl, and Peter Schröder. "CHARMS: a simple framework for adaptive simulation." In: *ACM Trans. Graph.* 21.3 (July 2002), pp. 281–290. ISSN: 0730-0301. DOI: 10.1145/566654.566578. URL: https://doi.org/10.1145/566654.566578.

[28] Paul S. Heckbert. "Survey of Texture Mapping." In: *IEEE Computer Graphics and Applications* 6.11 (1986), pp. 56–67. DOI: 10.1109/MCG.1986.276672.

[29] Hugues Hoppe, Tony DeRose, Tom Duchamp, Mark Halstead, Hubert Jin, John McDonald, Jean Schweitzer, and Werner Stuetzle. "Piecewise smooth surface reconstruction." In: *Proceedings of the 21st Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '94. New York, NY, USA: Association for Computing Machinery, 1994, pp. 295–302. ISBN: 0897916670. DOI: 10.1145/192161.192233. URL: https://doi.org/10.1145/192161.192233.

[30] Hugues Hoppe, Tony DeRose, Tom Duchamp, John McDonald, and Werner Stuetzle. "Surface reconstruction from unorganized points." In: *Proceedings of the 19th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '92. New York, NY, USA: Association for Computing Machinery, 1992, pp. 71–78. ISBN: 0897914791. DOI: 10.1145/133994.134011. URL: https://doi.org/10.1145/133994.134011.

[31] Hugues Hoppe, Tony DeRose, Tom Duchamp, John McDonald, and Werner Stuetzle. "Mesh optimization." In: *Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '93. Anaheim, CA: Association for Computing Machinery, 1993, pp. 19–26. ISBN: 0897916018. DOI: 10.1145/166117.166119. URL: https://doi.org/10.1145/166117.166119.

[32] Kai Hormann, Konrad Polthier, and Alia Sheffer. "Mesh parameterization: theory and practice." In: *ACM SIGGRAPH ASIA 2008 Courses*. SIGGRAPH Asia '08. Singapore: Association for Computing Machinery, 2008. ISBN: 9781450379243. DOI: 10.1145/1508044.1508091. URL: https://doi.org/10.1145/1508044.1508091.

[33] Thomas Hughes. *Finite element method: Linear static and dynamic finite element analysis*. Dover Publications, 2012.

[34] Wenzel Jakob, Marco Tarini, Daniele Panozzo, and Olga Sorkine-Hornung. "Instant field-aligned meshes." In: *ACM Trans. Graph.* 34.6 (Nov. 2015). ISSN: 0730-0301. DOI: 10.1145/2816795.2818078. URL: https://doi.org/10.1145/2816795.2818078.

[35] Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. "Poisson surface reconstruction." In: *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*. SGP '06. Cagliari, Sardinia, Italy: Eurographics Association, 2006, pp. 61–70. ISBN: 3905673363.

[36] Michael Kazhdan and Hugues Hoppe. "Screened poisson surface reconstruction." In: *ACM Trans. Graph.* 32.3 (July 2013). ISSN: 0730-0301. DOI: 10.1145/2487228.2487237. URL: https://doi.org/10.1145/2487228.2487237.

[37] Bruno Levy, Sylvain Petitjean, Nicolas Ray, and Jérome Maillot. "Least squares conformal maps for automatic texture atlas generation." In: *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*. 1st ed. New York, NY, USA: Association for Computing Machinery, 2023. ISBN: 9798400708978. URL: https://doi.org/10.1145/3596711.3596734.

[38] Er Li, Bruno Lévy, Xiaopeng Zhang, Wujun Che, Weiming Dong, and Jean-Claude Paul. "Meshless quadrangulation by global parameterization." In: *Computers & Graphics* 35.5 (2011), pp. 992–1000. ISSN: 0097-8493. DOI: https://doi.org/10.1016/j.cag.2011.05.003. URL: https://www.sciencedirect.com/science/article/pii/S009784931100121X.

[39] M. Lyon, M. Campen, and L. Kobbelt. "Quad Layouts via Constrained T-Mesh Quantization." In: *Computer Graphics Forum* 40.2 (2021), pp. 305–314. DOI: https://doi.org/10.1111/cgf.142634. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.142634. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.142634.

[40] Michelangelo Marsala, Angelos Mantzaflaris, and Bernard Mourrain. "G1 Spline Functions for Point Cloud Fitting." In: *Applied Mathematics and Computation* 460 (Feb. 2023), p. 128279. DOI: 10.1016/j.amc.2023.128279. URL: https://hal.science/hal-04002985.

[41] D.E. Muller and F.P. Preparata. "Finding the intersection of two convex polyhedra." In: *Theoretical Computer Science* 7.2 (1978), pp. 217–236. ISSN: 0304-3975. DOI: https://doi.org/10.1016/0304-3975(78)90051-8. URL: https://www.sciencedirect.com/science/article/pii/0304397578900518.

[42] A. C. Öztireli, G. Guennebaud, and M. Gross. "Feature Preserving Point Set Surfaces based on Non-Linear Kernel Regression." In: *Computer Graphics Forum* 28.2 (2009), pp. 493–501. DOI: https://doi.org/10.1111/j.1467-8659.2009.01388.x. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1467-8659.2009.01388.x. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1467-8659.2009.01388.x.

[43] Jonathan Palacios and Eugene Zhang. "Rotational symmetry field design on surfaces." In: *ACM Trans. Graph.* 26.3 (July 2007), 55–es. ISSN: 0730-0301. DOI: 10.1145/1276377.1276446. URL: https://doi.org/10.1145/1276377.1276446.

[44] Daniele Panozzo, Yaron Lipman, Enrico Puppo, and Denis Zorin. "Fields on symmetric surfaces." In: *ACM Trans. Graph.* 31.4 (July 2012). ISSN: 0730-0301. DOI: 10.1145/2185520.2185607. URL: https://doi.org/10.1145/2185520.2185607.

[45] Le Piegl and Wayne Tiller. *The NURBS book*. 2012.

[46] Nico Pietroni, Stefano Nuvoli, Thomas Alderighi, Paolo Cignoni, and Marco Tarini. "Reliable feature-line driven quad-remeshing." In: *ACM Trans. Graph.* 40.4 (July 2021). ISSN: 0730-0301. DOI: 10.1145/3450626.3459941. URL: https://doi.org/10.1145/3450626.3459941.

[47] Matthew N. O. Sadiku. "Numerical Techniques in Electromagnetics, Second edition." In: Weighted Residual Method. July 2000. Chap. 4.6. URL: http://ci.nii.ac.jp/ncid/BA48062805.

[48] R. Schnabel, R. Wahl, and R. Klein. "Efficient RANSAC for Point-Cloud Shape Detection." In: *Computer Graphics Forum* 26.2 (2007), pp. 214–226. DOI: https://doi.org/10.1111/j.1467-8659.2007.01016.x. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1467-8659.2007.01016.x. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1467-8659.2007.01016.x.

[49] Rajiv Shekhar, Elias Fayyad, Roni Yagel, and J. Fredrick Cornhill. "Octree-based decimation of marching cubes surfaces." In: *Proceedings of Seventh Annual IEEE Visualization '96* (1996), pp. 335–342. URL: https://api.semanticscholar.org/CorpusID:5366964.

[50] Kenshi Takayama, Daniele Panozzo, and Olga Sorkine-Hornung. "Pattern-Based Quadrangulation for N-Sided Patches." In: *Computer Graphics Forum* 33.5 (2014), pp. 177–184. DOI: https://doi.org/10.1111/cgf.12443. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.12443. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.12443.

[51] Marco Tarini. "Closed-form quadrangulation of n-sided patches." In: *Computers & Graphics* 107 (2022), pp. 60–65. ISSN: 0097-8493. DOI: https://doi.org/10.1016/j.cag.2022.06.015. URL: https://www.sciencedirect.com/science/article/pii/S0097849322001170.

[52] Marco Tarini, Enrico Puppo, Daniele Panozzo, Nico Pietroni, and Paolo Cignoni. *Simple Quad Domains for Field Aligned Mesh Parametrization*. 2011. URL: http://vcg.isti.cnr.it/Publications/2011/TPPPC11.

[53] Francesco Usai, Marco Livesu, Enrico Puppo, Marco Tarini, and Riccardo Scateni. "Extraction of the Quad Layout of a Triangle Mesh Guided by Its Curve Skeleton." In: *ACM Trans. Graph.* 35.1 (Dec. 2016). ISSN: 0730-0301. DOI: 10.1145/2809785. URL: https://doi.org/10.1145/2809785.

[54] Robert J. Vanderbei. "Integer Programming." In: *Linear Programming: Foundations and Extensions*. Cham: Springer International Publishing, 2020, pp. 389–414. ISBN: 978-3-030-39415-8. DOI: 10.1007/978-3-030-39415-8_23. URL: https://doi.org/10.1007/978-3-030-39415-8_23.

[55] R. Westermann, L. Kobbelt, and T. Ertl. "Real-time exploration of regular volume data by adaptive reconstruction of isosurfaces." In: *The Visual Computer* 15.2 (Apr. 1999), pp. 100–111. DOI: 10.1007/s003710050165. URL: https://doi.org/10.1007/s003710050165.

[56] Jane Wilhelms and Allen Van Gelder. "Octrees for faster isosurface generation." In: *ACM Trans. Graph.* 11.3 (July 1992), pp. 201–227. ISSN: 0730-0301. DOI: 10.1145/130881.130882. URL: https://doi.org/10.1145/130881.130882.

This document was typeset using the typographical look-and-feel `classicthesis` developed by André Miede and Ivo Pletikosić:

https://www.ctan.org/pkg/classicthesis

Final Version as of October 23, 2026 (classicthesis v4.8).